

JavaScript — 0 dan Expertgacha (O'zbek tilida)

Bu — JavaScript'ni **mutlaqo noldan professional darajagacha** o'rgatadigan to'liq qo'llanma. Hech qanday oldingi dasturlash tajribasi talab qilinmaydi: birinchi `console.log` dan tortib closure, `this`, async, prototiplar, design pattern va TypeScript ko'prigigacha.

Har bir modul: **nazariya + Why (nega shunday) + 20 ta masala (yechimi bilan)**.

Qanday ishlatish kerak

1. Modullarni **tartib bilan** o'qing (0 → 30). Har biri oldingisiga tayanadi, sakramang.
2. Har modul oxiridagi **20 ta masalani avval o'zingiz yeching**, keyin yashirin "► Yechimlar" bo'limiga qarang. Faqat o'qib ketsangiz — bilim o'rnashmaydi.
3. Kodni **brauzer konsolida** (F12 → Console) yoki **Node.js**da yozib sinab ko'ring. Dasturlash — yozish bilan o'rganiladi.
4. "Keng tarqalgan xatolar" va `Why` bloklarini e'tibordan qochirmang — ko'pchilik aynan shu joylarda qoqiladi.

Talablar (prerequisites)

Kerak	Daraja
Kompyuter, brauzer (Chrome/Firefox)	Asoslar
HTML/CSS	Shart emas (3-qism — DOM uchun ozgina foydali)
Matn muharriri (VS Code tavsiya etiladi)	Asoslar
Oldingi dasturlash tajribasi	Shart emas — noldan boshlaymiz

To'liq yo'l xaritasi (roadmap)

I bosqich — Asoslar

Qism	Modullar	Mavzular
1-QISM — Asoslar	0–4	Kirish, o'zgaruvchilar va turlar, operatorlar, shartlar (<code>if / switch</code>), sikllar
2-QISM — Strukturalar	5–8	Funksiyalar, massivlar va metodlar, obyektlar, String/Number metodlari

II bosqich — Brauzer va asinxronlik

Qism	Modullar	Mavzular
3-QISM — Brauzer (DOM)	9–12	DOM, events, forms va validation, localStorage
4-QISM — Asinxron JS	13–16	Callbacks va Event Loop, Promises, <code>async / await</code> , Fetch API va REST

III bosqich — Chuqur JS

Qism	Modullar	Mavzular
5-QISM (1-bo'lim) — OOP yadrosi	17–20	Scope/closure/hoisting, <code>this / call / apply / bind</code> , prototiplar, classlar
5-QISM (2-bo'lim)	21–23	ES6+ va modullar, error handling, regular expressions

IV bosqich — Expert

Qism	Modullar	Mavzular
6-QISM (1-bo'lim) – Expert	24–27	Functional programming, iterators/generators/Symbols, Proxy/Reflect, performance va memory
6-QISM (2-bo'lim) – Yakun	28–30	Design patterns, bundlers va build tools, TypeScript'ga ko'prik

Jami: **31 modul**, har birida **20 masala** — ~620 amaliy mashq.

Boshlashdan oldin

Kodni ishga tushirishning ikki oson yo'li:

1. **Brauzer konsoli (eng tez):** brauzerni och → **F12** → **Console** bo'limi → kod yoz, Enter bos.

```
console.log("Salom, dunyo!");
```



2. **Node.js (terminal):** nodejs.org dan o'rnat, `dastur.js` fayl yarat, `node dastur.js` bilan ishga tushir.

Tayyor bo'lsang — **1-QISM: Asoslar** → dan boshla.

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 [README](#) / [Mundarija](#) · [Keyingi: 2-qism — Strukturalar](#) →

1-QISM: ASOSLAR Boshlovchilar uchun to'liq qo'llanma. Har bir moduldan keyin **20 ta masala** (yechimi bilan) bor. Masalalarni avval **o'zingiz yechib ko'ring**, keyin yechimga qarang — yechim ► [Yechim](#) ostida yashiringan.

To'liq yo'l xaritasi (0 → Expert)

Bu qo'llanma 6 qismdan iborat. Sen hozir **1-qismni** o'qiyapsan.

1-QISM — ASOSLAR (*shu fayl*) - 0-modul: Kirish — JS nima va qanday ishlaydi - 1-modul: O'zgaruvchilar va ma'lumot turlari - 2-modul: Operatorlar - 3-modul: Shartli operatorlar (`if / else` , `switch`) - 4-modul: Sikllar (`for` , `while`)

2-QISM — STRUKTURALAR - 5-modul: Funksiyalar - 6-modul: Massivlar (arrays) va metodlar - 7-modul: Obyektlar (objects) - 8-modul: String va Number metodlari

3-QISM — BRAUZER (DOM) - 9-modul: DOM bilan ishlash - 10-modul: Events (hodisalar) - 11-modul: Forms va validation - 12-modul: Storage (localStorage)

4-QISM — ASINXRON JS - 13-modul: Callbacks va Event Loop - 14-modul: Promises - 15-modul: `async / await` - 16-modul: Fetch API va REST

5-QISM — CHUQUR JS - 17-modul: Scope, closures, hoisting - 18-modul: `this` , `call / apply / bind` - 19-modul: Prototypes va inheritance - 20-modul: Classes (ES6) - 21-modul: ES6+ (destructuring, spread, modules) - 22-modul: Error handling - 23-modul: Regular expressions

6-QISM — EXPERT - 24-modul: Functional programming - 25-modul: Iterators, generators, Symbols - 26-modul: Proxy, Reflect - 27-modul: Performance va memory - 28-modul: Design patterns - 29-modul: Bundlers va build tools - 30-modul: TypeScript'ga ko'prik

0-MODUL: Kirish — JavaScript nima va qanday ishlaydi

JavaScript nima?

JavaScript (qisqacha **JS**) — bu **dasturlash tili**. U birinchi marta 1995-yilda brauzerda veb-sahifalarni "jonlantirish" uchun yaratilgan: tugma bosilganda nimadir o'zgarsin, forma tekshirilsin, sahifa qayta yuklanmasdan yangilansin.

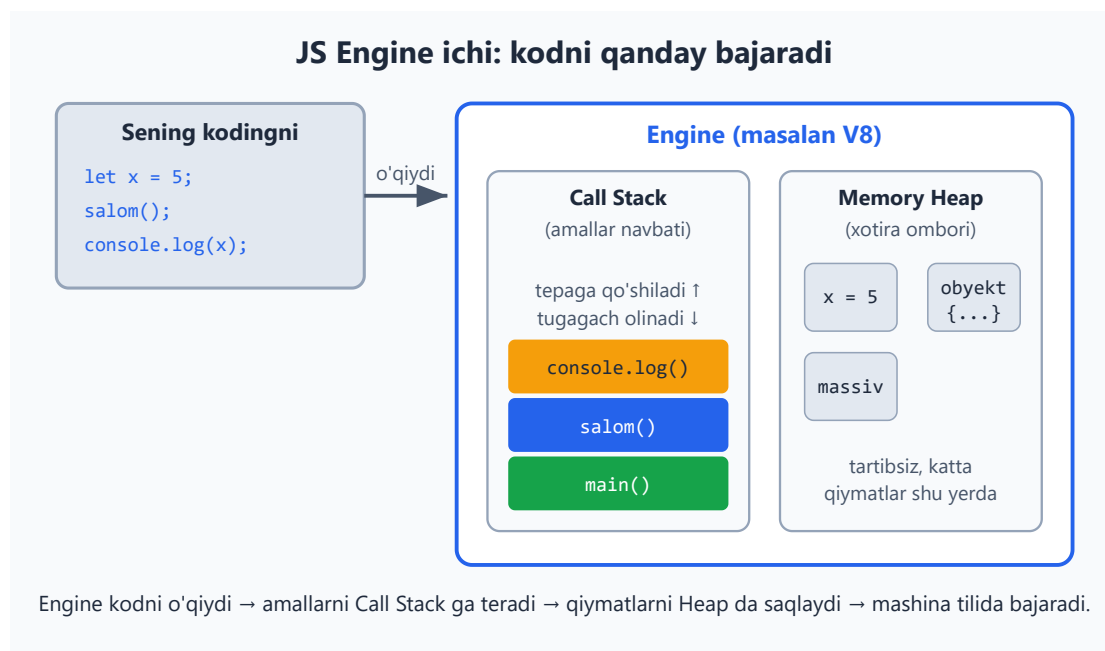
Bugun JS faqat brauzerda emas, **hamma joyda** ishlaydi: - **Brauzerda** (frontend) — sayt interaktivligi. - **Serverda** (Node.js orqali) — backend, API. - **Mobil ilovalar, desktop ilovalar**, hatto IoT qurilmalarda.

Nega o'rganish kerak? JS — internetning yagona "tug'ma" tili. Brauzer faqat JS'ni tushunadi. Frontend qilmoqchi bo'lsang, undan qochib qutula olmaysan.

JS qayerda ishlaydi (Engine)

Kodingni JS kodini **engine** (dvigatel) ishga tushiradi. Eng mashhuri — Google'ning **V8** engine'i (Chrome va Node.js ichida). Engine sening yozgan matningni o'qiydi, mashina tiliga aylantiradi va bajaradi.

Engine ichida ikki muhim qism bor: **Call Stack** (bajariladigan amallar navbati) va **Memory Heap** (o'zgaruvchi va obyekt qiymatlari saqlanadigan xotira):



Sen ikki yo'l bilan JS yoza olasan:

1. Brauzer konsoli (eng tez yo'l) Brauzerni och → **F12** (yoki o'ng tugma → "Inspect") → **Console** bo'limi. U yerga to'g'ridan-to'g'ri kod yozsang bo'ladi:



```
console.log("Salom, dunyo!");
```

`console.log()` — bu ekranga (konsolga) biror narsani **chop etish** uchun ishlatiladi. Bu sening eng yaqin do'sting bo'ladi: kod nima qilayotganini tekshirish uchun doim ishlatasan.

2. HTML fayl ichida

```
<!DOCTYPE html>
<html>
<head><title>Birinchi JS</title></head>
<body>
  <h1>Sahifa</h1>

  <script>
    console.log("Bu kod sahifa yuklanganda ishlaydi");
    alert("Salom!"); // brauzerda oyna chiqaradi
  </script>
</body>
</html>
```

`<script>` tegi ichidagi kod brauzer tomonidan bajariladi. Odatda `<script src="app.js"></script>` orqali alohida `.js` faylni ulaymiz — bu toza yondashuv.

Birinchi tushunchalar

- **Statement (buyruq)** — bitta amal. Odatda `;` (nuqta-vergul) bilan tugaydi.

```
console.log("birinchi");
console.log("ikkinchi");
```

- **Comment (izoh)** — engine o'qimaydigan, faqat odam uchun yozuv.

```
// bu bir qatorli izoh
/* bu
   ko'p qatorli izoh */
```

- **Case-sensitive** — JS katta-kichik harfni farqlaydi. `myName` va `myname` — ikki xil narsa.

Why: Izohlar kodni "tushuntirish" uchun emas, **nima uchun** shunday qilinganini tushuntirish uchun yoziladi. Yaxshi kod o'zini o'zi tushuntiradi; izoh esa sababni aytadi.

0-modul masalalari (20 ta)

Bularni brauzer konsolida (F12 → Console) bajaring.

1. Konsolga `Men JavaScript o'rganyapman` matnini chiqaring.
2. `alert()` yordamida brauzerda "Salom" degan oyna chiqaring.
3. Bir qatorli izoh yozing va uning ostiga `console.log` qo'ying.
4. Ko'p qatorli izoh (3 qator) yozing.
5. Ketma-ket 3 ta `console.log` yozing va natijani kuzating.
6. `console.log(2 + 2)` yozing — natija nima bo'ladi?
7. `console.log("2" + "2")` yozing. 6-masaladan farqi nimada?
8. `prompt("Isming nima?")` yozing — bu nima qiladi?
9. `console.log(typeof "salom")` natijasini toping.
10. Konsolda `console.error("Xato!")` va `console.warn("Ogohlantirish")` ni sinab ko'ring.
11. HTML fayl yaratib, ichiga `<script>` qo'ying va konsolga matn chiqaring.
12. `console.log("Salom", "dunyo", 123)` — vergul bilan ajratilganda nima bo'ladi?
13. `myName` va `myname` ni ikkita `console.log` da yozsangiz JS ularni bir xil deb biladimi?
14. `console.table([1, 2, 3])` ni sinab ko'ring.
15. `console.log("Qator1\nQator2")` — `\n` nima qiladi?
16. `console.log("Narx: " + 1000 + " so'm")` yozing.
17. Konsolda `Math.random()` ni 3 marta chaqiring — har safar bir xilmi?
18. `console.log(10 / 3)` natijasini ko'ring.
19. `console.clear()` nima qiladi?
20. Brauzer konsolida `1 + 1` deb yozsangiz (`console.log'siz`), natija chiqadimi?



► Yechimlar



```
// 1
console.log("Men JavaScript o'rganyapman");

// 2
alert("Salom");

// 3
// Bu izoh
console.log("Izoh ostidagi kod");

// 4
/* Birinchi qator
   Ikkinchi qator
   Uchinchi qator */

// 5
console.log("bir"); console.log("ikki"); console.log("uch");

// 6 -> 4 (raqamlar qo'shiladi)
console.log(2 + 2);

// 7 -> "22" (matnlar yopishadi). Farqi: tirnoq ichidagi narsa son emas,
matn (string).
console.log("2" + "2");

// 8 -> foydalanuvchidan matn so'raydigan oyna chiqaradi va kiritilganni
qaytaradi
prompt("Isming nima?");

// 9 -> "string"
console.log(typeof "salom");

// 10 -> error qizil, warn sariq rangda chiqadi
console.error("Xato!");
console.warn("Ogohlantirish");

// 12 -> "Salom dunyo 123" (probel bilan ajratib chiqaradi)
console.log("Salom", "dunyo", 123);

// 13 -> Yo'q, JS ularni ikki xil deb biladi (case-sensitive)

// 15 -> \n yangi qatorga o'tkazadi:
// Qator1
// Qator2
console.log("Qator1\nQator2");

// 16 -> "Narx: 1000 so'm"
console.log("Narx: " + 1000 + " so'm");

// 17 -> Yo'q, har safar boshqa tasodifiy son (0 dan 1 gacha)

// 18 -> 3.3333333333333335 (JS o'nlik kasrlarni shunday ko'rsatadi)

// 19 -> konsolni tozalaydi

// 20 -> Ha, konsol oxirgi ifoda natijasini avtomatik ko'rsatadi
```

1-MODUL: O'zgaruvchilar va ma'lumot turlari

O'zgaruvchi (Variable) nima?

O'zgaruvchi — bu **ma'lumotni saqlaydigan nomli quti**. Qutiga qiymat solib, keyin nom orqali undan foydalanasan.

```
let ism = "Oqil";  
let yosh = 25;  
  
console.log(ism); // Oqil  
console.log(yosh); // 25
```

Bu yerda `ism` va `yosh` — o'zgaruvchi nomlari, `"Oqil"` va `25` — ularning qiymatlari.

`let`, `const`, `var` — qaysi birini ishlatish?

JS'da o'zgaruvchi e'lon qilishning 3 usuli bor:

```
let a = 10;    // qiymatini keyin o'zgartirish mumkin  
const b = 20; // qiymati O'ZGARMAYDI (constant)  
var c = 30;    // eski usul — ishlatma
```

Qoida (yodlab ol): - Doim **`const`** ishlat. Qiymat o'zgarmasligi kerak bo'lsa — `const`. - Agar qiymat **o'zgarishi shart** bo'lsa (masalan, sanagich) — `let`. - **`var` 'ni hech qachon ishlatma**. U eski, muammoli (scope bilan bog'liq xatolar keltiradi — buni 17-modulda ko'ramiz).

```
const PI = 3.14;  
// PI = 3.15; // ❌ Xato! const'ni o'zgartirib bo'lmaydi  
  
let hisob = 0;  
hisob = hisob + 1; // ✅ let'ni o'zgartirsa bo'ladi
```

Why: `const` "default" tanlov. Sababi — kod **bashorat qilinadigan** bo'ladi. Qiymat o'zgarmasligini bilsang, kodni o'qish va xatoni topish osonlashadi. `let` 'ni faqat haqiqatan o'zgarish kerak bo'lganda ishlat.

Nomlash qoidalari

```
let userName;      // ✅ to'g'ri
let user_name;     // ✅ ishlaydi, lekin JS'da camelCase odat
let userName2;     // ✅ raqam bo'lsa bo'ladi (lekin boshida emas)

let 2user;         // ❌ raqam bilan boshlanmaydi
let user-name;     // ❌ tire bo'lmaydi
let let;           // ❌ band so'z (reserved word)
```

JS'da **camelCase** uslubi qo'llaniladi: `firstName`, `totalPrice`, `isActive`.

Ma'lumot turlari (Data Types)

JS'da 8 ta asosiy tur bor. Hozir eng muhimlarini ko'ramiz:

```
// 1. String (matn) - tirnoq ichida
let ism = "Oqil";
let shahar = 'Toshkent';
let salom = `Salom, ${ism}`; // template literal (orqa tirnoq)

// 2. Number (son) - butun va kasr
let yosh = 25;
let narx = 99.99;

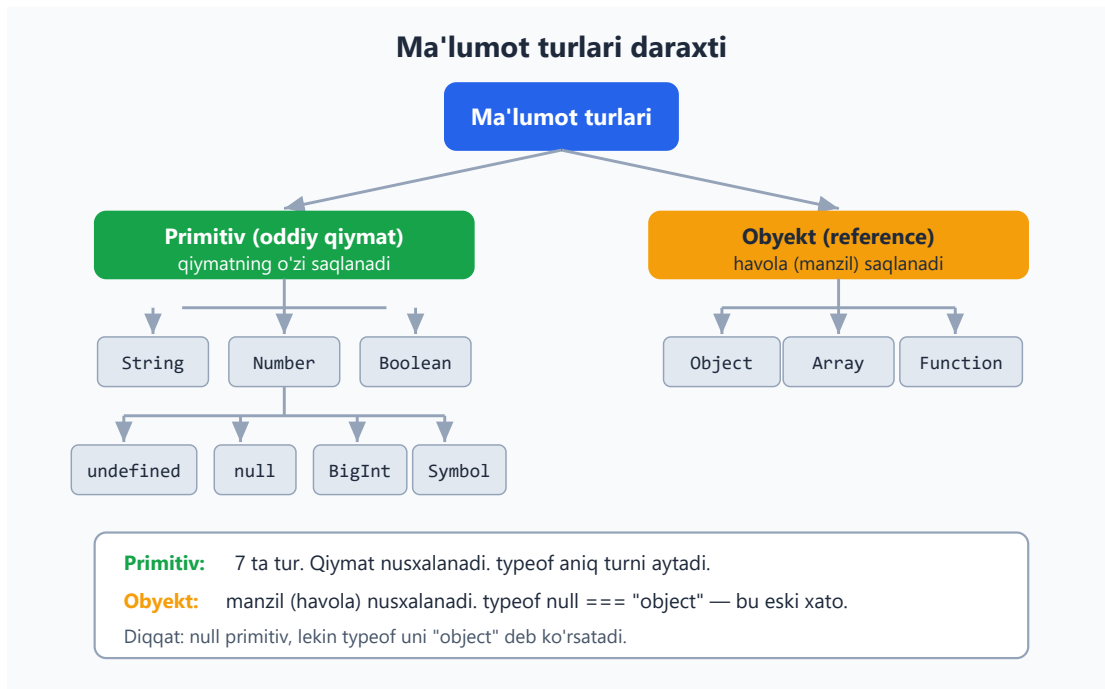
// 3. Boolean ( mantiqiy ) - faqat true yoki false
let faolmi = true;
let yopiqmi = false;

// 4. Undefined - qiymat berilmagan
let x;
console.log(x); // undefined

// 5. Null - "bo'sh" qiymat (ataylab)
let natija = null;

// 6. Object (obyekt) - keyingi qismda
// 7. Array (massiv) - keyingi qismda
// 8. BigInt, Symbol - keyinroq
```

Bu turlar ikki katta guruhga bo'linadi: **primitiv** (oddiy qiymatlar) va **obyekt** (havola orqali ishlaydigan murakkab turlar):



`typeof` — turni tekshirish

```
console.log(typeof "salom"); // "string"
console.log(typeof 42);      // "number"
console.log(typeof true);    // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null);     // "object" (bu JS'ning eski xatosi, esda tut)
```

Bu farqning amaliy oqibati bor: primitiv qiymat boshqa o'zgaruvchiga berilganda uning **nusxasi** ko'chiriladi, obyekt esa **havolasi** (xotiradagi manzili) ko'chiriladi — shuning uchun bir obyektни ikki nom orqali o'zgartirish ikkalasiga ham ta'sir qiladi:

Primitiv (nusxa) va Reference (havola)

Primitiv: qiymat NUSXALANADI

```
let a = 5;  
let b = a; // nusxa  
b = 99;
```

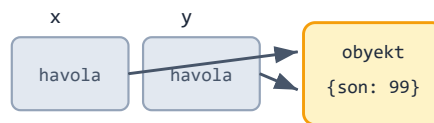


Har biri o'z qiymatini saqlaydi.
b ni o'zgartirsang, a o'zgarmaydi.

a = 5, b = 99

Obyekt: HAVOLA nusxalanadi

```
let x = {son: 5};  
let y = x; // havola  
y.son = 99;
```



x va y bitta obyektga ishora qiladi.
y orqali o'zgartirsang, x ham o'zgaradi!

x.son = 99, y.son = 99

Template literals (juda muhim)

Matn ichiga o'zgaruvchi qo'shishning eng toza yo'li — orqa tirnoq (` `) va `${...}`:

```
let ism = "Oqil";  
let yosh = 25;  
  
// Eski usul (noqulay):  
console.log("Mening ismim " + ism + ", yoshim " + yosh);  
  
// Yangi usul (toza):  
console.log(`Mening ismim ${ism}, yoshim ${yosh}`);
```

Tur o'zgartirish (Type Conversion)

```
// String -> Number  
let matn = "123";  
let son = Number(matn); // 123 (son)  
let son2 = parseInt("123px"); // 123 (oxiridagi matnni tashlaydi)  
let son3 = parseFloat("3.14m"); // 3.14  
  
// Number -> String  
let raqam = 456;  
let str = String(raqam); // "456"  
let str2 = raqam.toString(); // "456"  
  
// Booleanga  
let bool = Boolean(0); // false  
let bool2 = Boolean(1); // true
```

Why diqqat: `"5" + 3` natijasi `"53"` (matn), lekin `"5" - 3` natijasi `2` (son). `+` matnlarni yopishtiradi, qolgan amallar sonni majburlaydi. Bu JS'ning eng ko'p adashtiruvchi joyi — shuning uchun foydalanuvchidan kelgan ma'lumotni **doim** aniq turga o'zgartir.

1-modul masalalari (20 ta)

1. `ism` o'zgaruvchisiga o'z isimingni saqlang va konsolga chiqaring.
2. `const` orqali `PI = 3.14` yarating va uni o'zgartirishga urinib ko'ring — nima bo'ladi?
3. `yosh` o'zgaruvchisini `let` bilan yarating, keyin uni 1 ga oshiring.
4. Template literal yordamida `Salom, mening ismim X` matnini chiqaring (X — isiming).
5. `typeof` yordamida `"salom"`, `42`, `true`, `null` turlarini toping.
6. `"100"` matnini songa aylantirib, unga 50 qo'shing.
7. `"5" + 3` va `"5" - 3` natijalarini taqqoslang. Nega farq qiladi?
8. Son `789` ni matnga aylantiring va `typeof` bilan tekshiring.
9. `parseInt("42px")` va `parseFloat("3.99usd")` natijalarini toping.
10. Bir-biriga teng ikkita o'zgaruvchi yarating: `a = b = 10` o'rniga ikki qatorda yozing.
11. `undefined` va `null` orasidagi farqni izohli kod bilan ko'rsating.
12. Foydalanuvchidan `prompt` bilan yosh so'rang, uni songa aylantiring va `+10` qiling.
13. `Boolean(0)`, `Boolean("")`, `Boolean("salom")`, `Boolean(1)` — qaysilari `false`?
14. To'g'ri va noto'g'ri o'zgaruvchi nomlaridan 3 tadan misol yozing (izoh bilan).
15. `narx = 1500` va `valyuta = "so'm"` yarating, template literal bilan `1500 so'm` chiqaring.
16. `let x;` yozib, konsolga chiqaring — qiymati nima?
17. `10 + "5"`, `10 - "5"`, `"10" * "2"` natijalarini bashorat qiling, keyin tekshiring.
18. `isStudent` nomli boolean yarating va uni `if` siz to'g'ridan-to'g'ri chiqaring.
19. Ikki o'zgaruvchi (`a = 5`, `b = 10`) qiymatlarini **uchinchi o'zgaruvchi orqali** almashtiring.
20. `const user = "Ali"; user = "Vali";` — bu kod nega ishlamaydi? Tuzating.



► Yechimlar



```
// 1
const ism = "Oqil";
console.log(ism);

// 2 -> TypeError: Assignment to constant variable
const PI = 3.14;
// PI = 3.15; // ✗

// 3
let yosh = 25;
yosh = yosh + 1; // yoki yosh++
console.log(yosh); // 26

// 4
const ism4 = "Oqil";
console.log(`Salom, mening ismim ${ism4}`);

// 5
console.log(typeof "salom"); // "string"
console.log(typeof 42); // "number"
console.log(typeof true); // "boolean"
console.log(typeof null); // "object" (JS bug)

// 6 -> 150
console.log(Number("100") + 50);

// 7
console.log("5" + 3); // "53" -> + matnlarni yopishtiradi
console.log("5" - 3); // 2 -> - sonni majburlaydi

// 8
console.log(typeof String(789)); // "string"

// 9
console.log(parseInt("42px")); // 42
console.log(parseFloat("3.99usd")); // 3.99

// 10
let a = 10;
let b = 10;

// 11
let bersam; // undefined: hali qiymat yo'q
let boshataylab = null; // null: ataylab bo'sh qo'yildi

// 12
let kiritilgan = prompt("Yoshingiz?");
console.log(Number(kiritilgan) + 10);

// 13 -> 0 va "" false; "salom" va 1 true
console.log(Boolean(0)); // false
console.log(Boolean("")); // false
console.log(Boolean("salom")); // true
console.log(Boolean(1)); // true

// 14
let userName; // ✔ camelCase
```

```

let user_age; // ✅ ishlaydi
let total2; // ✅ raqam oxirida
// let 2total; // ❌ raqam bilan boshlanadi
// let my-var; // ❌ tire
// let const; // ❌ band so'z

// 15
const narx = 1500;
const valyuta = "so'm";
console.log(`${narx} ${valyuta}`);

// 16 -> undefined
let x;
console.log(x);

// 17
console.log(10 + "5"); // "105"
console.log(10 - "5"); // 5
console.log("10" * "2"); // 20

// 18
const isStudent = true;
console.log(isStudent);

// 19
let p = 5, q = 10;
let temp = p;
p = q;
q = temp;
console.log(p, q); // 10 5

// 20 -> const o'zgartirib bo'lmaydi. Tuzatish: let ishlatish
let user = "Ali";
user = "Vali"; // ✅

```

2-MODUL: Operatorlar

Operator — bu qiymatlar ustida amal bajaradigan belgi (+, -, >, va h.k.).

1. Arifmetik operatorlar

```

let a = 10, b = 3;

console.log(a + b); // 13  qo'shish
console.log(a - b); // 7   ayirish
console.log(a * b); // 30  ko'paytirish
console.log(a / b); // 3.333... bo'lish
console.log(a % b); // 1   QOLDIQ (modulo) — 10 ni 3 ga bo'lganda qoldiq
console.log(a ** b); // 1000 daraja (10^3)

```

% (qoldiq) juda foydali — masalan, son juft yoki toqligini aniqlash uchun:

```
console.log(10 % 2); // 0 -> juft
console.log(7 % 2);  // 1 -> toq
```

2. Tayinlash operatorlari (Assignment)

```
let x = 5;
x += 3; // x = x + 3 -> 8
x -= 2; // x = x - 2 -> 6
x *= 4; // x = x * 4 -> 24
x /= 2; // x = x / 2 -> 12
x %= 5; // x = x % 5 -> 2
```

3. Inkrement / Dekrement

```
let i = 5;
i++; // i = i + 1 -> 6
i--; // i = i - 1 -> 5

// ++ joyi muhim:
let a = 5;
console.log(a++); // 5 chiqaradi, KEYIN 6 bo'ladi (post-increment)
console.log(a);   // 6

let b = 5;
console.log(++b); // 6 (oldin oshiradi, keyin chiqaradi) (pre-increment)
```

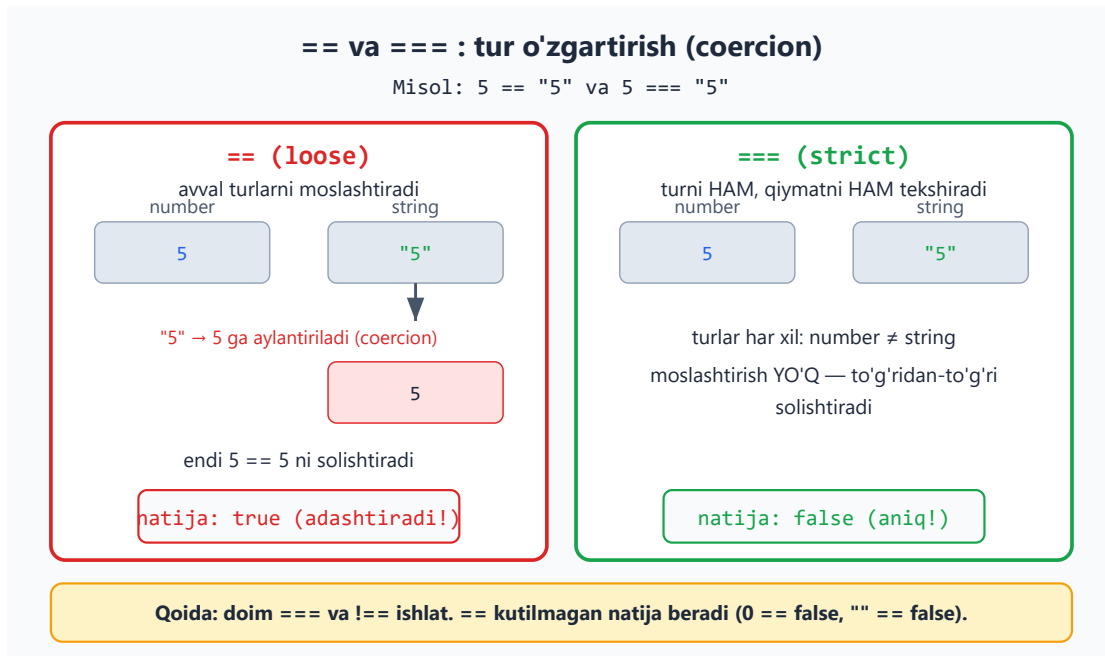
4. Taqqoslash operatorlari (natijasi `true` / `false`)

```
console.log(5 > 3);    // true
console.log(5 < 3);    // false
console.log(5 >= 5);   // true
console.log(5 <= 4);   // false

// === va == farqi (JUDA MUHIM)
console.log(5 == "5"); // true  (faqat qiymatni solishtiradi, turni e'tiborsiz qoldiradi)
console.log(5 === "5"); // false (qiymat VA turni solishtiradi)
console.log(5 != "5");  // false
console.log(5 !== "5"); // true
```

Why — eng muhim qoida: Doim `===` va `!==` ishlat, hech qachon `==` emas. `==` ma'lumot turini avtomatik o'zgartiradi va `0 == false`, `"" == false`, `null == undefined` kabi kutilmagan natijalar beradi. `===` esa aniq: qiymat ham, tur ham bir xil bo'lishi shart. Bu bug'larning katta qismini oldini oladi.

Quyidagi diagramma `==` ning turni qanday o'zgartirishini (coercion) va `===` ning nega aniqroq ekanini ko'rsatadi:



5. Mantiqiy operatorlar (Logical)

```
let yosh = 20;
let bormi = true;

// && (VA / AND) — ikkalasi ham true bo'lsa true
console.log(yosh >= 18 && bormi); // true

// || (YOKI / OR) — bittasi true bo'lsa true
console.log(yosh < 18 || bormi); // true

// ! (EMAS / NOT) — qiymatni teskari qiladi
console.log(!bormi); // false
```

Haqiqat jadvali: | A | B | A && B | A || B | |---|---|-----|-----| | true | true | true | true | | true | false | false | true | | false | true | false | true | | false | false | false | false |

6. Ternary operator (qisqa `if`)

```
let yosh = 20;
let xabar = yosh >= 18 ? "Voyaga yetgan" : "Voyaga yetmagan";
console.log(xabar); // "Voyaga yetgan"

// Sintaksis: shart ? agar_true : agar_false
```

Why: Ternary — qisqa, oddiy tanlovlar uchun. Lekin uni **ichma-ich** (nested) yozma — o'qib bo'lmaydigan kodga aylanadi. Murakkab shart bo'lsa, oddiy `if` ishlat.

2-modul masalalari (20 ta)

1. `17 % 5` natijasini toping va nima ekanini izohlang.
2. `2 ** 10` (2 ning 10-darajasi) ni hisoblang.
3. `x = 20` yarating, `+= 5`, `*= 2`, `-- 10` qo'llang, har bosqichni chiqaring.
4. Sonning juft yoki toqligini `%` orqali tekshiring (masalan, 14).
5. `let a = 5; a++` — `a++` chiqargan qiymat va keyingi qiymatni ko'rsating.
6. `++b` va `b++` orasidagi farqni misol bilan tushuntiring.
7. `5 == "5"` va `5 === "5"` natijalarini taqqoslang.
8. `0 == false`, `"" == false`, `null == undefined` — nega `true` chiqadi?
9. Yosh 18 dan katta **va** chiptasi borligini `&&` bilan tekshiring.
10. Foydalanuvchi admin **yoki** moderator ekanligini `||` bilan tekshiring.
11. `!true`, `!false`, `!0`, `!""` natijalarini toping.
12. Ternary bilan: son musbatmi yoki manfiymi — chiqaring.
13. `10 > 5 && 5 > 3 && 3 > 1` — natija nima?
14. `15 % 2 === 0 ? "juft" : "toq"` kodini yozing va sinab ko'ring.
15. Ikki sonni taqqoslab, kattasini ternary bilan chiqaring.
16. `true && false || true` — operatorlar ketma-ketligi bo'yicha natijani toping.
17. `let savol = 5; savol **= 2` — natija nima?
18. Foydalanuvchi yoshini so'rab, `>= 18` bo'lsa "kirsa bo'ladi" deb ternary bilan chiqaring.
19. `"10" - 5`, `"10" + 5`, `"10" * 2`, `"abc" - 5` — har birini bashorat qiling.
20. Bir sondan boshqasi nechta marta kattaligini `%` va `/` bilan ko'rsating (masalan, 17 va 5).



► Yechimlar



```
// 1 -> 2 (17 ni 5 ga bo'lganda qoldiq 2)
console.log(17 % 5);

// 2 -> 1024
console.log(2 ** 10);

// 3
let x = 20;
x += 5; console.log(x); // 25
x *= 2; console.log(x); // 50
x -= 10; console.log(x); // 40

// 4 -> 14 % 2 === 0 -> juft
console.log(14 % 2 === 0 ? "juft" : "toq");

// 5
let a = 5;
console.log(a++); // 5
console.log(a); // 6

// 6
let b = 5;
console.log(b++); // 5 (oldin chiqaradi)
let c = 5;
console.log(++c); // 6 (oldin oshiradi)

// 7
console.log(5 == "5"); // true
console.log(5 === "5"); // false

// 8 -> == turni o'zgartiradi: bularning hammasi "falsy" yoki bir-biriga
moslashtiriladi
console.log(0 == false); // true
console.log("" == false); // true
console.log(null == undefined); // true

// 9
let yosh = 20, chipta = true;
console.log(yosh > 18 && chipta); // true

// 10
let rol = "moderator";
console.log(rol === "admin" || rol === "moderator"); // true

// 11
console.log(!true); // false
console.log(!false); // true
console.log(!0); // true (0 = falsy)
console.log(!""); // true (" = falsy)

// 12
let son = -5;
console.log(son >= 0 ? "musbat" : "manfiy"); // manfiy

// 13 -> true
console.log(10 > 5 && 5 > 3 && 3 > 1);
```

```
// 14 -> "toq"
console.log(15 % 2 === 0 ? "juft" : "toq");

// 15
let m = 8, n = 12;
console.log(m > n ? m : n); // 12

// 16 -> true (&& birinchi: false; keyin false || true -> true)
console.log(true && false || true);

// 17 -> 25
let savol = 5;
savol **= 2;
console.log(savol);

// 18
let y = Number(prompt("Yoshingiz?"));
console.log(y >= 18 ? "kirsa bo'ladi" : "kirsa bo'lmaydi");

// 19
console.log("10" - 5); // 5
console.log("10" + 5); // "105"
console.log("10" * 2); // 20
console.log("abc" - 5); // NaN (Not a Number)

// 20
console.log(Math.floor(17 / 5)); // 3 marta
console.log(17 % 5); // 2 qoladi
```

3-MODUL: Shartli operatorlar (if / switch)

Dasturni "qaror qabul qilishga" o'rgatish — shartlar orqali bo'ladi.

if / else if / else

```
let yosh = 20;

if (yosh >= 18) {
  console.log("Voyaga yetgan");
} else {
  console.log("Voyaga yetmagan");
}
```

Bir nechta shart:

```
let ball = 85;

if (ball >= 90) {
```

```

    console.log("A'lo");
  } else if (ball >= 70) {
    console.log("Yaxshi");
  } else if (ball >= 50) {
    console.log("Qoniqarli");
  } else {
    console.log("Qoniqarsiz");
  }
}
// -> "Yaxshi"

```

Qoidalar: - `if` qavs ichidagi shart `true` bo'lsa, blok bajariladi. - `else if` — qo'shimcha shartlar (xohlagancha bo'lishi mumkin). - `else` — hech bir shart bajarilmasa (ixtiyoriy). - JS yuqoridan pastga tekshiradi va **birinchi true bo'lganida to'xtaydi**.

Truthy va Falsy (juda muhim tushuncha)

`if` ichida har qanday qiymat `true` yoki `false` ga "moslashadi". **Falsy** (false sanaluvchi) qiymatlar atigi 8 ta:

```
false, 0, "", null, undefined, NaN, 0n, -0
```

Qolgan hamma narsa truthy (true sanaluvchi):

```

if ("salom") console.log("ishlaydi"); // bo'sh bo'lmagan matn -> truthy
if (0) console.log("ishlamaydi");      // 0 -> falsy
if ([]) console.log("bo'sh massiv ham truthy!"); // [] -> truthy
(diqqat!)

```

Why: Buni bilish — kodni qisqartiradi. `if (ism !== "" && ism !== null)` o'rniga shunchaki `if (ism)` deb yozsa bo'ladi. Lekin diqqat: `0` ham haqiqiy qiymat bo'lishi mumkin (masalan, soni 0 ta mahsulot), shuning uchun raqamlar bilan ehtiyot bo'l.

Mantiqiy operatorlar bilan birga

```

let yosh = 25;
let haydovchilikGuvohnomasi = true;

if (yosh >= 18 && haydovchilikGuvohnomasi) {
  console.log("Mashina haydashi mumkin");
}

```

switch — ko'p variantli tanlov

Bitta qiymatni ko'p variantlar bilan solishtirganda `if/else if` o'rniga `switch` toza bo'ladi:

```
let kun = 3;

switch (kun) {
  case 1:
    console.log("Dushanba");
    break;
  case 2:
    console.log("Seshanba");
    break;
  case 3:
    console.log("Chorshanba");
    break;
  default:
    console.log("Noma'lum kun");
}
// -> "Chorshanba"
```

Diqqat — `break` shart! `break` bo'lmasa, JS keyingi `case` larga ham "oqib" o'tadi (fall-through). Buni ataylab ishlatish ham mumkin:

```
let harf = "a";
switch (harf) {
  case "a":
  case "e":
  case "i":
  case "o":
  case "u":
    console.log("Unli harf");
    break;
  default:
    console.log("Undosh harf");
}
```

Why `switch` vs `if`: `switch` faqat **aniq tenglik** (`===`) bilan ishlaydi. Diapazon kerak bo'lsa (`ball >= 90`), `if/else if` ishlat. Bitta qiymatni ko'p variant bilan solishtirsang — `switch` o'qishga qulayroq.

3-modul masalalari (20 ta)

1. Yoshni tekshiring: 18 dan katta bo'lsa "Voyaga yetgan", aks holda "Yo'q" chiqaring.
2. Ballni baholang: 90+ "A", 80+ "B", 70+ "C", aks holda "F".
3. Son musbat, manfiy yoki nol ekanligini aniqlang.

4. Foydalanuvchidan son so'rang, juft yoki toqligini `if` bilan chiqaring.
5. Falsy qiymatlardan 4 tasini `if` ichida sinab ko'ring.
6. `if ("0")` va `if (0)` — qaysi biri ishlaydi? Nega?
7. Yosh 18+ **va** guvohnoma bor bo'lsa "Haydashi mumkin" chiqaring.
8. Hafta kunini (1-7) `switch` bilan nomiga aylantiring.
9. Yil faslini oy raqamiga qarab `switch` bilan aniqlang (12,1,2 = qish).
10. Ikki sonning kattasini `if/else` bilan toping.
11. Foydalanuvchi parolini tekshiring: "1234" bo'lsa "To'g'ri", aks holda "Xato".
12. Son 3 ga **ham**, 5 ga **ham** bo'linsa "FizzBuzz" chiqaring.
13. Harf unli yoki undoshligini `switch` (fall-through) bilan aniqlang.
14. Yoshga qarab chipta narxini belgilang: bola(<12)=0, kattalar=50000.
15. `if (ism)` — ism bo'sh bo'lganda nima bo'ladi? Sinab ko'ring.
16. Uchburchak turlarini aniqlang: 3 tomon teng = teng tomonli, 2 tasi teng = teng yonli.
17. Foydalanuvchi rolini (`admin / user / guest`) `switch` bilan tekshirib, har biriga xabar bering.
18. Yilni tekshiring: kabisa yilmi? (4 ga bo'linsa, lekin 100 ga bo'linsa 400 ga ham bo'linishi kerak).
19. Ternary va `if` bilan bir xil masalani ikki usulda yeching.
20. Switch'da `break` ni olib tashlang va natijani kuzating — nima sodir bo'ladi?



► Yechimlar



```
// 1
let yosh = 20;
if (yosh > 18) console.log("Voyaga yetgan");
else console.log("Yo'q");

// 2
let ball = 85;
if (ball >= 90) console.log("A");
else if (ball >= 80) console.log("B");
else if (ball >= 70) console.log("C");
else console.log("F");

// 3
let son = -3;
if (son > 0) console.log("musbat");
else if (son < 0) console.log("manfiy");
else console.log("nol");

// 4
let n = Number(prompt("Son?"));
if (n % 2 === 0) console.log("juft");
else console.log("toq");

// 5
if (0) {} else console.log("0 falsy");
if ("") {} else console.log('"" falsy');
if (null) {} else console.log("null falsy");
if (undefined) {} else console.log("undefined falsy");

// 6 -> if("0") ISHLAYDI (bo'sh bo'lmagan matn truthy). if(0) ishlamaydi.
if ("0") console.log('"" truthy');
if (0) console.log("bu chiqmaydi");

// 7
let y = 20, guvohnoma = true;
if (y >= 18 && guvohnoma) console.log("Haydashi mumkin");

// 8
let kun = 3;
switch (kun) {
  case 1: console.log("Dushanba"); break;
  case 2: console.log("Seshanba"); break;
  case 3: console.log("Chorshanba"); break;
  case 4: console.log("Payshanba"); break;
  case 5: console.log("Juma"); break;
  case 6: console.log("Shanba"); break;
  case 7: console.log("Yakshanba"); break;
  default: console.log("Noto'g'ri");
}

// 9
let oy = 1;
switch (oy) {
  case 12: case 1: case 2: console.log("Qish"); break;
  case 3: case 4: case 5: console.log("Bahor"); break;
  case 6: case 7: case 8: console.log("Yoz"); break;
  case 9: case 10: case 11: console.log("Kuz"); break;
}
```

```

}

// 10
let a = 8, b = 12;
if (a > b) console.log(a); else console.log(b);

// 11
let parol = prompt("Parol?");
if (parol === "1234") console.log("To'g'ri");
else console.log("Xato");

// 12
let s = 15;
if (s % 3 === 0 && s % 5 === 0) console.log("FizzBuzz");

// 13
let harf = "e";
switch (harf) {
  case "a": case "e": case "i": case "o": case "u":
    console.log("Unli"); break;
  default: console.log("Undosh");
}

// 14
let yosh14 = 10;
if (yosh14 < 12) console.log("0 so'm");
else console.log("50000 so'm");

// 15 -> bo'sh matn falsy, shuning uchun else ishlaydi
let ism = "";
if (ism) console.log(`Salom ${ism}`);
else console.log("Ism kiritilmagan");

// 16
let t1 = 3, t2 = 3, t3 = 3;
if (t1 === t2 && t2 === t3) console.log("Teng tomonli");
else if (t1 === t2 || t2 === t3 || t1 === t3) console.log("Teng yonli");
else console.log("Turli tomonli");

// 17
let rol = "admin";
switch (rol) {
  case "admin": console.log("To'liq huquq"); break;
  case "user": console.log("Cheklangan huquq"); break;
  case "guest": console.log("Faqat ko'rish"); break;
  default: console.log("Noma'lum rol");
}

// 18
let yil = 2024;
if (yil % 400 === 0 || (yil % 4 === 0 && yil % 100 !== 0))
  console.log("Kabisa yil");
else console.log("Oddiy yil");

// 19
let z = 7;
// if usuli:
if (z > 5) console.log("katta"); else console.log("kichik");
// ternary usuli:
console.log(z > 5 ? "katta" : "kichik");

```

```
// 20 -> break bo'lmasa, mos case'dan keyingi HAMMA case bajariladi (fall-through)
```

4-MODUL: Sikllar (Loops)

Sikl — bir amalni **ko'p marta takrorlash** uchun. "1 dan 100 gacha chiqar" deb 100 ta `console.log` yozish o'rniga sikl ishlatasan.

for sikli

Eng ko'p ishlatiladigan sikl:

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}  
// 1, 2, 3, 4, 5
```

for ning 3 qismi (; bilan ajratilgan): 1. **Boshlash** (`let i = 1`) — bir marta, sikl boshida ishlaydi. 2. **Shart** (`i <= 5`) — har takrordan oldin tekshiriladi. `true` bo'lsa davom etadi. 3. **Qadam** (`i++`) — har takrordan keyin ishlaydi.

```
i=1 -> shart true -> blok -> i++ -> i=2 -> shart true -> ... -> i=6 ->  
shart false -> STOP
```

Misol — yig'indi hisoblash:

```
let yigindi = 0;  
for (let i = 1; i <= 100; i++) {  
  yigindi += i;  
}  
console.log(yigindi); // 5050
```

while sikli

Shart `true` bo'lguncha takrorlaydi. Necha marta takrorlanishini **oldindan bilmaganingda** ishlatiladi:

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
}
```

```
i++; // BUNI UNUTMA! aks holda cheksiz sikl
}
```

Diqqat — cheksiz sikl (infinite loop): Agar shart hech qachon `false` bo'lmasa, brauzer qotadi. `while` ichida shartni o'zgartiradigan qatorni (`i++`) doim qo'sh.

do...while sikli

`while` ga o'xshash, lekin **kamida bir marta** bajariladi (shart oxirida tekshiriladi):

```
let i = 10;
do {
  console.log(i); // 10 - shart false bo'lsa ham bir marta chiqadi
  i++;
} while (i <= 5);
```

break va continue

```
// break - siklni TO'LIQ to'xtatadi
for (let i = 1; i <= 10; i++) {
  if (i === 5) break;
  console.log(i); // 1, 2, 3, 4
}

// continue - joriy takrorni TASHLAB, keyingisiga o'tadi
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i); // 1, 2, 4, 5 (3 tashlandi)
}
```

for...of — to'plam elementlari bo'ylab

Massiv yoki matn elementlarini birma-bir olish uchun (eng toza usul — buni 6-modulda chuqurroq ko'ramiz):

```
const mevalar = ["olma", "uzum", "banan"];
for (const meva of mevalar) {
  console.log(meva);
}
// olma, uzum, banan
```

Ichma-ich sikllar (Nested loops)

```
for (let i = 1; i <= 3; i++) {  
  for (let j = 1; j <= 3; j++) {  
    console.log(`${i} x ${j} = ${i * j}`);  
  }  
}
```

Why diqqat: Ichma-ich sikllar "qimmat" — agar tashqi sikl N marta, ichki M marta aylansa, blok $N \times M$ marta bajariladi. Katta ma'lumotlarda bu sekinlashtiradi (bu — algoritmlardagi $O(n^2)$ tushunchasi). Iloji bo'lsa ichki siklni boshqa yo'l bilan almashtir.

4-modul masalalari (20 ta)

- 1 dan 10 gacha sonlarni `for` bilan chiqaring.
- 10 dan 1 gacha teskari sanang.
- 1 dan 100 gacha sonlar yig'indisini hisoblang.
- 1 dan 20 gacha faqat **juft** sonlarni chiqaring.
- 1 dan 50 gacha faqat **toq** sonlarni chiqaring.
- 5 ning ko'paytma jadvalini (5x1 dan 5x10 gacha) chiqaring.
- `while` bilan 1 dan 5 gacha sanang.
- `do...while` bilan shart noto'g'ri bo'lsa ham bir marta chiqishini ko'rsating.
- 100 dan 0 gacha 10 lik qadam bilan sanang (100, 90, 80...).
- Berilgan sonning faktorialini hisoblang ($5! = 120$).
- `break` bilan: 1 dan 100 gacha sanab, 7 ga yetganda to'xtang.
- `continue` bilan: 1 dan 10 gacha sanab, 5 ni tashlab keting.
- 1 dan 100 gacha 3 ga bo'linadigan sonlarni chiqaring.
- Matndagi harflar sonini `for...of` bilan sanang.
- 2 dan 10 gacha har bir sonning kvadratini chiqaring.
- Ichma-ich sikl bilan 3×3 ko'paytma jadvalini chiqaring.
- Foydalanuvchidan son so'rab, shu songacha sanang.
- 1 dan 50 gacha sonlar ichidan tub (prime) sonlarni toping.
- `*` belgilaridan piramida chizing (1, 2, 3, 4, 5 qatorli).
- Massiv `[10, 20, 30, 40]` elementlari yig'indisini sikl bilan toping.



► Yechimlar



```
// 1
for (let i = 1; i <= 10; i++) console.log(i);

// 2
for (let i = 10; i >= 1; i--) console.log(i);

// 3 -> 5050
let yig = 0;
for (let i = 1; i <= 100; i++) yig += i;
console.log(yig);

// 4
for (let i = 2; i <= 20; i += 2) console.log(i);

// 5
for (let i = 1; i <= 50; i += 2) console.log(i);

// 6
for (let i = 1; i <= 10; i++) console.log(`5 x ${i} = ${5 * i}`);

// 7
let i = 1;
while (i <= 5) { console.log(i); i++; }

// 8
let j = 100;
do { console.log(j); j++; } while (j <= 5); // 100 bir marta chiqadi

// 9
for (let i = 100; i >= 0; i -= 10) console.log(i);

// 10 -> 120
let n = 5, fact = 1;
for (let k = 1; k <= n; k++) fact *= k;
console.log(fact);

// 11
for (let i = 1; i <= 100; i++) {
  if (i === 7) break;
  console.log(i); // 1..6
}

// 12
for (let i = 1; i <= 10; i++) {
  if (i === 5) continue;
  console.log(i); // 5 tashlanadi
}

// 13
for (let i = 1; i <= 100; i++) {
  if (i % 3 === 0) console.log(i);
}

// 14
const matn = "JavaScript";
let count = 0;
for (const harf of matn) count++;
```

```

console.log(count); // 10

// 15
for (let i = 2; i <= 10; i++) console.log(`${i}^2 = ${i * i}`);

// 16
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    console.log(`${i} x ${j} = ${i * j}`);
  }
}

// 17
let limit = Number(prompt("Songacha sanay?"));
for (let i = 1; i <= limit; i++) console.log(i);

// 18 - tub sonlar
for (let son = 2; son <= 50; son++) {
  let tubmi = true;
  for (let d = 2; d < son; d++) {
    if (son % d === 0) { tubmi = false; break; }
  }
  if (tubmi) console.log(son);
}

// 19 - piramida
for (let i = 1; i <= 5; i++) {
  let qator = "";
  for (let j = 1; j <= i; j++) qator += "*";
  console.log(qator);
}
// *
// **
// ***
// ****
// *****

// 20 -> 100
const arr = [10, 20, 30, 40];
let sum = 0;
for (let i = 0; i < arr.length; i++) sum += arr[i];
console.log(sum);

```

✓ 1-qism yakuni

Tabriklayman — endi sen: - O'zgaruvchilar va ma'lumot turlarini bilasan (`const / let`, `string/number/boolean`) - Operatorlarni ishlata olasan (`===` ni esda tut!) - Shartlar bilan dasturni "qaror qabul qilishga" o'rgatasan - Sikllar bilan amallarni takrorlaysan

Bu — JS'ning poydevori. Bularsiz hech narsa qurilmaydi.

Keyingi qadam (2-qism)

- **Funksiyalar** — kodni qayta ishlatiladigan bloklarga bo'lish
- **Massivlar** va ularning kuchli metodlari (`map` , `filter` , `reduce`)
- **Obyektlar** — real dunyo ma'lumotlarini modellashtirish

Maslahat: 2-qismga o'tishdan oldin yuqoridagi 100 ta masalaning **kamida 70 tasini mustaqil** yeching. Ko'rib o'rganish — bilim emas; yozib o'rganish — bilim.

 [README / Mundarija](#) · [Keyingi: 2-qism — Strukturalar](#) →

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 README · [← 1-qism — Asoslar](#) · Keyingi: [3-qism — Brauzer \(DOM\)](#) →

2-QISM: STRUKTURALAR Funksiyalar, massivlar, obyektlar — real dasturlar shulardan quriladi. Har bir moduldan keyin **20 ta masala** (yechimi bilan). Avval o'zing yech, keyin [► Yechim](#) ga qara.

1-qismda asoslarni ko'rgan eding (o'zgaruvchilar, operatorlar, shartlar, sikllar). Endi shu asoslar ustiga **strukturalar** quramiz.

5-MODUL: Funksiyalar

Funksiya nima va nega kerak?

Funksiya — bu **nom berilgan, qayta ishlatiladigan kod bloki**. Bir amalni bir necha joyda takrorlash o'rniga, uni bir marta funksiyaga o'rab, nom orqali chaqirasan.

Funksiyasiz kod (yomon — takrorlanadi):

```
console.log("Salom, Ali!");
console.log("Salom, Vali!");
console.log("Salom, Hasan!");
```



Funksiya bilan (yaxshi — DRY: Don't Repeat Yourself):

```
function salomBer(ism) {
  console.log(`Salom, ${ism}!`);
}

salomBer("Ali");
salomBer("Vali");
salomBer("Hasan");
```



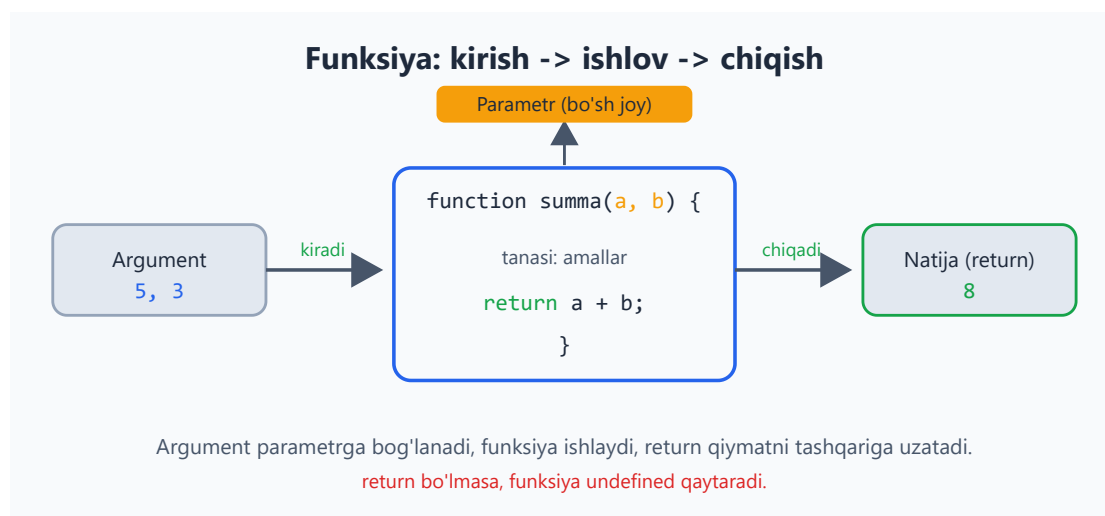
Why: Funksiyalar — toza kodning asosi. Bir mantiqni bir joyda saqlasang, o'zgartirish kerak bo'lganda **faqat bitta joyni** tahrirlaysan. Bu — xatoni kamaytiradi va kodni tushunarli qiladi.

Funksiya e'lon qilish (Declaration)

```
function summa(a, b) { // a, b - PARAMETRLAR
  return a + b;        // return - natijani QAYTARADI
}

let natija = summa(5, 3); // 5, 3 - ARGUMENTLAR
console.log(natija);      // 8
```

Atamalar: - **Parametr** — funksiya e'lonidagi "bo'sh joy" (a, b). - **Argument** — funksiyaning chaqiriganda beradigan haqiqiy qiymat (5, 3).



return — natijani qaytarish

Bu eng muhim tushuncha. `return` funksiya natijasini **tashqariga uzatadi**.

`console.log` esa shunchaki ekranga chiqaradi.

```
function kvadrat(n) {
  return n * n; // natijani qaytaradi
}

let x = kvadrat(4); // x = 16 - natijani saqlay olamiz
console.log(x * 2); // 32 - natija bilan ishlay olamiz

// return'siz funksiya:
function kvadrat2(n) {
  console.log(n * n); // faqat chiqaradi
}
let y = kvadrat2(4); // 16 chiqaradi, lekin y = undefined!
```

Why diqqat: `console.log` ≠ `return`. `console.log` — bu "ko'rsatish", `return` — bu "qaytarish". Funksiya boshqa kodga foyda berishi uchun **natijani return qilishi**

kerak. Junior'lar eng ko'p shu yerda kalishadi.

`return` funksiyani **darhol to'xtatadi** — undan keyingi kod ishlamaydi:

```
function tekshir(yosh) {  
  if (yosh < 18) {  
    return "Voyaga yetmagan"; // shu yerda to'xtaydi  
  }  
  return "Voyaga yetgan";  
}
```

Function Expression (funksiyani o'zgaruvchiga saqlash)

Funksiyani qiymat sifatida o'zgaruvchiga ham saqlash mumkin:

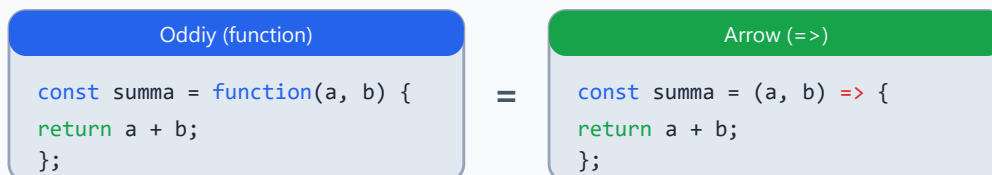
```
const kvadrat = function(n) {  
  return n * n;  
};  
  
console.log(kvadrat(5)); // 25
```

Arrow Functions (zamonaviy, qisqa yozuv)

ES6'dan keyin funksiyani qisqaroq yozish usuli:

```
// Oddiy:  
const summa = function(a, b) {  
  return a + b;  
};  
  
// Arrow:  
const summa2 = (a, b) => {  
  return a + b;  
};  
  
// Agar faqat return bo'lsa — qavs va return'ni tashlash mumkin  
(implicit return):  
const summa3 = (a, b) => a + b;  
  
// Bitta parametr bo'lsa — qavslarni ham tashlash mumkin:  
const kvadrat = n => n * n;
```

Oddiy funksiya vs Arrow funksiya



Arrowni yanada qisqartirish



Ikkalasi bir xil ishlaydi; arrow qisqaroq (lekin this boshqacha — 18-modul).

Misollar:

```
const juftmi = n => n % 2 === 0;  
const salom = ism => `Salom, ${ism}`;  
const yigindi = (a, b, c) => a + b + c;  
  
console.log(juftmi(4)); // true  
console.log(salom("Oqil")); // "Salom, Oqil"  
console.log(yigindi(1, 2, 3)); // 6
```

Why arrow: Arrow funksiyalar qisqa va massiv metodlarida (`map`, `filter`) juda qulay — keyingi modulda ko'rasan. Lekin ularda `this` boshqacha ishlaydi (buni 18-modulda chuqur ko'ramiz). Hozircha: oddiy hisob-kitob funksiyalari uchun arrow ishlat.

Default parametrlar

Argument berilmasa, standart qiymat ishlatiladi:

```
function salom(ism = "mehmon") {  
  return `Salom, ${ism}!`;  
}  
  
console.log(salom()); // "Salom, mehmon!"  
console.log(salom("Oqil")); // "Salom, Oqil!"
```

Rest parametrlar (...)

Noma'lum sondagi argumentlarni bitta massivga yig'adi:

```
function yigindi(...sonlar) {
  let jami = 0;
  for (const son of sonlar) jami += son;
  return jami;
}

console.log(yigindi(1, 2, 3)); // 6
console.log(yigindi(10, 20, 30, 40)); // 100
```

Higher-Order Functions (funksiyani argument sifatida)

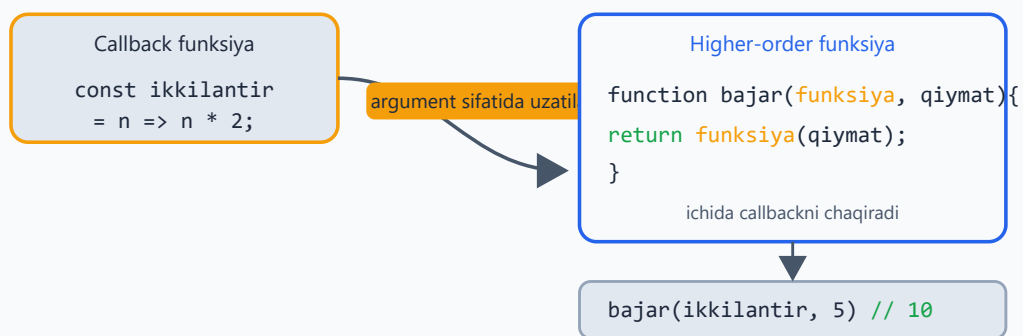
JS'da funksiya — bu ham qiymat. Demak, funksiyaning boshqa funksiyaga **argument sifatida** uzatish mumkin. Bu — JS'ning eng kuchli xususiyatlaridan biri:

```
function bajar(funksiya, qiymat) {
  return funksiya(qiymat);
}

const ikkilantir = n => n * 2;
console.log(bajar(ikkilantir, 5)); // 10
```

Argument sifatida uzatilgan funksiya **callback** deb ataladi. `map`, `filter`, `setTimeout` — hammasi callback qabul qiladi.

Higher-order function: funksiya — argument sifatida



`map`, `filter`, `setTimeout` — hammasi callback (funksiya) qabul qiladigan higher-order funksiyalar.

Why — toza arxitektura: Iloji bo'lsa **pure function** (sof funksiya) yoz: bir xil kirishga doim bir xil chiqish beradigan, tashqi narsani o'zgartirmaydigan funksiya. Bunday funksiyalarni test qilish va qayta ishlatish oson. (Backend'da DDD/SOLID'da ham shu prinsip — funksiya bitta ishni qilsin, side-effect'siz.)

```
// ✓ Pure: faqat kirishga bog'liq, tashqarini buzmaydi
const qoshish = (a, b) => a + b;
```

```
// ❌ Impure: tashqi o'zgaruvchini o'zgartiradi (side-effect)
let jami = 0;
const qoshishYomon = n => { jami += n; };
```

5-modul masalalari (20 ta)

1. `salomBer(ism)` funksiyasini yozing — "Salom, X!" qaytarsin (return bilan).
2. Ikki sonni qo'shadigan funksiya yozing.
3. `console.log` va `return` farqini bitta misolda ko'rsating.
4. Yuqoridagi qo'shish funksiyasini **function expression** ko'rinishida yozing.
5. Uni **arrow function**ga aylantiring (bir qatorda).
6. `salom(ism = "mehmon")` — default parametr bilan yozing.
7. Uchburchak yuzasini hisoblovchi funksiya: $(asos * balandlik) / 2$.
8. `juftmi(n)` — son juft bo'lsa `true` qaytarsin.
9. Ikki sondan kattasini qaytaradigan funksiya yozing.
10. Rest parametr (`...`) bilan cheksiz sonlar yig'indisini hisoblang.
11. Foizni hisoblang: `foiz(son, foiz)` → son ning shuncha foizini qaytarsin.
12. Stringni teskari aylantiruvchi funksiya (masalan, "abc" → "cba").
13. `kvadrat = n => ...` arrow funksiyasini bir qatorda yozing.
14. Yoshga qarab chipta narxini qaytaring ($<12 = 0$, aks holda 50000).
15. Higher-order: funksiyani argument sifatida qabul qilib chaqiruvchi funksiya yozing.
16. `kobpaytir(a, b = 1)` — ikkinchi argument berilmasa 1 bo'lsin.
17. Massivdagi eng katta sonni qaytaruvchi funksiya yozing (sikl bilan).
18. Funksiya yozing: son bersa, uning kvadrati va kubini **massiv** qilib qaytarsin.
19. Pure va impure funksiyaga bittadan misol yozing (izoh bilan).
20. Mini-kalkulyator: `hisobla(a, b, amal)` — amal "+", "-", "*", "/" bo'lsin.



► Yechimlar



```
// 1
function salomBer(ism) { return `Salom, ${ism}!`; }
console.log(salomBer("Oqil"));

// 2
function qoshish(a, b) { return a + b; }

// 3
function f1(n) { return n * 2; } // qaytaradi, saqlash mumkin
function f2(n) { console.log(n * 2); } // faqat chiqaradi, undefined qaytaradi
let r = f1(5); // r = 10
f2(5); // 10 chiqadi, lekin natijani saqlay olmaymiz

// 4
const qoshish4 = function(a, b) { return a + b; };

// 5
const qoshish5 = (a, b) => a + b;

// 6
const salom6 = (ism = "mehmon") => `Salom, ${ism}!`;
console.log(salom6()); // Salom, mehmon!
console.log(salom6("Oqil")); // Salom, Oqil!

// 7
const yuza = (asos, balandlik) => (asos * balandlik) / 2;
console.log(yuza(10, 5)); // 25

// 8
const juftmi = n => n % 2 === 0;

// 9
const kattasi = (a, b) => a > b ? a : b;

// 10
const yigindi = (...sonlar) => {
  let jami = 0;
  for (const s of sonlar) jami += s;
  return jami;
};
console.log(yigindi(1, 2, 3, 4)); // 10

// 11
const foiz = (son, foizMiqdori) => (son * foizMiqdori) / 100;
console.log(foiz(200, 15)); // 30

// 12
const teskari = str => {
  let natija = "";
  for (const harf of str) natija = harf + natija;
  return natija;
};
console.log(teskari("abc")); // cba

// 13
const kvadrat = n => n * n;
```

```

// 14
const chiptaNarxi = yosh => yosh < 12 ? 0 : 50000;

// 15
const bajar = (funksiya, qiymat) => funksiya(qiymat);
console.log(bajar(n => n * 3, 4)); // 12

// 16
const kobaytir = (a, b = 1) => a * b;
console.log(kobaytir(5)); // 5
console.log(kobaytir(5, 3)); // 15

// 17
function engKatta(arr) {
  let max = arr[0];
  for (const son of arr) if (son > max) max = son;
  return max;
}
console.log(engKatta([3, 9, 2, 7])); // 9

// 18
const kvadratVaKub = n => [n * n, n * n * n];
console.log(kvadratVaKub(3)); // [9, 27]

// 19
const pure = (a, b) => a + b; // ✅ tashqarini buzmaydi
let hisob = 0;
const impure = n => { hisob += n; }; // ❌ tashqi hisob'ni o'zgartiradi

// 20
function hisobla(a, b, amal) {
  switch (amal) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/": return b !== 0 ? a / b : "0 ga bo'lib bo'lmaydi";
    default: return "Noma'lum amal";
  }
}
console.log(hisobla(10, 5, "*")); // 50

```

6-MODUL: Massivlar (Arrays) va metodlar

Massiv nima?

Massiv — bu **tartiblangan ro'yxat**. Ko'p qiymatni bitta o'zgaruvchida saqlaydi.

```

const mevalar = ["olma", "uzum", "banan"];
const sonlar = [10, 25, 7, 100];
const aralash = [1, "matn", true, null]; // turlari aralash bo'lishi
mumkin

```

Elementga kirish va indekslar

Massiv **0 dan** sanaladi (birinchi element — indeks 0):

```
const mevalar = ["olma", "uzum", "banan"];

console.log(mevalar[0]); // "olma"
console.log(mevalar[1]); // "uzum"
console.log(mevalar[2]); // "banan"
console.log(mevalar[10]); // undefined (yo'q element)

console.log(mevalar.length); // 3 - uzunlik
console.log(mevalar[mevalar.length-1]); // "banan" - oxirgi element
```

Element qo'shish va o'chirish

```
const arr = [1, 2, 3];

arr.push(4); // OXIRIGA qo'shadi -> [1,2,3,4]
arr.pop(); // OXIRGINI o'chiradi -> [1,2,3]
arr.unshift(0); // BOSHIGA qo'shadi -> [0,1,2,3]
arr.shift(); // BIRINCHINI o'chiradi -> [1,2,3]
```

Eslab qol: `push` / `pop` — oxiri bilan, `shift` / `unshift` — boshi bilan ishlaydi.

Qidirish metodlari

```
const sonlar = [10, 20, 30, 40];

console.log(sonlar.includes(20)); // true - bormi?
console.log(sonlar.indexOf(30)); // 2 - indeks nechada?
console.log(sonlar.indexOf(99)); // -1 - topilmasa -1
console.log(sonlar.find(n => n > 25)); // 30 - birinchi mos element
console.log(sonlar.findIndex(n => n > 25)); // 2 - uning indeksi
```

Qism olish: `slice` va `splice`

```
const arr = [1, 2, 3, 4, 5];

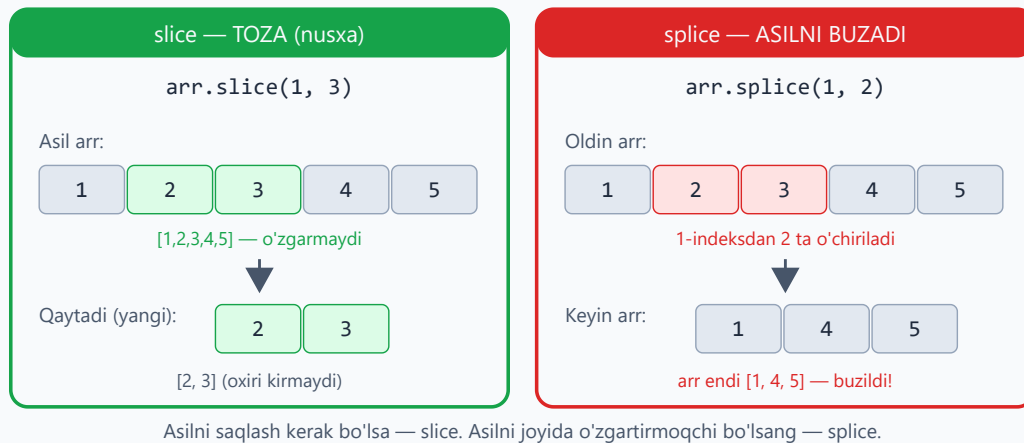
// slice(boshi, oxiri) - QISMNI NUSXALAYDI, asilni buzmaydi
console.log(arr.slice(1, 3)); // [2, 3] (oxiri kirmaydi)
console.log(arr); // [1,2,3,4,5] - o'zgarmadi
```

```
// splice — asil massivni O'ZGARTIRADI (o'chiradi/qo'shadi)
arr.splice(1, 2); // 1-indeksdan 2 ta o'chir -> [1, 4, 5]
```

Why diqqat: `slice` — toza (nusxa qaytaradi). `splice` — asilni buzadi. Nomlar o'xshash, ish boshqa. Funksional uslubda asil ma'lumotni saqlash uchun ko'pincha `slice` afzal.

slice vs splice — o'xshash nom, boshqa ish

```
const arr = [1, 2, 3, 4, 5]
```



★ Eng muhim 3 metod: `map`, `filter`, `reduce`

Bu uchta — JS'da kundalik ishlatiladigan eng kuchli vositalar. Sikl yozish o'rniga shularni ishlat.

`map` — har elementni o'zgartiradi (yangi massiv qaytaradi)

```
const sonlar = [1, 2, 3, 4];

const ikkilik = sonlar.map(n => n * 2);
console.log(ikkilik); // [2, 4, 6, 8]
console.log(sonlar); // [1, 2, 3, 4] — asil o'zgarmaydi

const narxlar = [100, 200, 300];
const formatlangan = narxlar.map(n => `${n} so'm`);
console.log(formatlangan); // ["100 so'm", "200 so'm", "300 so'm"]
```

`map` = "har biriga bir amal qo'llab, yangi massiv yasa".

`filter` — shartga mos elementlarni saralaydi

```
const sonlar = [1, 2, 3, 4, 5, 6];

const juftlar = sonlar.filter(n => n % 2 === 0);
console.log(juftlar); // [2, 4, 6]

const kattalar = sonlar.filter(n => n > 3);
console.log(kattalar); // [4, 5, 6]
```

filter = "shart true bo'lganlarni qoldir".

reduce — massivni bitta qiymatga "yig'adi"

Bu eng murakkabi, lekin juda kuchli. Massivni bitta natijaga aylantiradi (yig'indi, ko'paytma, eng katta, va h.k.):

```
const sonlar = [1, 2, 3, 4];

const yigindi = sonlar.reduce((jami, son) => jami + son, 0);
console.log(yigindi); // 10
```

reduce qanday ishlaydi (qadam-baqadam):

```
boshlang'ich jami = 0 (ikkinchi argument)
jami=0, son=1 -> 0+1 = 1
jami=1, son=2 -> 1+2 = 3
jami=3, son=3 -> 3+3 = 6
jami=6, son=4 -> 6+4 = 10
natija = 10
```

map / filter / reduce — massivni qanday o'zgartiradi?

map
har elementni o'zgartiradi
1 2 3 4 $\xrightarrow{n \Rightarrow n * 2}$ 2 4 6 8
massiv (bir xil uzunlik)

filter
shartga mos qoldiradi
1 2 3 4 $\xrightarrow{n \% 2 === 0}$ 2 4
massiv (qisqaroq)

reduce
bitta qiymatga yig'adi
1 2 3 4 $\xrightarrow{(j, n) \Rightarrow j + n}$ 10
bitta qiymat

Uchchalasi ham asil massivni buzmaydi (immutable). map -> massiv, filter -> massiv, reduce -> qiymat.

1-qism bilan bog'lanish: Eslaysanmi, 1-qismda massiv yig'indisini `for` sikli bilan topgan eding (5-6 qator). `reduce` bilan bu **bir qator**: `arr.reduce((a, b) => a + b, 0)`. Mana shu — kodni qisqartirish va o'qishni osonlashtirish.

`forEach` — har element bo'ylab (natija qaytarmaydi)

```
const ismlar = ["Ali", "Vali", "Hasan"];
ismlar.forEach(ism => console.log(`Salom, ${ism}`));
```

`forEach` vs `map`: `map` **yangi massiv qaytaradi**, `forEach` esa faqat amal bajaradi (qaytarmaydi). Yangi massiv kerak bo'lsa — `map`.

`sort`, `reverse`

```
const sonlar = [30, 5, 100, 25];

// ⚠ Oddiy sort sonlarni MATN sifatida saralaydi (xato!):
console.log([10, 2, 30].sort()); // [10, 2, 30] -> aslida [10, 2, 30]
noto'g'ri tartib

// To'g'ri usul — comparator funksiya:
sonlar.sort((a, b) => a - b); // o'sish: [5, 25, 30, 100]
sonlar.sort((a, b) => b - a); // kamayish: [100, 30, 25, 5]

const arr = [1, 2, 3];
arr.reverse(); // [3, 2, 1]
```

Why diqqat: `sort()` argumentsiz sonlarni **string** sifatida saralaydi (`"10" < "2"` chunki `"1" < "2"`). Sonlar uchun **doim** `(a, b) => a - b` ber. Bu — ko'p uchraydigan bug.

`join`, `split`

```
const ismlar = ["Ali", "Vali", "Hasan"];
console.log(ismlar.join(", ")); // "Ali, Vali, Hasan" — massiv -> matn

const matn = "Ali,Vali,Hasan";
console.log(matn.split(", ")); // ["Ali", "Vali", "Hasan"] — matn ->
massiv
```

`some`, `every`

```
const sonlar = [2, 4, 6, 8];

console.log(sonlar.every(n => n % 2 === 0)); // true - HAMMASI juftmi?
console.log(sonlar.some(n => n > 5));        // true - KAMIDA BITTASI 5
dan kattami?
```

Spread (...) massivlar bilan

```
const a = [1, 2, 3];
const b = [4, 5, 6];

const birlashgan = [...a, ...b]; // [1,2,3,4,5,6]
const nusxa = [...a];           // toza nusxa (asilni buzmaydi)
const yangi = [0, ...a, 4];      // [0,1,2,3,4]
```

Destructuring (massivdan ajratib olish)

```
const arr = [10, 20, 30];

const [birinchi, ikkinchi] = arr;
console.log(birinchi, ikkinchi); // 10 20

const [a, , c] = arr; // ikkinchini tashlab
console.log(a, c); // 10 30
```

Boshqa foydali metodlar (concat , flat , flatMap , at , findLast)

Eng muhim metodlardan tashqari, bilib qo'yish foydali bo'lganlari:

```
// concat - ikki massivni birlashtiradi (yangi massiv qaytaradi)
console.log([1, 2].concat([3, 4])); // [1, 2, 3, 4]
// (zamonaviy yo'l: [...[1,2], ...[3,4]] - spread)

// flat - ichma-ich massivni "yoyadi"
console.log([1, [2, 3], [4, [5]]].flat()); // [1, 2, 3, 4, [5]]
(1 daraja)
console.log([1, [2, [3, [4]]]].flat(Infinity)); // [1, 2, 3, 4]
(to'liq)

// flatMap - map + flat(1) birga (har element massiv qaytarsa, yoyadi)
console.log([1, 2, 3].flatMap(x => [x, x * 10])); // [1, 10, 2, 20, 3, 30]
```

```
// at - indeks bo'yicha element; MANFIY indeks oxiridan sanaydi
console.log([10, 20, 30].at(-1)); // 30 (oxirgi - arr[arr.length-1] o'rniga)
console.log([10, 20, 30].at(0)); // 10

// findLast - find kabi, lekin OXIRIDAN qidiradi
console.log([5, 12, 8, 130, 44].findLast(x => x > 10)); // 44
```

Why `at(-1)` : oxirgi elementni olishning eng toza yo'li. Eski usul — `arr[arr.length - 1]` (uzun); yangi usul — `arr.at(-1)`. `flat` / `flatMap` esa "massiv ichida massiv" bilan ishlaganda (masalan, guruhlangan ma'lumotni yoyishda) juda qulay.

6-modul masalalari (20 ta)

1. `["q", "w", "e"]` massiv yarating va 2-elementni chiqaring.
2. Massivga `push` bilan element qo'shing, `pop` bilan o'chiring.
3. Massiv uzunligi va oxirgi elementini chiqaring.
4. `includes` bilan element bor-yoqligini tekshiring.
5. `indexOf` bilan elementning indeksini toping.
6. `forEach` bilan har bir ismni "Salom, X" qilib chiqaring.
7. `map`: `[1,2,3,4]` har sonini 3 ga ko'paytiring.
8. `filter`: `[1..10]` ichidan faqat juft sonlarni oling.
9. `reduce`: `[10,20,30]` yig'indisini toping.
10. `reduce`: massivdagi eng katta sonni toping.
11. `find`: `[3,7,12,5]` ichidan birinchi 10 dan katta sonni toping.
12. `[30, 5, 100, 25]` ni o'sish tartibida saralang (to'g'ri usulda).
13. Xuddi shu massivni kamayish tartibida saralang.
14. `["a", "b", "c"]` ni teskari aylantiring.
15. `["Ali", "Vali"]` ni `join` bilan `"Ali & Vali"` qiling.
16. `"olma-uzum-banan"` ni `split` bilan massivga aylantiring.
17. `[1,2,3,4,5]` dan `slice` bilan o'rtadagi 3 tasini oling.
18. Ikki massivni `spread` bilan birlashtiring.
19. `some` va `every` bilan: massivdagi sonlar hammasi musbatmi? Kamida bittasi 100 dan kattami?
20. **Murakkab**: mahsulotlar massivi `[{nom:"A", narx:100}, {nom:"B", narx:50}, {nom:"C", narx:200}]` — narxi 100 dan oshganlarning **jami narxini** toping (filter +

reduce).



```
// 1
const arr1 = ["q", "w", "e"];
console.log(arr1[2]); // "e"

// 2
const arr2 = [1, 2, 3];
arr2.push(4); // [1,2,3,4]
arr2.pop(); // [1,2,3]

// 3
const arr3 = [5, 10, 15];
console.log(arr3.length); // 3
console.log(arr3[arr3.length - 1]); // 15

// 4
console.log([10, 20, 30].includes(20)); // true

// 5
console.log([10, 20, 30].indexOf(30)); // 2

// 6
["Ali", "Vali", "Hasan"].forEach(ism => console.log(`Salom, ${ism}`));

// 7
console.log([1, 2, 3, 4].map(n => n * 3)); // [3,6,9,12]

// 8
console.log([1,2,3,4,5,6,7,8,9,10].filter(n => n % 2 === 0)); //
[2,4,6,8,10]

// 9 -> 60
console.log([10, 20, 30].reduce((jami, n) => jami + n, 0));

// 10 -> 9
console.log([3, 9, 2, 7].reduce((max, n) => n > max ? n : max));

// 11 -> 12
console.log([3, 7, 12, 5].find(n => n > 10));

// 12 -> [5, 25, 30, 100]
console.log([30, 5, 100, 25].sort((a, b) => a - b));

// 13 -> [100, 30, 25, 5]
console.log([30, 5, 100, 25].sort((a, b) => b - a));

// 14 -> ["c","b","a"]
console.log(["a", "b", "c"].reverse());

// 15 -> "Ali & Vali"
console.log(["Ali", "Vali"].join(" & "));

// 16 -> ["olma","uzum","banan"]
console.log("olma-uzum-banan".split("-"));

// 17 -> [2,3,4]
console.log([1, 2, 3, 4, 5].slice(1, 4));
```

```
// 18 -> [1,2,3,4,5,6]
console.log([...[1, 2, 3], ...[4, 5, 6]]);

// 19
const nums = [2, 5, 8, 120];
console.log(nums.every(n => n > 0)); // true
console.log(nums.some(n => n > 100)); // true

// 20 -> 300
const mahsulotlar = [
  { nom: "A", narx: 100 },
  { nom: "B", narx: 50 },
  { nom: "C", narx: 200 },
];
const jami = mahsulotlar
  .filter(m => m.narx > 100)
  .reduce((sum, m) => sum + m.narx, 0);
console.log(jami); // 200... e'tibor ber: faqat narx>100, ya'ni faqat C(200). Natija 200.
// Agar narx>=100 kerak bo'lsa: filter(m => m.narx >= 100) -> A(100)+C(200)=300
```

7-MODUL: Obyektlar (Objects)

Obyekt nima?

Massiv — tartiblangan ro'yxat (indeks bilan). Obyekt esa — **nom-qiymat juftliklari** (key: value). Real dunyo narsalarini modellashtirish uchun ishlatiladi.

```
const user = {
  ism: "Oqil",
  yosh: 25,
  shahar: "Toshkent",
  faolmi: true,
};
```

Bu yerda `ism`, `yosh` — **kalitlar (keys)**, `"Oqil"`, `25` — **qiymatlar (values)**.

Xususiyatga kirish: nuqta va kvadrat qavs

```
const user = { ism: "Oqil", yosh: 25 };

// Nuqta (dot) — eng ko'p ishlatiladi:
console.log(user.ism); // "Oqil"
console.log(user.yosh); // 25

// Kvadrat qavs (bracket) — kalit o'zgaruvchida bo'lsa:
```

```
console.log(user["ism"]); // "Oqil"

let kalit = "yosh";
console.log(user[kalit]); // 25 – dinamik kalit
```

Why bracket: Kalit nomini **o'zgaruvchidan** olganingda yoki kalitda probel/maxsus belgi bo'lsa, kvadrat qavs kerak. Aks holda nuqta toza va qulay.

Qo'shish, o'zgartirish, o'chirish

```
const user = { ism: "Oqil" };

user.yosh = 25;           // qo'shish
user.ism = "Oqiljon";    // o'zgartirish
delete user.yosh;        // o'chirish

console.log(user); // { ism: "Oqiljon" }
```

Metodlar (obyekt ichidagi funksiyalar)

Obyektga funksiya qo'shsang, u **metod** deyiladi. `this` — joriy obyektga ishora qiladi:

```
const user = {
  ism: "Oqil",
  yosh: 25,
  salomBer() {
    return `Salom, men ${this.ism}`;
  },
};

console.log(user.salomBer()); // "Salom, men Oqil"
```

this haqida qisqacha: `this` — "shu obyektning o'zi". `this.ism = user.ism`. `this` JS'da eng chalkash mavzu — uni to'liq 18-modulda ko'ramiz. Hozircha: metod ichida o'z obyektining xususiyatlariga `this.` orqali murojaat qil.

`Object.keys`, `Object.values`, `Object.entries`

Obyekt ustida aylanish uchun:

```
const user = { ism: "Oqil", yosh: 25, shahar: "Toshkent" };

console.log(Object.keys(user)); // ["ism", "yosh", "shahar"]
console.log(Object.values(user)); // ["Oqil", 25, "Toshkent"]
```

```

console.log(Object.entries(user)); // [{"ism","Oqil"}, {"yosh",25},
...]

// Obyekt bo'ylab aylanish:
for (const [kalit, qiymat] of Object.entries(user)) {
  console.log(`${kalit}: ${qiymat}`);
}

```

Ichma-ich obyektlar (Nested)

```

const user = {
  ism: "Oqil",
  manzil: {
    shahar: "Toshkent",
    tuman: "Yunusobod",
  },
};

console.log(user.manzil.shahar); // "Toshkent"

// Xavfsiz kirish (optional chaining) – xususiyat yo'q bo'lsa ham xato
bermaydi:
console.log(user.manzil?.davlat); // undefined (xato emas)
console.log(user.ishjoy?.nom);    // undefined (ishjoy yo'q, lekin xato
yo'q)

```

Why ?. (optional chaining): `user.ishjoy.nom` — agar `ishjoy` yo'q bo'lsa, JS xato (Cannot read properties of undefined) beradi va dastur to'xtaydi. `?.` esa — yo'q bo'lsa shunchaki `undefined` qaytaradi, xato bermaydi. API'dan kelgan ma'lumot bilan ishlaganda hayotni saqlaydi.

Obyektlar massivi (juda muhim — real ma'lumotlar shunday)

API'dan keladigan ma'lumotlar odatda obyektlar massivi ko'rinishida bo'ladi:

```

const users = [
  { ism: "Ali", yosh: 20 },
  { ism: "Vali", yosh: 30 },
  { ism: "Hasan", yosh: 25 },
];

// Faqat ismlarni olish:
const ismlar = users.map(u => u.ism); // ["Ali", "Vali", "Hasan"]

// 25 dan kattalarni saralash:
const kattalar = users.filter(u => u.yosh > 25); // [{Vali, 30}]

```

```
// O'rtacha yosh:  
const ortalacha = users.reduce((s, u) => s + u.yosh, 0) / users.length;  
// 25
```

Destructuring (obyektdan ajratib olish)

```
const user = { ism: "Oqil", yosh: 25, shahar: "Toshkent" };  
  
const { ism, yosh } = user;  
console.log(ism, yosh); // "Oqil" 25  
  
// Yangi nom berish:  
const { ism: foydalanuvchiIsmi } = user;  
console.log(foydalanuvchiIsmi); // "Oqil"  
  
// Default qiymat:  
const { davlat = "O'zbekiston" } = user;  
console.log(davlat); // "O'zbekiston" (yo'q edi, default ishladi)
```

Destructuring funksiya parametrlarida juda foydali:

```
function userKorsat({ ism, yosh }) {  
  return `${ism}, ${yosh} yosh`;  
}  
console.log(userKorsat(user)); // "Oqil, 25 yosh"
```

Spread (...) obyektlar bilan

```
const user = { ism: "Oqil", yosh: 25 };  
  
// Nusxa olish:  
const nusxa = { ...user };  
  
// Yangi xususiyat qo'shib nusxa:  
const yangilangan = { ...user, shahar: "Toshkent" };  
  
// Birlashtirib (o'ngdagi ustun keladi):  
const birlashgan = { ...user, yosh: 26 }; // yosh 26 bo'ladi
```

Why spread: Obyektni **buzmasdan** yangi nusxa yasash uchun. Bu — funksional/immutable yondashuv. React, Vuex/Pinia state'ida doim shunday qilinadi: asil state'ni o'zgartirmasdan yangi nusxa qaytarasan.

Shorthand property (qisqa yozuv)

Kalit va o'zgaruvchi nomi bir xil bo'lsa, qisqartirish mumkin:

```
const ism = "Oqil";
const yosh = 25;

// Uzun:
const user1 = { ism: ism, yosh: yosh };

// Qisqa (shorthand):
const user2 = { ism, yosh }; // bir xil natija
```

JSON (ma'lumot almashish formati)

JSON — obyektни **matnga** aylantirib, server bilan almashish formati. API bilan ishlaganda har kuni ko'rasan:

```
const user = { ism: "Oqil", yosh: 25 };

// Obyekt -> JSON matn (serverga yuborish uchun):
const json = JSON.stringify(user);
console.log(json); // '{"ism":"Oqil","yosh":25}'

// JSON matn -> Obyekt (serverdan kelganni o'qish uchun):
const obyekt = JSON.parse(json);
console.log(obyekt.ism); // "Oqil"
```

7-modul masalalari (20 ta)

1. `user` obyektini yarating: `ism`, `yosh`, `shahar`.
2. Uning `ism` ini nuqta bilan, `yosh` ini kvadrat qavs bilan chiqaring.
3. Obyektga yangi `email` xususiyatini qo'shing.
4. `yosh` ni o'zgartiring va `shahar` ni o'chiring (`delete`).
5. Obyektga `salomBer()` metodini qo'shing (`this.ism` bilan).
6. `Object.keys` bilan barcha kalitlarni chiqaring.
7. `Object.values` bilan barcha qiymatlarni chiqaring.
8. `Object.entries` + `for...of` bilan har bir juftlikni `"kalit: qiymat"` ko'rinishida chiqaring.
9. Ichma-ich obyekt yarating (`manzil` ichida `shahar`, `tuman`) va `shahar` ga kiring.

10. `?.` (optional chaining) bilan mavjud bo'lmagan xususiyatga xavfsiz kiring.
11. 3 ta userdan iborat obyektlar massivi yarating.
12. `map` bilan ulardan faqat ismlarni ajratib oling.
13. `filter` bilan 25 dan katta yoshdagilarni oling.
14. `reduce` bilan o'rtacha yoshni hisoblang.
15. Destructuring bilan `user` dan `ism` va `yosh` ni ajratib oling.
16. Destructuringda yangi nom bering (`ism` → `nomi`).
17. `spread` bilan obyekt nusxasini oling va bitta xususiyatni o'zgartiring.
18. Shorthand property bilan obyekt yarating (`const x=1, y=2` dan).
19. Obyektni `JSON.stringify` bilan matnga, keyin `JSON.parse` bilan qayta obyektga aylantiring.
20. **Murakkab:** talabalar massivi `[{ism, ball}]` — eng yuqori ballik talabaning **ismini** toping.



► Yechimlar



```
// 1
const user = { ism: "Oqil", yosh: 25, shahar: "Toshkent" };

// 2
console.log(user.ism); // "Oqil"
console.log(user["yosh"]); // 25

// 3
user.email = "oqil@mail.uz";

// 4
user.yosh = 26;
delete user.shahar;

// 5
const u5 = {
  ism: "Oqil",
  salomBer() { return `Salom, men ${this.ism}`; },
};
console.log(u5.salomBer());

// 6
console.log(Object.keys(user)); // ["ism", "yosh", "email"]

// 7
console.log(Object.values(user));

// 8
for (const [k, v] of Object.entries(user)) console.log(`${k}: ${v}`);

// 9
const u9 = { ism: "Oqil", manzil: { shahar: "Toshkent", tuman: "Yunusobod" } };
console.log(u9.manzil.shahar); // "Toshkent"

// 10
console.log(u9.ishjoy?.nom); // undefined (xato yo'q)

// 11
const users = [
  { ism: "Ali", yosh: 20 },
  { ism: "Vali", yosh: 30 },
  { ism: "Hasan", yosh: 25 },
];

// 12 -> ["Ali", "Vali", "Hasan"]
console.log(users.map(u => u.ism));

// 13 -> [{Vali,30}]
console.log(users.filter(u => u.yosh > 25));

// 14 -> 25
console.log(users.reduce((s, u) => s + u.yosh, 0) / users.length);

// 15
const { ism, yosh } = user;
console.log(ism, yosh);
```

```

// 16
const { ism: nomi } = user;
console.log(nomi); // "Oqil"

// 17
const nusxa = { ...user, yosh: 100 };
console.log(nusxa.yosh); // 100, asil user buzilmaydi

// 18
const x = 1, y = 2;
const obj = { x, y };
console.log(obj); // {x:1, y:2}

// 19
const json = JSON.stringify(user);
const qayta = JSON.parse(json);
console.log(qayta.ism);

// 20 -> "Vali"
const talabalar = [
  { ism: "Ali", ball: 70 },
  { ism: "Vali", ball: 95 },
  { ism: "Hasan", ball: 85 },
];
const eng = talabalar.reduce((max, t) => t.ball > max.ball ? t : max);
console.log(eng.ism); // "Vali"

```

8-MODUL: String va Number metodlari

Matn (string) va son (number) bilan ishlashning tayyor vositalari. Bularni yodlash shart emas — kerak bo'lganda qarasans. Lekin eng ko'p ishlatiladiganlarini bilish kerak.

String metodlari

```

const matn = "  Salom Dunyo  ";

console.log(matn.length);           // 15 - uzunlik
console.log(matn.toUpperCase());    // "  SALOM DUNYO  "
console.log(matn.toLowerCase());    // "  salom dunyo  "
console.log(matn.trim());           // "Salom Dunyo" - chetidagi
probellarni oladi

```

Qism olish va qidirish

```

const str = "JavaScript";

```

Almashtirish va bo'lish

To'ldirish (`padStart` , `padEnd`)

Number metodlari

Why diqqat: `toFixed` **matn (string)** qaytaradi, son emas! `(5).toFixed(2)` → `"5.00"`. Agar keyin hisob qilmoqchi bo'lsang, qayta `Number(...)` ga o'rab ol.

Math obyekti

Matematik amallar uchun tayyor funksiyalar:

```
console.log(Math.round(4.5)); // 5 - eng yaqinga yaxlitlash
console.log(Math.floor(4.9)); // 4 - pastga (har doim kichraytiradi)
console.log(Math.ceil(4.1)); // 5 - yuqoriga (har doim kattalashtiradi)

console.log(Math.max(3, 9, 2)); // 9
console.log(Math.min(3, 9, 2)); // 2
console.log(Math.abs(-7)); // 7 - modul (musbat qiladi)
console.log(Math.pow(2, 10)); // 1024 - daraja (yoki 2 ** 10)
console.log(Math.sqrt(144)); // 12 - kvadrat ildiz
```

Tasodifiy son (Math.random)

Math.random() — 0 dan 1 gacha (1 kirmaydi) tasodifiy son:

```
console.log(Math.random()); // masalan 0.7234...

// 1 dan 100 gacha tasodifiy butun son:
const tasodif = Math.floor(Math.random() * 100) + 1;
console.log(tasodif);

// Umumiy formula: min dan max gacha:
function tasodifiy(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(tasodifiy(1, 6)); // zar tashlash (1-6)
```

Why formula: `Math.random() * (max - min + 1)` diapazonni kengaytiradi, `Math.floor` butun qiladi, `+ min` boshlang'ichni siljitadi. Bu formulani yodlab ol — o'yinlar, test variantlari, hamma joyda kerak.

8-modul masalalari (20 ta)

1. `"javascript"` ni katta harfga aylantiring.
2. `" salom "` chetidagi probellarni `trim` bilan oling.
3. `"Dasturlash"` so'zining uzunligini toping.
4. `"JavaScript"` dan `slice` bilan `"Java"` ni ajratib oling.
5. `"Salom Dunyo"` da `"Dunyo"` ni `"JS"` ga almashtiring.
6. `"Ali Vali Hasan"` ni `split` bilan so'zlar massiviga aylantiring.

7. `"Programming"` da `"gram"` borligini `includes` bilan tekshiring.
8. `"index.js"` `".js"` bilan tugaydimi? (`endsWith`)
9. `"7"` ni `padStart` bilan `"007"` qiling.
10. `"=".repeat(20)` bilan 20 ta `=` chiqaring.
11. `"JavaScript"` ning 0-harfini `charAt` va `[0]` bilan oling.
12. `3.14159` ni `toFixed` bilan 2 xonagacha qoldiring.
13. `Math.round`, `Math.floor`, `Math.ceil` ni `4.5` ga qo'llang.
14. 1 dan 100 gacha tasodifiy butun son chiqaring.
15. `Math.max` va `Math.min` bilan `[15, 3, 99, 27]` dan eng katta/kichikni toping.
16. `Math.abs(-42)` natijasini toping.
17. `Math.sqrt` bilan 169 ning ildizini, `Math.pow` bilan 3 ning kubini toping.
18. `parseInt("99 dollar")` va `parseFloat("19.99$")` natijalarini toping.
19. `Number.isInteger` bilan `10` va `10.5` ni tekshiring.
20. **Murakkab:** son `1234567` ni `"1 234 567 so'm"` ko'rinishida formatlang (har 3 raqamga probel). (Maslahat: `toLocaleString`)



```
// 1 -> "JAVASCRIPT"
console.log("javascript".toUpperCase());

// 2 -> "salom"
console.log("  salom  ".trim());

// 3 -> 10
console.log("Dasturlash".length);

// 4 -> "Java"
console.log("JavaScript".slice(0, 4));

// 5 -> "Salom JS"
console.log("Salom Dunyo".replace("Dunyo", "JS"));

// 6 -> ["Ali", "Vali", "Hasan"]
console.log("Ali Vali Hasan".split(" "));

// 7 -> true
console.log("Programming".includes("gram"));

// 8 -> true
console.log("index.js".endsWith(".js"));

// 9 -> "007"
console.log("7".padStart(3, "0"));

// 10
console.log("=".repeat(20));

// 11 -> "J"
console.log("JavaScript".charAt(0));
console.log("JavaScript"[0]);

// 12 -> "3.14"
console.log((3.14159).toFixed(2));

// 13
console.log(Math.round(4.5)); // 5
console.log(Math.floor(4.5)); // 4
console.log(Math.ceil(4.5)); // 5

// 14
console.log(Math.floor(Math.random() * 100) + 1);

// 15
console.log(Math.max(15, 3, 99, 27)); // 99
console.log(Math.min(15, 3, 99, 27)); // 3
// Massivdan: Math.max(...[15,3,99,27])

// 16 -> 42
console.log(Math.abs(-42));

// 17
console.log(Math.sqrt(169)); // 13
console.log(Math.pow(3, 3)); // 27
```

```
// 18
console.log(parseInt("99 dollar")); // 99
console.log(parseFloat("19.99$")); // 19.99

// 19
console.log(Number.isInteger(10)); // true
console.log(Number.isInteger(10.5)); // false

// 20 -> "1 234 567"
const son = 1234567;
console.log(son.toLocaleString("ru-RU") + " so'm"); // "1 234 567 so'm"
// Yoki qo'lda regex bilan:
console.log(son.toString().replace(/\B(?=(\d{3})+(?! \d))/g, " ") + " so'm");
```

✓ 2-qism yakuni

Endi sen real dastur qura olasan: - **Funksiyalar** bilan kodni bo'laklaysan (`return` ≠ `console.log` !) - **Massivlar** va `map` / `filter` / `reduce` bilan ma'lumotni qayta ishlaysan - **Obyektlar** bilan real dunyo ma'lumotlarini modellashtirasan - **Obyektlar massivi** + metodlar — bu API'dan keladigan ma'lumot bilan ishlashning aynan o'zi - `String`/`Number`/`Math` metodlarini ishlatasan

`map`, `filter`, `reduce`, obyekt destructuring, `spread` — bularni puxta o'zlashtir. Frontend'da (Vue/React) va backend'da (Node) har kuni shularni ishlatasan.

Keyingi qadam (3-qism)

- **DOM** — HTML sahifani JS bilan o'zgartirish (matn, rang, element qo'shish)
- **Events** — tugma bosish, forma yuborish, klaviatura
- **Forms** va validation
- **localStorage** — ma'lumotni brauzerda saqlash

Maslahat: 3-qismga o'tishdan oldin shu yerdagi 20-masalalarni (har modulning oxirigisi — eng murakkab) albatta yech. Ular real loyihaga eng yaqin masalalar.

 [README](#) · [← 1-qism — Asoslar](#) · [Keyingi: 3-qism — Brauzer \(DOM\)](#) →

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 [README](#) · [← 2-qism — Strukturalar](#) · [Keyingi: 4-qism — Asinxron JS](#) →

3-QISM: BRAUZER (DOM) Endi JS'ni HTML sahifa bilan bog'laymiz: matn o'zgartirish, tugma bosish, forma tekshirish, ma'lumot saqlash. Har bir moduldan keyin **20 ta masala** (yechimi bilan). Modullar bo'ylab **Todo-app** quramiz.

1-2 qismda toza JS (mantiq) o'rgandik. Endi shu mantiqni **ko'rinadigan natijaga** ulaymiz.

9-MODUL: DOM bilan ishlash

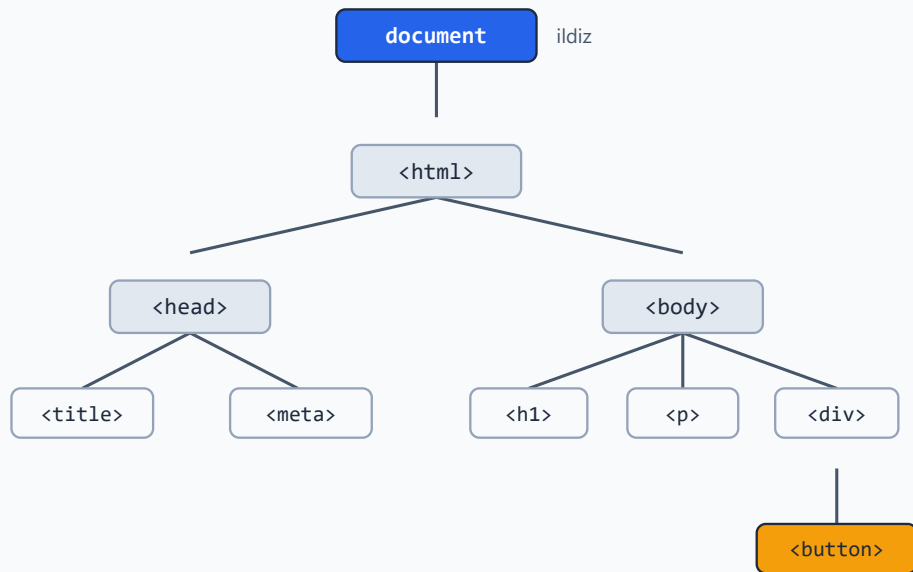
DOM nima?

DOM (Document Object Model) — brauzer sening HTML'ingni o'qib, uni **obyektlar daraxti** ko'rinishida xotirada saqlaydi. JS shu daraxt orqali sahifani o'qiydi va o'zgartiradi.

```
document
├── html
│   └── body
│       ├── h1
│       ├── p
│       └── div
│           └── button
```

Har bir HTML tegi — DOM'da bitta **node (tugun)**. JS shu node'larni topib, ularning matni, rangi, atributlarini o'zgartira oladi. `document` — butun sahifaning "ildizi".

DOM daraxti (tugunlar iyerarxiyasi)



Har bir HTML tegi -- daraxtdagi bitta tugun (node). JS shu tugunlarni topib o'zgartiradi.

Element tanlash (Selecting)

Avval elementni "ushlab olishing" kerak, keyin u bilan ishlaysan.

```
// ID bo'yicha (eng tez):
const sarlavha = document.getElementById("sarlavha");

// CSS selektor bo'yicha - BIRINCHI mosini (eng zamonaviy, eng moslashuvchan):
const tugma = document.querySelector(".btn"); // class
const div = document.querySelector("#asosiy"); // id
const birinchiLi = document.querySelector("ul li"); // ichma-ich

// CSS selektor bo'yicha - HAMMA mosini (NodeList qaytaradi):
const hammaLi = document.querySelectorAll("li");
```

Why `querySelector`: U CSS selektorlarining hammasini tushunadi (`.class`, `#id`, `div > p`, `[data-id]`). Shuning uchun zamonaviy kodda **default tanlov** — `querySelector` / `querySelectorAll`. `getElementById` biroz tezroq, lekin farqi sezilmaydi.

querySelector -- bitta tugunni tanlash

```
document.querySelector(".btn")
```



".btn" selektori bo'yicha qidiradi



querySelectorAll esa hammasini oladi

-> bitta element qaytadi

Matnni o'qish va o'zgartirish

```
const el = document.querySelector("#xabar");

// O'qish:
console.log(el.textContent); // ichidagi matn

// Yozish:
el.textContent = "Yangi matn"; // FAQAT matn (xavfsiz)
el.innerHTML = "<b>Qalin</b> matn"; // HTML sifatida talqin qiladi
```

Why – XSS xavfsizligi (muhim!): `textContent` matnni shunchaki matn deb yozadi. `innerHTML` esa ichidagi HTML teglarini **bajaradi**. Agar foydalanuvchidan kelgan ma'lumotni `innerHTML` ga qo'ysang, u `<script>` yuborib saytingni buzishi mumkin (XSS hujumi). **Qoida:** matn uchun doim `textContent`, `innerHTML` ni faqat o'zing ishonchli HTML yozganingda ishlat.

Atributlarni o'zgartirish

```
const rasm = document.querySelector("img");

rasm.src = "yangi.jpg"; // to'g'ridan-to'g'ri
rasm.setAttribute("alt", "Tavsif"); // setAttribute orqali
console.log(rasm.getAttribute("src")); // o'qish

const link = document.querySelector("a");
link.href = "https://example.com";
```

```
const input = document.querySelector("input");
console.log(input.value); // input qiymati - .value orqali (textContent
emas!)
```

Class bilan ishlash (`classList`) — eng muhim

Rang/ko'rinishni o'zgartirish uchun **inline style emas**, **class** ishlat (CSS'da uslub aniqlangan bo'ladi):

```
const el = document.querySelector(".karta");

el.classList.add("faol"); // class qo'shadi
el.classList.remove("faol"); // class olib tashlaydi
el.classList.toggle("faol"); // bor bo'lsa oladi, yo'q bo'lsa
qo'shadi
el.classList.contains("faol"); // bormi? -> true/false
```

Why class, style emas: `el.style.color = "red"` — uslubni JS'ga aralashtiradi.
`el.classList.add("xato")` — uslub CSS'da qoladi, JS faqat "qaysi holatda" ekanini boshqaradi. Bu — mas'uliyatni ajratish (separation of concerns). Toggle bilan dark mode, active tab kabi narsalar bir qatorda hal bo'ladi.

Style'ni to'g'ridan-to'g'ri o'zgartirish (kamroq)

```
const el = document.querySelector("#quti");
el.style.color = "white";
el.style.backgroundColor = "blue"; // CSS: background-color -> JS:
backgroundColor (camelCase)
el.style.display = "none"; // yashirish
```

Yangi element yaratish va qo'shish

Bu — ma'lumotdan ro'yxat yasashning asosi:

```
// 1. Element yarat
const yangiLi = document.createElement("li");

// 2. Ichini to'ldir
yangiLi.textContent = "Yangi vazifa";
yangiLi.classList.add("vazifa");

// 3. Sahifaga qo'sh
const royxat = document.querySelector("ul");
```

```
royxat.appendChild(yangiLi); // oxiriga qo'shadi
// royxat.prepend(yangiLi); // boshiga qo'shadi
```

Element o'chirish

```
const el = document.querySelector(".vazifa");
el.remove(); // o'zini o'chiradi
```

Daraxt bo'ylab harakat (Traversal)

```
const el = document.querySelector(".farzand");

el.parentElement; // ota element
el.children; // bolalari (HTMLCollection)
el.nextElementSibling; // keyingi qo'shni
el.previousElementSibling; // oldingi qo'shni
```

Massivdan ro'yxat yasash (eng amaliy namuna)

```
const vazifalar = ["O'qish", "Yozish", "Mashq qilish"];
const ul = document.querySelector("#vazifalar");

vazifalar.forEach(vazifa => {
  const li = document.createElement("li");
  li.textContent = vazifa;
  ul.appendChild(li);
});
```

Why diqqat — performance: Sikl ichida har safar DOM'ga `appendChild` qilish — ko'p elementda sekin. Katta ro'yxatda `innerHTML` ni bir marta yig'ib qo'yish yoki `DocumentFragment` ishlatish tezroq. Hozircha kichik ro'yxatlar uchun bu yetarli — lekin bu tushunchani esda tut.

9-modul masalalari (20 ta)

Quyidagi HTML asosini ishlatib (`.html` fayl yaratib, `<body>` ichiga qo'ying):

```
<h1 id="sarlavha">Salom</h1>
<p class="matn">Bu paragraf</p>

```

```
<ul id="royxat"><li>Bir</li><li>Ikki</li></ul>
<input id="kiritish" type="text">
<div id="quti">Quti</div>
<script src="app.js"></script>
```

1. `getElementById` bilan `#sarlavha` ni tanlab, matnini "Yangilandi" qiling.
2. `querySelector` bilan `.matn` ni tanlang va matnini o'qing.
3. `querySelectorAll` bilan barcha `<li>` larni oling va sonini chiqaring.
4. `#sarlavha` ning `textContent` ini konsolga chiqaring.
5. `#quti` ichiga `innerHTML` bilan `<b>Qalin</b>` qo'shing.
6. `textContent` va `innerHTML` farqini bitta misolda ko'rsating.
7. `#quti` ga `classList.add` bilan `"faol"` class qo'shing.
8. `#quti` da `classList.toggle("faol")` ni 2 marta chaqirib natijani kuzating.
9. `classList.contains` bilan `#quti` da `"faol"` borligini tekshiring.
10. `#quti` ning matn rangini `style.color` bilan qizil qiling.
11. `#rasm` ga `setAttribute` bilan yangi `alt` bering.
12. `#rasm` ning `src` ini "yangi.jpg" ga o'zgartiring.
13. `createElement` + `appendChild` bilan `#royxat` ga uchinchi `<li>` qo'shing.
14. `#royxat` boshiga `prepend` bilan `<li>` qo'shing.
15. Birinchi `<li>` ni `remove()` bilan o'chiring.
16. Birinchi `<li>` ning `parentElement` ini topib, uning teg nomini chiqaring.
17. `querySelectorAll("li")` + `forEach` bilan har bir `<li>` ga `"element"` class qo'shing.
18. `#kiritish` input qiymatini (`value`) konsolga chiqaring.
19. `createElement` bilan `<div>` yasab, ichiga `<h3>` va `<p>` joylang (card).
20. **Todo qadami:** `["O'qish", "Yozish", "Kodlash"]` massividan `#royxat` ichida `<li>` lar yasab chiqaring (render funksiyasi).



```
// 1
document.getElementById("sarlavha").textContent = "Yangilandi";

// 2
console.log(document.querySelector(".matn").textContent);

// 3
console.log(document.querySelectorAll("li").length); // 2

// 4
console.log(document.querySelector("#sarlavha").textContent);

// 5
document.querySelector("#quti").innerHTML = "<b>Qalin</b>";

// 6
const q = document.querySelector("#quti");
q.textContent = "<b>matn</b>"; // ekranda <b>matn</b> ko'rinadi (teg
bajarilmaydi)
q.innerHTML = "<b>matn</b>"; // ekranda matn QALIN bo'lib chiqadi

// 7
document.querySelector("#quti").classList.add("faol");

// 8 -> 1-chaqiriq qo'shadi, 2-chaqiriq oladi (natija: yo'q)
const qu = document.querySelector("#quti");
qu.classList.toggle("faol"); // qo'shildi
qu.classList.toggle("faol"); // olindi

// 9
console.log(document.querySelector("#quti").classList.contains("faol"));

// 10
document.querySelector("#quti").style.color = "red";

// 11
document.querySelector("#rasm").setAttribute("alt", "Yangi tavsif");

// 12
document.querySelector("#rasm").src = "yangi.jpg";

// 13
const li13 = document.createElement("li");
li13.textContent = "Uch";
document.querySelector("#royxat").appendChild(li13);

// 14
const li14 = document.createElement("li");
li14.textContent = "No1";
document.querySelector("#royxat").prepend(li14);

// 15
document.querySelector("#royxat li").remove();

// 16 -> "UL"
console.log(document.querySelector("#royxat li").parentElement.tagName);
```

```
// 17
document.querySelectorAll("li").forEach(li => li.classList.add("element"));

// 18
console.log(document.querySelector("#kiritish").value);

// 19
const card = document.createElement("div");
const h3 = document.createElement("h3");
h3.textContent = "Sarlavha";
const p = document.createElement("p");
p.textContent = "Tavsif matni";
card.appendChild(h3);
card.appendChild(p);
document.body.appendChild(card);

// 20 - render funksiyasi
function render(vazifalar) {
  const ul = document.querySelector("#royxat");
  ul.innerHTML = ""; // avval tozalaymiz
  vazifalar.forEach(v => {
    const li = document.createElement("li");
    li.textContent = v;
    ul.appendChild(li);
  });
}
render(["O'qish", "Yozish", "Kodlash"]);
```

10-MODUL: Events (hodisalar)

Event nima?

Event — sahifada sodir bo'ladigan **hodisa**: tugma bosildi, matn yozildi, sichqoncha harakatlandi, forma yuborildi. JS shu hodisalarni "tinglab", javob beradi.

addEventListener — hodisani tinglash

```
const tugma = document.querySelector("#tugma");

tugma.addEventListener("click", function() {
  console.log("Tugma bosildi!");
});

// Arrow bilan (zamonaviy):
tugma.addEventListener("click", () => {
  alert("Salom!");
});
```

Sintaksis: `element.addEventListener("hodisa_nomi", funksiya)`. Bosilganda funksiya ishlaydi.

Why `addEventListener`, `onclick` emas: HTML'da `<button onclick="...">` yozma. `addEventListener` ning afzalligi: bitta elementga **bir nechta** listener ulay olasan, HTML'ni JS'dan ajratasan, va `removeEventListener` bilan o'chira olasan. Bu — toza yondashuv.

Ko'p ishlatiladigan hodisalar

```
el.addEventListener("click", fn); // bosish
el.addEventListener("dblclick", fn); // ikki marta bosish
el.addEventListener("input", fn); // matn yozilganda (har harfda)
el.addEventListener("change", fn); // qiymat o'zgarib, fokus
ketganda
el.addEventListener("submit", fn); // forma yuborilganda
el.addEventListener("keydown", fn); // klavisha bosilganda
el.addEventListener("keyup", fn); // klavisha qo'yib yuborilganda
el.addEventListener("mouseover", fn); // sichqoncha ustiga kelganda
el.addEventListener("mouseout", fn); // sichqoncha chiqib ketganda
el.addEventListener("focus", fn); // input fokus olganda
el.addEventListener("blur", fn); // input fokus yo'qotganda
```

Event obykti (`e`)

Listener funksiyasiga brauzer **hodisa haqida ma'lumot** beradi — buni odatda `e` yoki `event` deb nomlaymiz:

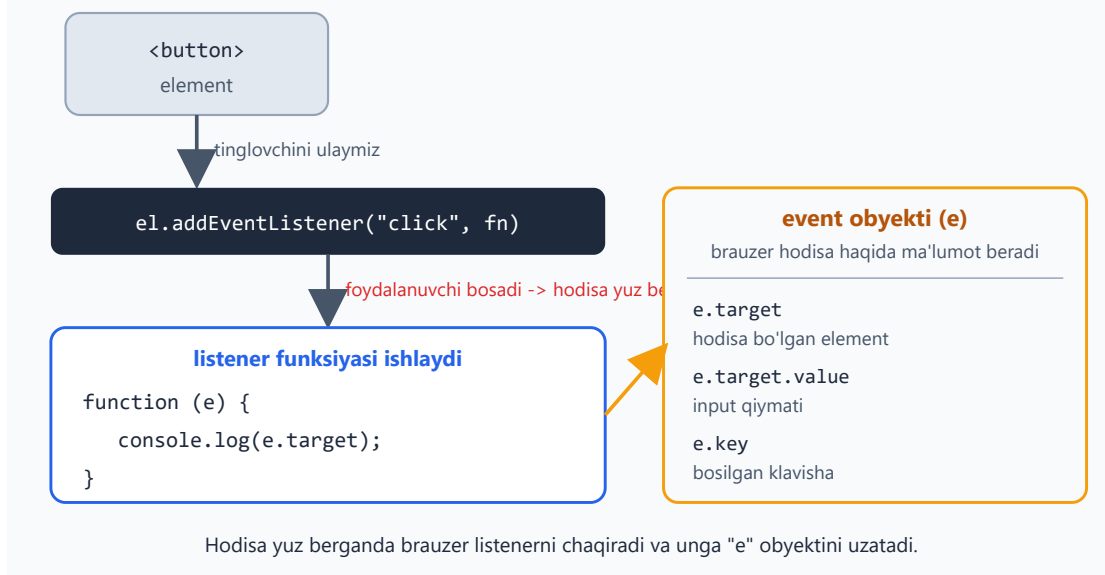
```
const input = document.querySelector("#kiritish");

input.addEventListener("input", (e) => {
  console.log(e.target); // hodisa sodir bo'lgan element
  console.log(e.target.value); // input ichidagi matn (har yozilganda)
});

document.addEventListener("keydown", (e) => {
  console.log(e.key); // bosilgan klavisha (masalan "Enter", "a",
  "ArrowUp")
});
```

`e` ning eng muhim qismlari: - `e.target` — hodisa sodir bo'lgan element. - `e.target.value` — input/textarea/select qiymati. - `e.key` — bosilgan klavisha (klaviatura hodisalarida). - `e.preventDefault()` — brauzerning standart xatti-harakatini to'xtatadi.

addEventListener va event obykti (e)



preventDefault() — standart harakatni to'xtatish

```
const link = document.querySelector("a");  
link.addEventListener("click", (e) => {  
  e.preventDefault(); // link sahifaga o'tmaydi  
  console.log("Bosildi, lekin o'tmadik");  
});  
  
// Eng ko'p ishlatiladigan joy — forma (sahifa qayta yuklanmasligi uchun):  
form.addEventListener("submit", (e) => {  
  e.preventDefault(); // sahifa yangilanmaydi  
  // o'z mantiqing...  
});
```

Why: Forma yuborilganda brauzer odatda sahifani **qayta yuklaydi** (eski usul). SPA (Vue/React) va AJAX'da bu kerak emas — `e.preventDefault()` shu standart yuklanishni to'xtatadi, sen o'zing JS bilan ish ko'rasan.

Hisoblagich misoli (DOM + Event birga)

```
<p id="son">0</p>  
<button id="qosh">+1</button>  
<script>  
  let hisob = 0;  
  const sonEl = document.querySelector("#son");
```

```
document.querySelector("#qosh").addEventListener("click", () => {
  hisob++;
  sonEl.textContent = hisob; // ekranni yangilaymiz
});
</script>
```

Event Delegation (muhim pattern)

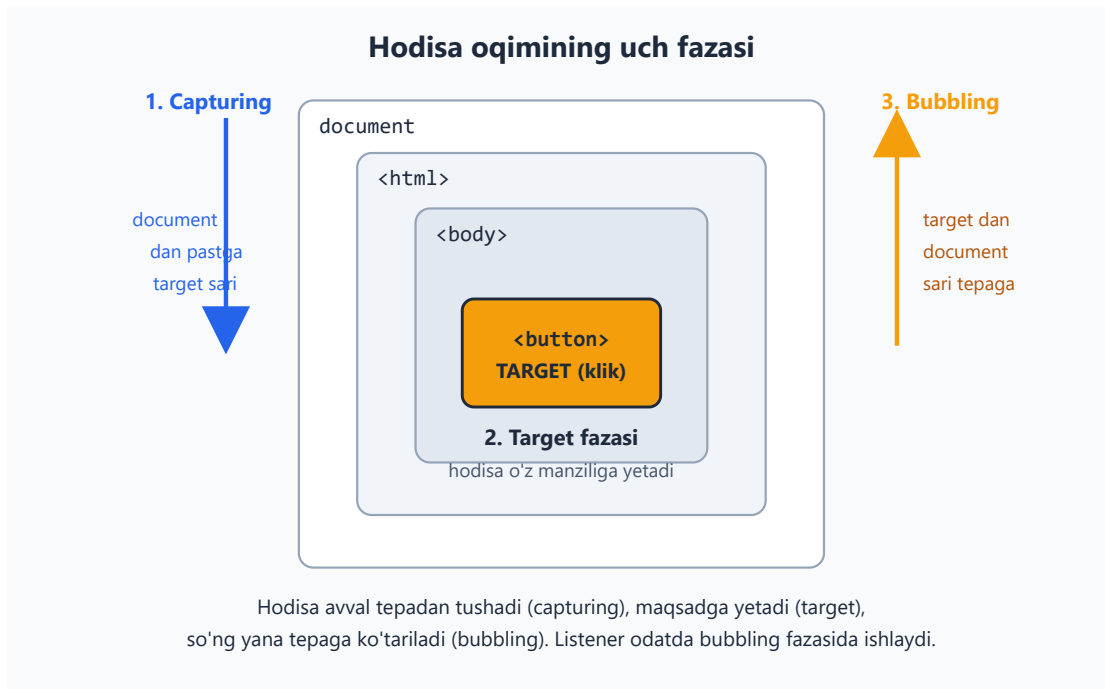
Ko'p elementga alohida listener ulash o'rniga, **ota elementga bitta** listener ulab, `e.target` orqali qaysi bola bosilganini aniqlaysan:

```
const royxat = document.querySelector("#royxat");

royxat.addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    e.target.classList.toggle("bajarildi");
  }
});
```

Why delegation: Bu — eng kuchli pattern. Sabablari: (1) keyin **dinamik qo'shilgan** elementlar ham avtomatik ishlaydi (yangi `<li>` ga alohida listener kerak emas); (2) 100 ta listener o'rniga **bitta** — tez va kam xotira. Todo-app, jadval, ro'yxat — hammasida shuni ishlat.

Delegation aynan shu sababli ishlaydi: bola element bosilganda hodisa undan ota elementlar sari **tepaga ko'tariladi** (bubbling). Aslida har hodisa uch fazadan o'tadi — avval tepadan tushadi (capturing), maqsadga yetadi (target), so'ng yana tepaga ko'tariladi (bubbling):



Bir nechta elementga listener (`forEach`)

```
const tugmalar = document.querySelectorAll(".tugma");
tugmalar.forEach(tugma => {
  tugma.addEventListener("click", () => {
    console.log("Bosildi:", tugma.textContent);
  });
});
```

`data-*` atributlari (`dataset`)

Elementga maxsus ma'lumot biriktirish uchun:

```
<button data-id="42" data-amal="ochir">O'chirish</button>
```

```
tugma.addEventListener("click", (e) => {
  console.log(e.target.dataset.id); // "42"
  console.log(e.target.dataset.amal); // "ochir"
});
```

Why `data-*`: Element bilan birga "qaysi yozuvga tegishli" degan ma'lumotni saqlaydi. Todo-app'da har `<li>` ga `data-id` qo'yib, o'chirishda qaysi vazifani o'chirishni bilasan.

10-modul masalalari (20 ta)

HTML asosi:

```
<h1 id="natija">Natija</h1>
<button id="btn">Bosing</button>
<input id="kiritish" type="text">
<select id="tanlov"><option>Bir</option><option>Ikki</option>
</select>
<input type="checkbox" id="belgi">
<ul id="royxat"><li data-id="1">A</li><li data-id="2">B</li></ul>
<div class="tugmalar"><button>X</button><button>Y</button></div>
```

1. `#btn` bosilganda konsolga "Bosildi" chiqaring.
2. `#btn` bosilganda `#natija` matnini "Tugma bosildi" qiling.
3. Hisoblagich: `#btn` har bosilganda `#natija` da son 1 ga oshsin.
4. `#kiritish` ga yozilganda (`input`) yozilgan matn `#natija` da jonli ko'rinsin.
5. `e.target.value` ni `input` hodisasida olib chiqaring.
6. `keydown` : bosilgan klavishani (`e.key`) `#natija` da ko'rsating.
7. `#kiritish` da Enter bosilganda "Yuborildi" chiqaring.
8. `#tanlov` (`change`) o'zgarganda tanlangan qiymatni chiqaring.
9. `#belgi` checkbox holatini (`checked`) bosilganda chiqaring.
10. `#btn` ga `mouseover` / `mouseout` da rangini o'zgartiring.
11. `#btn` bosilganda `#natija` ni `display:none` / `block` bilan yashiring/ko'rsating (toggle).
12. Linkka `preventDefault` qo'llab, bosilganda o'tmasligini ta'minlang.
13. `dblclick` : `#natija` ikki marta bosilganda matnini tozalang.
14. **Event delegation**: `#royxat` ga listener qo'yib, bosilgan `<li>` ga `"faol"` class qo'shing.
15. `.tugmalar` ichidagi har bir tugmaga `forEach` bilan listener qo'shing (matnini chiqarsin).
16. `#royxat` `<li>` bosilganda `e.target.dataset.id` ni chiqaring.
17. `removeEventListener` bilan listener'ni bir martadan keyin o'chiring.
18. `#kiritish` `focus` / `blur` da border rangini o'zgartiring (`classList`).
19. `keydown` bilan ArrowUp/ArrowDown bosilganda son oshsin/kamaysin.
20. **Todo qadami**: `#kiritish` ga vazifa yozib `#btn` bosilganda, vazifani massivga qo'shib `#royxat` ni qayta render qiling.



```
// 1
document.querySelector("#btn").addEventListener("click", () =>
  console.log("Bosildi"));

// 2
document.querySelector("#btn").addEventListener("click", () => {
  document.querySelector("#natija").textContent = "Tugma bosildi";
});

// 3
let hisob = 0;
document.querySelector("#btn").addEventListener("click", () => {
  hisob++;
  document.querySelector("#natija").textContent = hisob;
});

// 4
document.querySelector("#kiritish").addEventListener("input", (e) => {
  document.querySelector("#natija").textContent = e.target.value;
});

// 5
document.querySelector("#kiritish").addEventListener("input", (e) => {
  console.log(e.target.value);
});

// 6
document.addEventListener("keydown", (e) => {
  document.querySelector("#natija").textContent = e.key;
});

// 7
document.querySelector("#kiritish").addEventListener("keydown", (e) => {
  if (e.key === "Enter") console.log("Yuborildi");
});

// 8
document.querySelector("#tanlov").addEventListener("change", (e) => {
  console.log(e.target.value);
});

// 9
document.querySelector("#belgi").addEventListener("change", (e) => {
  console.log(e.target.checked); // true/false
});

// 10
const btn = document.querySelector("#btn");
btn.addEventListener("mouseover", () => btn.style.backgroundColor = "blue");
btn.addEventListener("mouseout", () => btn.style.backgroundColor = "");

// 11
document.querySelector("#btn").addEventListener("click", () => {
  const n = document.querySelector("#natija");
  n.style.display = n.style.display === "none" ? "block" : "none";
});
```

```

// 12
document.querySelector("a")?.addEventListener("click", (e) => {
  e.preventDefault();
  console.log("O'tmadik");
});

// 13
document.querySelector("#natija").addEventListener("dblclick", (e) => {
  e.target.textContent = "";
});

// 14 - event delegation
document.querySelector("#royxat").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") e.target.classList.toggle("faol");
});

// 15
document.querySelectorAll(".tugmalar button").forEach(b => {
  b.addEventListener("click", () => console.log(b.textContent));
});

// 16
document.querySelector("#royxat").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") console.log(e.target.dataset.id);
});

// 17
function birMarta() {
  console.log("Faqat bir marta");
  document.querySelector("#btn").removeEventListener("click", birMarta);
}
document.querySelector("#btn").addEventListener("click", birMarta);
// (yoki: addEventListener("click", fn, { once: true }))

// 18
const inp = document.querySelector("#kiritish");
inp.addEventListener("focus", () => inp.classList.add("faol"));
inp.addEventListener("blur", () => inp.classList.remove("faol"));

// 19
let qiymat = 0;
document.addEventListener("keydown", (e) => {
  if (e.key === "ArrowUp") qiymat++;
  if (e.key === "ArrowDown") qiymat--;
  document.querySelector("#natija").textContent = qiymat;
});

// 20 - Todo qo'shish
const vazifalar = [];
function render() {
  const ul = document.querySelector("#royxat");
  ul.innerHTML = "";
  vazifalar.forEach(v => {
    const li = document.createElement("li");
    li.textContent = v;
    ul.appendChild(li);
  });
}
document.querySelector("#btn").addEventListener("click", () => {
  const input = document.querySelector("#kiritish");
  if (input.value.trim() !== "") {

```

```
vazifalar.push(input.value.trim());  
input.value = "";  
render();  
}  
});
```

11-MODUL: Forms va validation

Forma — foydalanuvchidan ma'lumot olishning asosiy yo'li. JS bilan uni tekshiramiz (validation) va qayta ishlaymiz.

Forma qiymatlarini olish

```
<form id="forma">  
  <input id="ism" type="text">  
  <input id="parol" type="password">  
  <input id="rozilik" type="checkbox">  
  <select id="shahar">  
    <option value="tsh">Toshkent</option>  
    <option value="smq">Samarqand</option>  
  </select>  
  <button type="submit">Yuborish</button>  
</form>
```

```
const ism = document.querySelector("#ism").value;      // matn  
const parol = document.querySelector("#parol").value;  
const rozimi = document.querySelector("#rozilik").checked; //  
true/false  
const shahar = document.querySelector("#shahar").value; // "tsh"  
yoki "smq"
```

Eslab qol: Matn maydonlari uchun `.value`, checkbox/radio uchun `.checked`.

submit hodisasi + preventDefault

```
const forma = document.querySelector("#forma");  
  
forma.addEventListener("submit", (e) => {  
  e.preventDefault(); // sahifa qayta yuklanmaydi  
  
  const ism = document.querySelector("#ism").value;
```

```
console.log("Yuborildi:", ism);
});
```

Why submit, click emas: Listener'ni tugmaga emas, **formaga submit** ga ulash kerak. Sababi: foydalanuvchi Enter bosib ham forma yuborishi mumkin — **submit** ikkalasini ham ushlaydi. `e.preventDefault()` siz sahifa yangilanib, hamma ma'lumot yo'qoladi.

Oddiy validatsiya

```
forma.addEventListener("submit", (e) => {
  e.preventDefault();

  const ism = document.querySelector("#ism").value.trim();
  const parol = document.querySelector("#parol").value;

  // Bo'sh tekshiruv:
  if (ism === "") {
    console.log("Ism kiritilmagan");
    return; // davom etmaymiz
  }

  // Uzunlik tekshiruvi:
  if (parol.length < 6) {
    console.log("Parol kamida 6 ta belgi bo'lsin");
    return;
  }

  console.log("Hammasi to'g'ri!");
});
```

Email tekshiruvi

```
const email = document.querySelector("#email").value;

// Oddiy (includes bilan):
if (!email.includes("@") || !email.includes(".")) {
  console.log("Email noto'g'ri");
}

// Aniqroq (regex bilan — 23-modulda chuqur ko'ramiz):
const pattern = /^[^s@]+@[^s@]+\.[^s@]+$/;
if (!pattern.test(email)) {
  console.log("Email formati xato");
}
```

Xato xabarini ekranda ko'rsatish

Konsol emas, foydalanuvchiga ko'rinadigan xabar:

```
<input id="ism" type="text">  
<span id="ismXato" class="xato"></span>
```

```
const ismXato = document.querySelector("#ismXato");  
  
if (ism === "") {  
  ismXato.textContent = "Ism majburiy";  
  document.querySelector("#ism").classList.add("xato-border");  
} else {  
  ismXato.textContent = "";  
  document.querySelector("#ism").classList.remove("xato-border");  
}
```

Real-time validatsiya (`input` hodisasida)

Foydalanuvchi yozar ekan tekshirish (UX yaxshilanadi):

```
const parol = document.querySelector("#parol");  
const xabar = document.querySelector("#parolXato");  
  
parol.addEventListener("input", () => {  
  if (parol.value.length < 6) {  
    xabar.textContent = "Yana " + (6 - parol.value.length) + " ta belgi  
kerak";  
  } else {  
    xabar.textContent = "✓ Yaxshi";  
  }  
});
```

FormData (barcha qiymatlarni bir vaqtda)

Ko'p maydonli formada qulay:

```
forma.addEventListener("submit", (e) => {  
  e.preventDefault();  
  const data = new FormData(forma);  
  
  console.log(data.get("ism")); // name="ism" maydonidan  
  // Obyektga aylantirish:  
  const obyekt = Object.fromEntries(data);
```

```
console.log(obyekt); // { ism: "...", parol: "..." }
});
```

Eslatma: `FormData` ishlashi uchun input'larda `name` atributi bo'lishi kerak (`<input name="ism">`).

11-modul masalalari (20 ta)

HTML asosi:

```
<form id="forma">
  <input id="ism" name="ism" type="text">
  <input id="email" name="email" type="email">
  <input id="parol" name="parol" type="password">
  <input id="tasdiq" name="tasdiq" type="password">
  <input id="yosh" name="yosh" type="number">
  <input id="rozilik" type="checkbox">
  <select id="shahar"><option value="tsh">Toshkent</option></select>
  <span id="xato" class="xato"></span>
  <button type="submit">Yuborish</button>
</form>
```

1. Formaga `submit` listener qo'shib, `preventDefault` qiling.
2. `#ism` qiymatini submitda olib chiqaring.
3. `#ism` bo'sh bo'lsa "Ism majburiy" deb `#xato` da ko'rsating.
4. `#parol` 6 belgidan qisqa bo'lsa xato bering.
5. `#email` da `@` yo'q bo'lsa xato bering.
6. Email'ni regex bilan tekshiring (`test`).
7. `#rozilik` belgilanmagan bo'lsa "Shartlarga rozi bo'ling" deng.
8. `#shahar` tanlangan qiymatini chiqaring.
9. `#yosh` 0 dan kichik yoki bo'sh bo'lsa xato bering.
10. `#parol` va `#tasdiq` mos kelmasa "Parollar mos emas" deng.
11. Xato bo'lsa input'ga `"xato-border"` class qo'shing, to'g'ri bo'lsa oling.
12. Hamma to'g'ri bo'lsa "Muvaffaqiyatli!" deb ko'rsatib, formani tozalang (`reset`).
13. `FormData` bilan barcha qiymatlarni obyekt qilib oling.
14. `#parol` ga real-time (`input`) validatsiya qo'shing.
15. Submit tugmasini `#rozilik` belgilanmaguncha `disabled` qiling.
16. `#yosh` faqat son ekanini tekshiring (`isNaN`).

17. `#ism` kamida 3 ta belgi bo'lsin (trim'dan keyin).
18. Telefon `" +998 "` bilan boshlanishini tekshiring (`startsWith`).
19. Barcha xatolarni **bir vaqtda** yig'ib, ro'yxat qilib ko'rsating (massiv).
20. **To'liq forma:** `ism(≥3)`, `email(regex)`, `parol(≥6)`, `tasdiq(mos)` – hammasini tekshirib, faqat to'liq to'g'ri bo'lsa "Ro'yxatdan o'tdingiz" deng.



```
const forma = document.querySelector("#forma");
const xato = document.querySelector("#xato");

// 1
forma.addEventListener("submit", (e) => e.preventDefault());

// 2
forma.addEventListener("submit", (e) => {
  e.preventDefault();
  console.log(document.querySelector("#ism").value);
});

// 3
function tekshir3() {
  const ism = document.querySelector("#ism").value.trim();
  if (ism === "") { xato.textContent = "Ism majburiy"; return false; }
  return true;
}

// 4
const parol = document.querySelector("#parol");
if (parol.value.length < 6) xato.textContent = "Parol ≥ 6 belgi";

// 5
const email = document.querySelector("#email");
if (!email.value.includes("@")) xato.textContent = "Email noto'g'ri";

// 6
const pattern = /^[^s@]+@[^s@]+\.[^s@]+$/;
if (!pattern.test(email.value)) xato.textContent = "Email formati xato";

// 7
if (!document.querySelector("#rozilik").checked)
  xato.textContent = "Shartlarga rozi bo'ling";

// 8
console.log(document.querySelector("#shahar").value);

// 9
const yosh = document.querySelector("#yosh").value;
if (yosh === "" || Number(yosh) <= 0) xato.textContent = "Yosh noto'g'ri";

// 10
const tasdiq = document.querySelector("#tasdiq").value;
if (parol.value !== tasdiq) xato.textContent = "Parollar mos emas";

// 11
function belgi(el, xatomi) {
  if (xatomi) el.classList.add("xato-border");
  else el.classList.remove("xato-border");
}

// 12
function muvaffaqiyat() {
  xato.textContent = "Muvaffaqiyatli!";
  forma.reset();
}
```

```

// 13
forma.addEventListener("submit", (e) => {
  e.preventDefault();
  const data = Object.fromEntries(new FormData(forma));
  console.log(data); // { ism, email, parol, tasdiq, yosh }
});

// 14
parol.addEventListener("input", () => {
  xato.textContent = parol.value.length < 6
    ? `Yana ${6 - parol.value.length} ta belgi`
    : "✓ Yaxshi";
});

// 15
const rozilik = document.querySelector("#rozilik");
const btn = forma.querySelector("button");
rozilik.addEventListener("change", () => {
  btn.disabled = !rozilik.checked;
});

// 16
if (isNaN(Number(yosh)) || yosh === "") xato.textContent = "Faqat son";

// 17
if (document.querySelector("#ism").value.trim().length < 3)
  xato.textContent = "Ism ≥ 3 belgi";

// 18 (telefon maydonini #ism deb faraz qildik)
const tel = "+998901234567";
if (!tel.startsWith("+998")) xato.textContent = "+998 bilan boshlansin";

// 19 - barcha xatolarni yig'ish
function hammaXatolar() {
  const xatolar = [];
  const ism = document.querySelector("#ism").value.trim();
  if (ism.length < 3) xatolar.push("Ism ≥ 3 belgi");
  if (!pattern.test(email.value)) xatolar.push("Email noto'g'ri");
  if (parol.value.length < 6) xatolar.push("Parol ≥ 6 belgi");
  return xatolar;
}

// 20 - to'liq forma
forma.addEventListener("submit", (e) => {
  e.preventDefault();
  const xatolar = [];
  const ism = document.querySelector("#ism").value.trim();
  const em = document.querySelector("#email").value;
  const pa = document.querySelector("#parol").value;
  const ta = document.querySelector("#tasdiq").value;

  if (ism.length < 3) xatolar.push("Ism kamida 3 ta belgi");
  if (!pattern.test(em)) xatolar.push("Email formati noto'g'ri");
  if (pa.length < 6) xatolar.push("Parol kamida 6 ta belgi");
  if (pa !== ta) xatolar.push("Parollar mos kelmadi");

  if (xatolar.length > 0) {
    xato.innerHTML = xatolar.map(x => `• ${x}`).join("<br>");
  } else {
    xato.textContent = "Ro'yxatdan o'tdingiz ✓";
  }
});

```

```
    forma.reset();  
  }  
});
```

12-MODUL: localStorage (ma'lumot saqlash)

Web Storage nima?

`localStorage` — brauzerda **ma'lumot saqlash** imkoniyati. Sahifa yangilangandan keyin ham, hatto brauzer yopilib qayta ochilsa ham ma'lumot saqlanib qoladi. Hisoblagich, theme tanlovi, savatcha — shu yerda saqlanadi.

```
// Saqlash:  
localStorage.setItem("ism", "Oqil");  
  
// O'qish:  
const ism = localStorage.getItem("ism"); // "Oqil"  
  
// O'chirish:  
localStorage.removeItem("ism");  
  
// Hammasini tozalash:  
localStorage.clear();
```

localStorage VS sessionStorage

```
localStorage.setItem("a", "1"); // brauzer yopilsa ham QOLADI  
sessionStorage.setItem("b", "2"); // tab/oyna yopilsa O'CHADI
```

	localStorage	sessionStorage
Yashash muddati	Doimiy (qo'lda o'chirilguncha)	Tab yopilguncha
Hajmi	~5-10 MB	~5-10 MB
Ishlatilishi	Theme, til, savatcha	Vaqtinchalik holat

⚠ Faqat matn saqlaydi!

localStorage **faqat string** saqlaydi. Son saqlasang ham, qaytib string bo'lib chiqadi:

```
localStorage.setItem("yosh", 25);  
const yosh = localStorage.getItem("yosh");  
console.log(yosh); // "25" (string!)  
console.log(typeof yosh); // "string"  
console.log(Number(yosh) + 1); // 26 - qayta songa aylantirish kerak
```

Obyekt/massiv saqlash (JSON bilan)

Obyekt yoki massivni saqlash uchun avval **matnga** aylantirasan (7-modulda ko'rgan JSON):

```
const user = { ism: "Oqil", yosh: 25 };  
  
// Saqlash: obyekt -> JSON matn  
localStorage.setItem("user", JSON.stringify(user));  
  
// O'qish: JSON matn -> obyekt  
const saqlangan = JSON.parse(localStorage.getItem("user"));  
console.log(saqlangan.ism); // "Oqil"
```

```
// Massiv ham xuddi shunday:  
const vazifalar = ["O'qish", "Yozish"];  
localStorage.setItem("vazifalar", JSON.stringify(vazifalar));  
const olingan = JSON.parse(localStorage.getItem("vazifalar"));
```

Why diqqat: localStorage.setItem("user", user) deb to'g'ridan-to'g'ri obyekt bersang, u "[object Object]" bo'lib saqlanadi (buzilgan). Obyekt/massiv uchun **doim** JSON.stringify / JSON.parse ishlat.

Mavjudligini tekshirish

getItem topa olmasa null qaytaradi:

```
const ism = localStorage.getItem("ism");  
if (ism === null) {  
  console.log("Hali saqlanmagan");  
} else {  
  console.log("Salom, " + ism);  
}  
  
// Yoki qisqa (default bilan):  
const til = localStorage.getItem("til") || "uz";
```

Amaliy: Dark mode (theme) saqlash

```
<button id="theme">Theme almashtirish</button>
<script>
  // Sahifa yuklanganda saqlangan theme'ni qo'llash:
  const saqlangan = localStorage.getItem("theme");
  if (saqlangan === "dark") {
    document.body.classList.add("dark");
  }

  document.querySelector("#theme").addEventListener("click", () => {
    document.body.classList.toggle("dark");
    // Holatni saqlash:
    const hozir = document.body.classList.contains("dark") ? "dark" :
"light";
    localStorage.setItem("theme", hozir);
  });
</script>
```

Why — xavfsizlik eslatmasi: `localStorage` ga **maxfiy ma'lumot saqlama** (parol, JWT token, bank ma'lumotlari). Sababi: u oddiy matn ko'rinishida turadi va har qanday JS (jumladan XSS orqali kirgan zararli kod) uni o'qiy oladi. Faqat zararsiz, foydalanuvchi qulayligi uchun ma'lumotlarni saqla (theme, til, UI holati).

12-modul masalalari (20 ta)

1. `localStorage.setItem` bilan `"ism"` ga o'z isimingni saqlang.
2. `getItem` bilan uni o'qib, konsolga chiqaring.
3. `removeItem` bilan `"ism"` ni o'chiring.
4. `clear()` bilan hammasini tozalang.
5. `"ism"` saqlanmagan bo'lsa (`null`) `"Mehmon"` deb chiqaring.
6. Son `100` ni saqlang, o'qing, songa aylantirib `+50` qiling.
7. `{ism:"Oqil", yosh:25}` obyektini `JSON.stringify` bilan saqlang.
8. Uni `JSON.parse` bilan qaytarib o'qing va `.ism` ni chiqaring.
9. `["a", "b", "c"]` massivini saqlang va o'qing.
10. Hisoblagich: tugma bosilganda son oshib, `localStorage` da saqlansin (sahifa yangilansa ham qolsin).
11. Theme toggle: dark/light holatini saqlang.
12. Sahifa yuklanganda saqlangan theme'ni `body` ga qo'llang.
13. Ism saqlang; keyingi tashrifda "Yana xush kelibsiz, X" deng.

14. Saqlangan massivga yangi element qo'shib, qayta saqlang.
15. Saqlangan massivdan element o'chirib, qayta saqlang.
16. `"til"` saqlanmagan bo'lsa default `"uz"` ni ishlatib (`|| "uz"`).
17. `sessionStorage` ga ma'lumot saqlab, `localStorage` dan farqini tushuntiring.
18. Oxirgi tashrif vaqtini (`new Date().toISOString()`) saqlang va ko'rsating.
19. `localStorage` da nechta kalit borligini (`localStorage.length`) chiqaring.
20. **TO'LIQ TODO-APP:** input + tugma + ro'yxat. Vazifa qo'shish, o'chirish (delegation), va `localStorage` da saqlash (sahifa yangilansa ham qolsin).



```
// 1
localStorage.setItem("ism", "Oqil");

// 2
console.log(localStorage.getItem("ism")); // "Oqil"

// 3
localStorage.removeItem("ism");

// 4
localStorage.clear();

// 5
console.log(localStorage.getItem("ism") ?? "Mehmon");

// 6 -> 150
localStorage.setItem("son", 100);
console.log(Number(localStorage.getItem("son")) + 50);

// 7
localStorage.setItem("user", JSON.stringify({ ism: "Oqil", yosh: 25 }));

// 8 -> "Oqil"
console.log(JSON.parse(localStorage.getItem("user")).ism);

// 9
localStorage.setItem("harflar", JSON.stringify(["a", "b", "c"]));
console.log(JSON.parse(localStorage.getItem("harflar")));

// 10 - saqlanadigan hisoblagich
let hisob = Number(localStorage.getItem("hisob")) || 0;
document.querySelector("#btn").addEventListener("click", () => {
  hisob++;
  localStorage.setItem("hisob", hisob);
  document.querySelector("#natija").textContent = hisob;
});

// 11 & 12 - theme
if (localStorage.getItem("theme") === "dark")
  document.body.classList.add("dark");
document.querySelector("#theme").addEventListener("click", () => {
  document.body.classList.toggle("dark");
  localStorage.setItem("theme", document.body.classList.contains("dark") ?
"dark" : "light");
});

// 13
const eski = localStorage.getItem("ism");
if (eski) console.log(`Yana xush kelibsiz, ${eski}`);
localStorage.setItem("ism", "Oqil");

// 14
const arr14 = JSON.parse(localStorage.getItem("harflar")) || [];
arr14.push("d");
localStorage.setItem("harflar", JSON.stringify(arr14));

// 15
```

```

let arr15 = JSON.parse(localStorage.getItem("harflar")) || [];
arr15 = arr15.filter(x => x !== "b"); // "b" ni o'chir
localStorage.setItem("harflar", JSON.stringify(arr15));

// 16
const til = localStorage.getItem("til") || "uz";
console.log(til);

// 17
sessionStorage.setItem("vaqtinchalik", "123");
// Farqi: tab yopilsa o'chadi; localStorage qoladi.

// 18
localStorage.setItem("oxirgiTashrif", new Date().toISOString());
console.log(localStorage.getItem("oxirgiTashrif"));

// 19
console.log(localStorage.length);

// 20 - TO'LIQ TODO-APP (alohida pastda)

```

20-masala: To'liq Todo-app (HTML + JS)

```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>Todo App</title>
  <style>
    body { font-family: sans-serif; max-width: 400px; margin: 40px auto; }
    li { cursor: pointer; padding: 6px; list-style: none; }
    li.bajarildi { text-decoration: line-through; color: gray; }
    .ochir { color: red; margin-left: 10px; }
  </style>
</head>
<body>
  <h1>Vazifalar</h1>
  <input id="kiritish" type="text" placeholder="Yangi vazifa...">
  <button id="qosh">Qo'shish</button>
  <ul id="royxat"></ul>

  <script>
    // localStorage'dan yuklab olamiz (bo'lmasa bo'sh massiv)
    let vazifalar = JSON.parse(localStorage.getItem("vazifalar")) || [];

    const ul = document.querySelector("#royxat");
    const input = document.querySelector("#kiritish");

    // Saqlash funksiyasi
    function saqla() {
      localStorage.setItem("vazifalar", JSON.stringify(vazifalar));
    }

    // Render funksiyasi
    function render() {
      ul.innerHTML = "";
      vazifalar.forEach((v, index) => {
        const li = document.createElement("li");
        li.textContent = v.matn;

```

```

        if (v.bajarildi) li.classList.add("bajarildi");
        li.dataset.index = index;

        const ochir = document.createElement("span");
        ochir.textContent = "X";
        ochir.className = "ochir";
        ochir.dataset.ochir = index;
        li.appendChild(ochir);

        ul.appendChild(li);
    });
}

// Qo'shish
document.querySelector("#qosh").addEventListener("click", () => {
    const matn = input.value.trim();
    if (matn === "") return;
    vazifalar.push({ matn, bajarildi: false });
    input.value = "";
    saqla();
    render();
});

// Enter bilan ham qo'shish
input.addEventListener("keydown", (e) => {
    if (e.key === "Enter") document.querySelector("#qosh").click();
});

// Event delegation: bosilganda bajarildi/o'chirish
ul.addEventListener("click", (e) => {
    // O'chirish tugmasi bosildimi?
    if (e.target.dataset.ochir !== undefined) {
        vazifalar.splice(Number(e.target.dataset.ochir), 1);
    }
    // Vazifa matni bosildimi -> bajarildi toggle
    else if (e.target.tagName === "LI") {
        const i = Number(e.target.dataset.index);
        vazifalar[i].bajarildi = !vazifalar[i].bajarildi;
    }
    saqla();
    render();
});

// Boshlang'ich render
render();
</script>
</body>
</html>

```

✓ 3-qism yakuni

Endi sen **haqiqiy interaktiv sahifa** qura olasan: - **DOM** bilan elementlarni tanlash, matn/atribut/class o'zgartirish, yangi element yasash - **Events** bilan tugma, klaviatura, forma hodisalariga javob berish - **Event delegation** — eng muhim pattern (dinamik

elementlar uchun) - **Forms** validatsiyasi va `FormData` - **localStorage** bilan ma'lumotni saqlash (obyekt — `JSON` orqali)

Va eng muhimi — bularning hammasini birlashtirib **to'liq Todo-app** yozding. Bu — frontend'ning "Hello World"i: render → event → state o'zgarishi → qayta render → saqlash. Aynan shu sikl Vue/React'da ham ishlaydi (faqat ular buni avtomatlashtiradi).

Keyingi qadam (4-qism) — Asinxron JS

Bu — eng muhim va eng chalkash qism. Server bilan gaplashish: - **Callbacks** va Event Loop (JS qanday "kutadi") - **Promises** — `.then()`, `.catch()` - **async / await** — zamonaviy, toza usul - **Fetch API** — backend'dan ma'lumot olish (real API'lar bilan)

Maslahat: 4-qismga o'tishdan oldin Todo-app'ni o'zing noldan, qarab emas, **yoddan yozib ko'r**. Agar yoza olsang — DOM, events va state'ni tushunding degani.

 [README](#) · [← 2-qism — Strukturalar](#) · [Keyingi: 4-qism — Asinxron JS](#) →

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 README · [← 3-qism — Brauzer \(DOM\)](#) · Keyingi: [5-qism \(1-bo'lim\) — Chuqur JS](#) →

4-QISM: ASINXRON JS JS qanday "kutadi", server bilan qanday gaplashadi. Bu — eng muhim va eng ko'p chalkashtiradigan qism. Har bir moduldan keyin **20 ta masala** (yechimi bilan).

1-3 qismda JS darhol ishlaydigan kod yozdik. Lekin real hayotda biz **kutishimiz** kerak: serverdan javob, fayl yuklanishi, taymer. Bu — asinxron dunyosi.

Avval: JS bitta yo'lakli (single-threaded)

JS bir vaqtning o'zida **faqat bitta** ishni bajaradi. U "bitta oshpaz" kabi — bir vaqtda bir taom tayyorlaydi.

Muammo: agar serverdan ma'lumot so'rasak va u 3 soniya kutsa, JS shu 3 soniya **butun sahifani muzlatib** turishi kerakmi? Yo'q! Foydalanuvchi tugma bosa olmaydi, hech narsa ishlamaydi — bu dahshat.

Yechim: JS uzoq vaqt talab qiladigan ishni (taymer, server so'rovi) **fonga** beradi va keyingi koddan davom etadi. Ish tugagach, natijani qayta oladi. Mana shu — **asinxronlik**.

13-MODUL: Callbacks va Event Loop

`setTimeout` — kechiktirish

Eng oddiy asinxron misol — biror amalni keyinroq bajarish:

```
console.log("Boshlandi");

setTimeout(() => {
  console.log("2 soniyadan keyin");
}, 2000); // 2000 ms = 2 soniya

console.log("Tugadi");
```

```
// Natija:  
// Boshlandi  
// Tugadi          <- bu DARHOL chiqadi (kutmaydi!)  
// 2 soniyadan keyin <- 2 soniyadan so'ng
```

Why diqqat: `setTimeout` JS'ni **to'xtatib qo'ymaydi**. U "2 soniyadan keyin shu funksiyani ishga tushir" deb fonga beradi va keyingi qatorga o'tadi. Shuning uchun "Tugadi" "2 soniyadan keyin"dan oldin chiqadi.

`setInterval` — takroriy bajarish

```
let son = 0;  
const id = setInterval(() => {  
  son++;  
  console.log(son);  
  if (son === 5) clearInterval(id); // 5 da to'xtatamiz  
}, 1000); // har 1 soniyada
```

`clearInterval(id)` / `clearTimeout(id)` — taymerni to'xtatadi. Shuning uchun taymerning `id` sini saqlab qol.

Callback nima?

Callback — boshqa funksiyaga **argument sifatida uzatilgan** funksiya. "Ish tugagach, mana buni chaqir" degani. `setTimeout` ning birinchi argumenti — callback.

```
function salomBer(ism, callback) {  
  console.log(`Salom, ${ism}`);  
  callback(); // ish tugagach chaqiramiz  
}  
  
salomBer("Oqil", () => {  
  console.log("Callback ishladi");  
});
```

5-modulda "higher-order function" ko'rgan eding — callback aynan shu. JS'da funksiya — qiymat, shuning uchun uni uzatish mumkin.

Error-first callback (Node.js patterni)

Eski JS/Node kodida xato birinchi argument sifatida uzatiladi:

```
function malumotOl(callback) {  
  setTimeout(() => {
```

```

    const xato = null;
    const data = { ism: "Oqil" };
    callback(xato, data); // (xato, natija)
  }, 1000);
}

malumotOl((err, data) => {
  if (err) {
    console.log("Xato:", err);
    return;
  }
  console.log("Natija:", data);
});

```

⚠ Callback Hell (callback do'zaxi)

Ketma-ket asinxron amallar kerak bo'lsa, callbacklar ichma-ich joylashib ketadi — o'qib bo'lmaydigan "piramida":

```

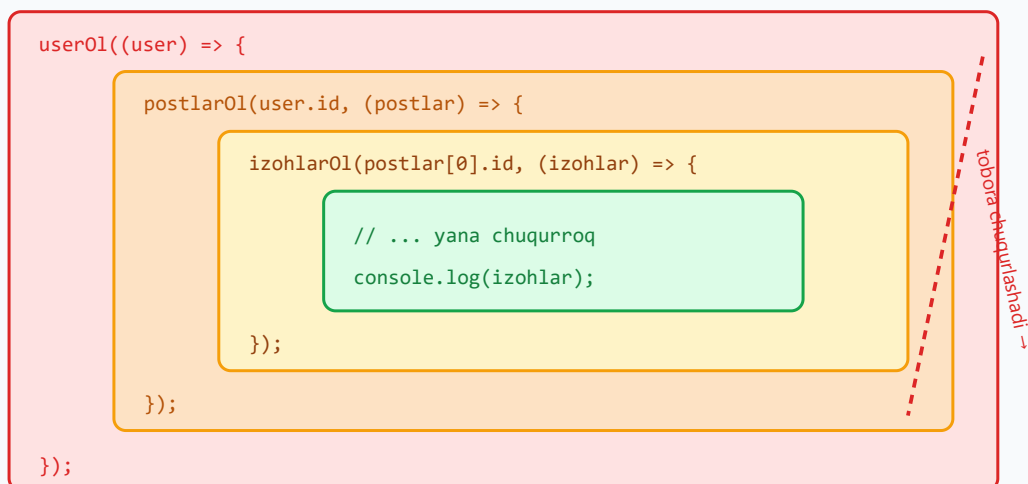
malumotOl((err, user) => {
  postlarOl(user.id, (err, postlar) => {
    izohlarOl(postlar[0].id, (err, izohlar) => {
      // ... yana chuqurroq
      console.log(izohlar);
    });
  });
});

```

Why muammo: Bu kod o'ngga "qiyshayib" ketadi, xatolarni boshqarish takrorlanadi, o'qish qiyin. Aynan shu muammoni hal qilish uchun **Promises** (14-modul) va **async/await** (15-modul) yaratilgan. Callbacklarni tushunish kerak, lekin yangi kodda Promise ishlat.

Callback Hell (callback do'zaxi)

Har bosqich oldingisining ichida — kod o'ngga "qiyshayadi"



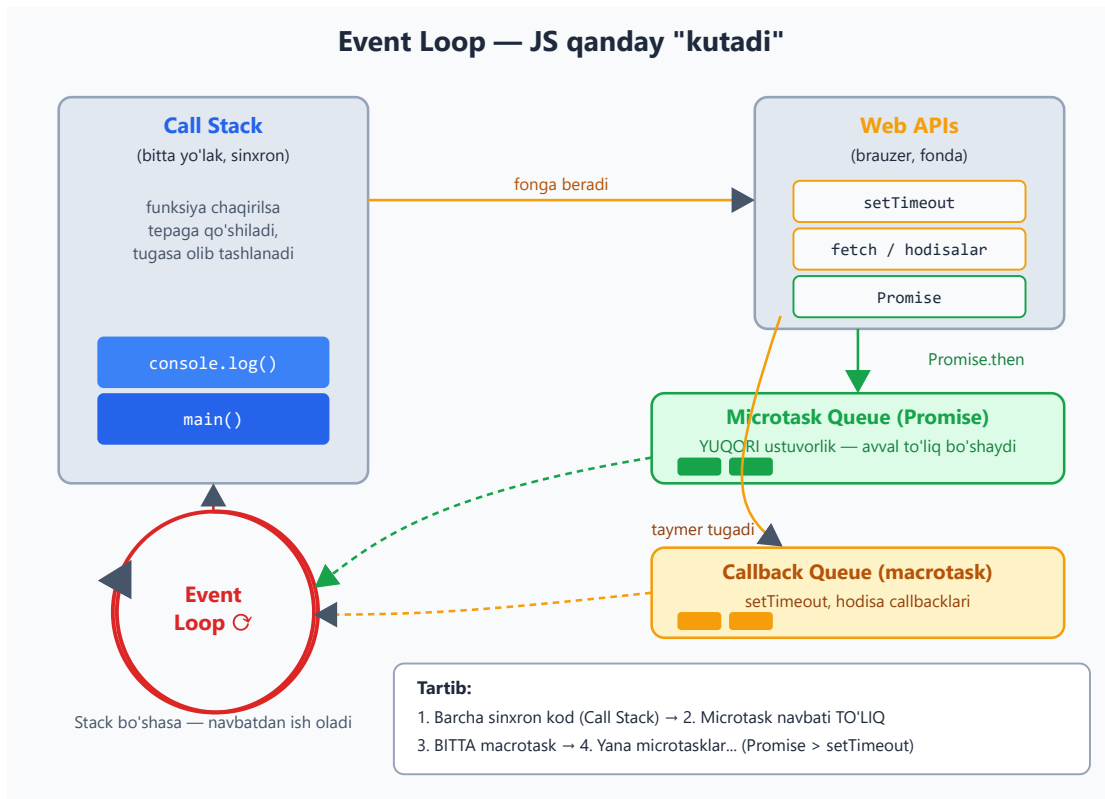
O'qish qiyin, xato boshqaruvi takrorlanadi — yechim: Promise va async/await

🗨 Event Loop — JS qanday "kutadi"

Bu — JS'ning eng muhim va eng chalkash tushunchasi. Diqqat bilan o'qi.

JS quyidagi qismlardan iborat: 1. **Call Stack** — joriy bajarilayotgan kod (sinxron). Bitta yo'lak. 2. **Web APIs** — brauzer beradigan "fon ishchilar": `setTimeout`, `fetch`, hodisalar. Ular **alohida** ishlaydi. 3. **Callback Queue** (macrotask) — `setTimeout`, hodisa callbacklari navbati. 4. **Microtask Queue** — **Promise** callbacklari navbati (yuqori ustuvorlikka ega). 5. **Event Loop** — "nazoratchi": Call Stack bo'shasa, navbatdan ish oladi.

Qoidalar: - Avval **barcha sinxron kod** ishlaydi (Call Stack). - Stack bo'shagach, Event Loop **avval Microtask navbatini to'liq** bo'shatadi (Promiselar). - Keyin **bitta** Macrotask (`setTimeout`) oladi. - Yana microtasklarni bo'shatadi, va hokazo.



Klassik misol (intervyularda ko'p so'raladi):

```
console.log("1");

setTimeout(() => console.log("2"), 0); // macrotask

Promise.resolve().then(() => console.log("3")); // microtask

console.log("4");

// Natija: 1, 4, 3, 2
```

Nima uchun shunday:

- 1) "1" – sinxron, darhol
- 2) setTimeout -> Web API ga ketadi, callback navbatga qo'yiladi (macrotask)
- 3) Promise.then -> microtask navbatga
- 4) "4" – sinxron, darhol
- sinxron kod tugadi, Call Stack bo'sh ---
- 5) Microtask navbati: "3" chiqadi (Promise macrotask'dan ustun!)
- 6) Macrotask navbati: "2" chiqadi

Why bu muhim: Bu tushuncha "nega Promise setTimeout'dan oldin ishladi?" degan savolga javob beradi. Microtask (Promise) doim macrotask (setTimeout)'dan oldin

bajariladi. `setTimeout(fn, 0)` ham "darhol" emas — sinxron kod va microtasklardan keyin ishlaydi.

Klassik tuzoq: `var` sikl ichida (`let` bilan tuzatish)

```
// ❌ var bilan — hammasi 3 chiqaradi:
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 3, 3, 3
}

// ✅ let bilan — to'g'ri:
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0, 1, 2
}
```

Why: `var` butun siklda **bitta** `i` ni ishlatadi. Taymer ishlaganda sikl allaqachon tugagan (`i=3`). `let` esa har takrorda **yangi** `i` yaratadi. Bu — `var` ni ishlatmaslikning yana bir sababi (1-modulda aytilgandek).

13-modul masalalari (20 ta)

1. `setTimeout` bilan 2 soniyadan keyin "Salom" chiqaring.
2. "Boshlandi", `setTimeout("o'rtada", 0)`, "Tugadi" — qaysi tartibda chiqadi?
3. `console.log("1")`, `setTimeout(..."2"... , 1000)`, `console.log("3")` tartibini bashorat qiling.
4. `setInterval` bilan har soniyada son chiqaring.
5. `clearInterval` bilan 5 ta sanagandan keyin to'xtating.
6. `setTimeout` ni ishga tushirib, darhol `clearTimeout` bilan bekor qiling.
7. `salomBer(ism, callback)` funksiyasi yozing — salom aytib, callbackni chaqirsin.
8. `[1,2,3]` har elementi uchun callback chaqiruvchi `harBiri(arr, cb)` yozing.
9. Ketma-ket 3 ta `setTimeout` (callback ichida callback) yozing.
10. Funksiya ishini tugatib, callbackni chaqirsin (natijani uzatib).
11. **Event loop:** 1, `setTimeout` 2, `Promise.then` 3, 4 — tartibni yozing.
12. Uchta `setTimeout` (300ms, 100ms, 200ms) — qaysi tartibda chiqadi?
13. `setTimeout` zanjiri bilan 1 dan 3 gacha har soniyada sanang.
14. Soxta fetch: `setTimeout` ichida `{ism:"Oqil"}` ni callbackka uzating.
15. Error-first callback: `cb(err, data)` patternini ishlatib, xato holatini boshqaring.

16. Sinxron va asinxron kod farqini bitta misolda ko'rsating.
17. `var` bilan sikl ichida `setTimeout` muammosini ko'rsating, keyin `let` bilan tuzating.
18. `queueMicrotask` bilan microtask qo'shing va `setTimeout` dan oldin chiqishini ko'ring.
19. **Callback hell:** 3 darajali ichma-ich callback yozing (user → postlar → izohlar).
20. **Murakkab:** 18-masaladagi callback hell'ni yozib, har bosqichni izohlang. (Keyingi modulda buni Promise bilan tozalaymiz.)



► Yechimlar



```
// 1
setTimeout(() => console.log("Salom"), 2000);

// 2 -> Boshlandi, Tugadi, o'rtada
console.log("Boshlandi");
setTimeout(() => console.log("o'rtada"), 0);
console.log("Tugadi");

// 3 -> 1, 3, 2
console.log("1");
setTimeout(() => console.log("2"), 1000);
console.log("3");

// 4
let s4 = 0;
const id4 = setInterval(() => console.log(++s4), 1000);
// (to'xtatish uchun: clearInterval(id4))

// 5
let s5 = 0;
const id5 = setInterval(() => {
  console.log(++s5);
  if (s5 === 5) clearInterval(id5);
}, 1000);

// 6
const t6 = setTimeout(() => console.log("ishlamaydi"), 1000);
clearTimeout(t6); // bekor qilindi

// 7
function salomBer(ism, callback) {
  console.log(`Salom, ${ism}`);
  callback();
}
salomBer("Oqil", () => console.log("tugadi"));

// 8
function harBiri(arr, cb) {
  for (const el of arr) cb(el);
}
harBiri([1, 2, 3], n => console.log(n * 10));

// 9
setTimeout(() => {
  console.log("1");
  setTimeout(() => {
    console.log("2");
    setTimeout(() => console.log("3"), 500);
  }, 500);
}, 500);

// 10
function ishBajar(callback) {
  const natija = 2 + 2;
  callback(natija);
}
ishBajar(r => console.log("Natija:", r)); // 4
```

```

// 11 -> 1, 4, 3, 2
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");

// 12 -> 100, 200, 300 (vaqt bo'yicha, kod tartibida emas)
setTimeout(() => console.log("300"), 300);
setTimeout(() => console.log("100"), 100);
setTimeout(() => console.log("200"), 200);

// 13
function sana(n) {
  if (n > 3) return;
  setTimeout(() => {
    console.log(n);
    sana(n + 1);
  }, 1000);
}
sana(1);

// 14
function soxtaFetch(callback) {
  setTimeout(() => callback({ ism: "Oqil" }), 1000);
}
soxtaFetch(data => console.log(data));

// 15
function malumotOl(cb) {
  setTimeout(() => {
    const xato = null; // xato bo'lsa: "Server xatosi"
    cb(xato, { ism: "Oqil" });
  }, 1000);
}
malumotOl((err, data) => {
  if (err) return console.log("Xato:", err);
  console.log("Data:", data);
});

// 16
console.log("Sinxron 1");
setTimeout(() => console.log("Asinxron"), 0);
console.log("Sinxron 2");
// Sinxron 1, Sinxron 2, Asinxron

// 17
for (var i = 0; i < 3; i++) setTimeout(() => console.log("var:", i), 10); //
3,3,3
for (let j = 0; j < 3; j++) setTimeout(() => console.log("let:", j), 10); //
0,1,2

// 18 -> microtask setTimeout'dan oldin
setTimeout(() => console.log("macrotask"), 0);
queueMicrotask(() => console.log("microtask"));
// microtask, macrotask

// 19 & 20 - callback hell
function userOl(cb) { setTimeout(() => cb({ id: 1 }), 500); }
function postlarOl(userId, cb) { setTimeout(() => cb([ { id: 10 } ]), 500); }
function izohlarOl(postId, cb) { setTimeout(() => cb([ "Zo'r!" ]), 500); }

```

```

user01((user) => {
  postlar01(user.id, (postlar) => {
    izohlar01(postlar[0].id, (izohlar) => {
      console.log(izohlar); // ["Zo'r!"]
      // Diqqat: kod o'ngga qiyshayib ketdi -> callback hell
    });
  });
});

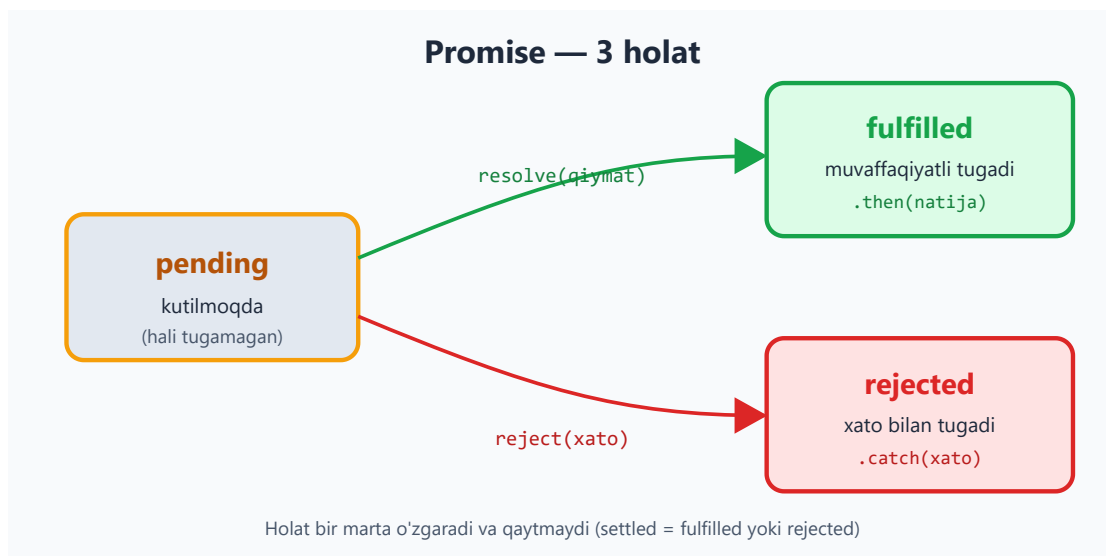
```

14-MODUL: Promises

Promise nima?

Promise ("va'da") — kelajakda **tugaydigan amal natijasini** ifodalovchi obyekt. "Hozir natija yo'q, lekin keyinroq beraman (yoki xato chiqaraman)" degan va'da.

Promise 3 holatda bo'ladi: - **pending** — kutilmoqda (hali tugamagan) - **fulfilled** — muvaffaqiyatli tugadi (`resolve` chaqirildi) - **rejected** — xato bilan tugadi (`reject` chaqirildi)



Promise yaratish

```

const vada = new Promise((resolve, reject) => {
  // asinxron ish...
  const muvaffaqiyat = true;

  if (muvaffaqiyat) {

```

```
    resolve("Hammasi yaxshi"); // -> fulfilled
  } else {
    reject("Xato yuz berdi"); // -> rejected
  }
});
```

`.then()`, `.catch()`, `.finally()`

Promise natijasini olish:

```
vada
  .then(natija => {
    console.log("Muvaffaqiyat:", natija); // resolve qiymati
  })
  .catch(xato => {
    console.log("Xato:", xato); // reject qiymati
  })
  .finally(() => {
    console.log("Har holda ishlaydi"); // ikkalasida ham
  });
```

- `.then` — promise muvaffaqiyatli bo'lsa.
- `.catch` — xato bo'lsa.
- `.finally` — natijadan qat'i nazar (tozalash uchun, masalan loading'ni o'chirish).

Amaliy: `kechikish` funksiyasi (bularni mashqlarda ishlatamiz)

```
function kechikish(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

kechikish(2000).then(() => console.log("2 soniya o'tdi"));
```

Soxta API (tarmoqsiz mashq qilish uchun):

```
function soxtaApi(data, ms = 1000) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(data), ms);
  });
}

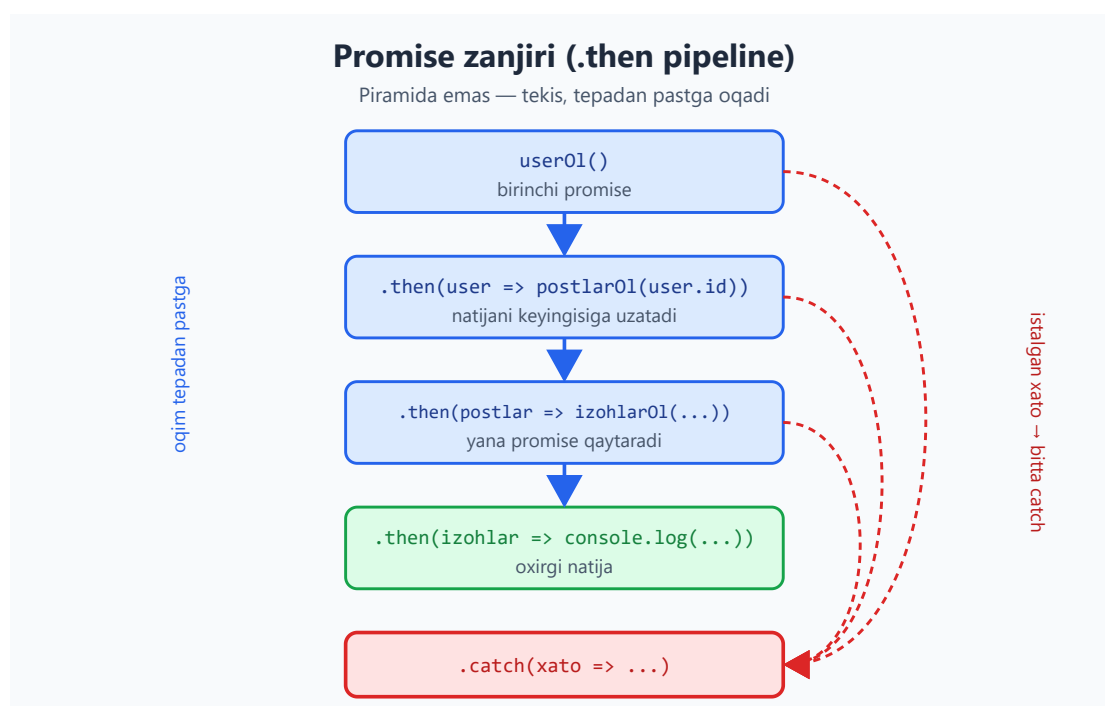
soxtaApi({ ism: "Oqil" }).then(user => console.log(user));
```

Promise zanjiri (Chaining) — callback hell'ning yechimi

`.then` ichida **yana promise qaytarsang**, ular ketma-ket bajariladi — piramida emas, **tekis** zanjir:

```
user01()
  .then(user => postlar01(user.id)) // user.id ni keyingisiga uzatadi
  .then(postlar => izohlar01(postlar[0].id))
  .then(izohlar => console.log(izohlar))
  .catch(xato => console.log("Xato:", xato)); // BITTA catch hammasini
ushlaydi
```

Why zanjir yaxshiroq: Callback hell'dagi o'ngga qiyshayish yo'qoladi — kod tepadan pastga, o'qish oson. Eng muhimi: bitta `.catch` butun zanjirdagi har qanday xatoni ushlaydi (callbacklarda har bosqichda alohida tekshirish kerak edi).



`.then` ichida qiymat o'zgartirish

Har `.then` qaytargan qiymat keyingisiga uzatiladi:

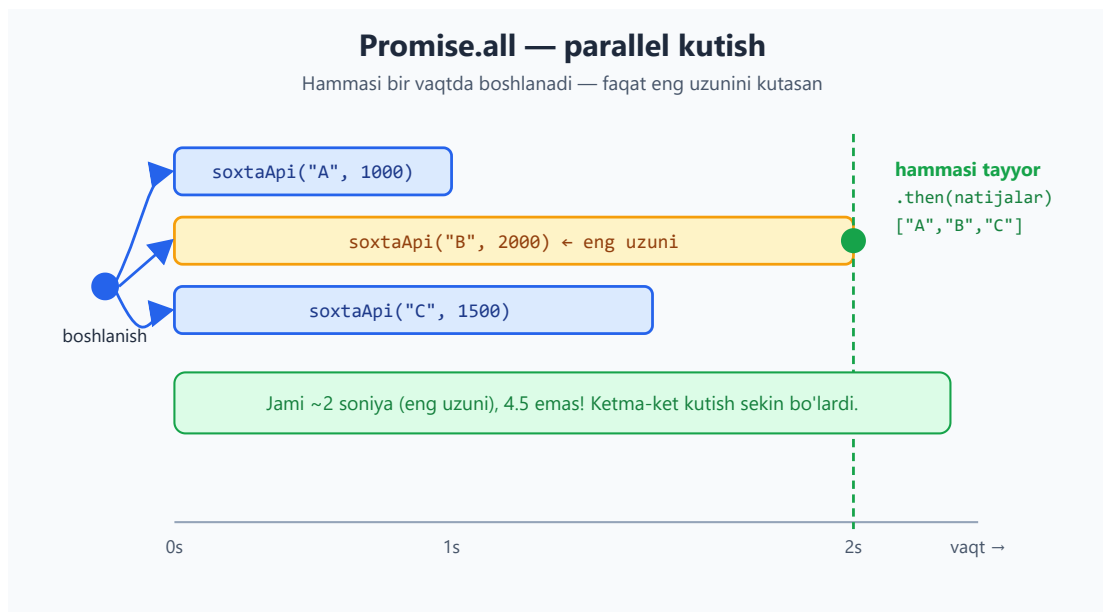
```
soxtaApi(5)
  .then(x => x * 2) // 10
  .then(x => x + 1) // 11
  .then(x => console.log(x)); // 11
```

Promise.all — bir nechtasini parallel kutish

Bir nechta promise'ni **bir vaqtda** ishga tushirib, hammasi tugashini kutish:

```
Promise.all([
  soxtaApi("A", 1000),
  soxtaApi("B", 2000),
  soxtaApi("C", 1500),
]).then(natijalar => {
  console.log(natijalar); // ["A", "B", "C"] - massiv
});
// Jami ~2 soniya (eng uzuni), 4.5 emas! Chunki PARALLEL.
```

Why Promise.all: Agar 3 ta mustaqil so'rov bo'lsa, ularni ketma-ket kutish (`await` birin-ketin) sekin — vaqtlar qo'shiladi. `Promise.all` hammasini **bir vaqtda** boshlaydi, faqat eng uzunini kutasan. Backend'ga bir nechta mustaqil so'rov yuborganda buni ishlat.



Diqqat: `Promise.all` — agar **bittasi** xato bersa, **butun** zanjir rad etiladi (fail-fast).

Promise.allSettled — hammasi (xato bo'lsa ham)

Hatto ba'zilari xato bersa ham, hammasining natijasini ko'rmoqchi bo'lsang:

```
Promise.allSettled([
  soxtaApi("A"),
  Promise.reject("Xato!"),
]).then(natijalar => {
  console.log(natijalar);
  // [{status:"fulfilled", value:"A"}, {status:"rejected",
```

```
reason: "Xato!" } ]
});
```

Promise.race — birinchi tugagani

Qaysi promise birinchi tugasa (resolve YOKI reject), o'shani oladi:

```
Promise.race([
  soxtaApi("Tez", 500),
  soxtaApi("Sekin", 3000),
]).then(natija => console.log(natija)); // "Tez"
```

Promise.race ning amaliy ishlatilishi — **timeout**: API yoki vaqt tugashi, qaysi biri oldin kelsa:

```
function timeout(ms) {
  return new Promise( (_, reject) =>
    setTimeout(() => reject("Vaqt tugadi"), ms)
  );
}

Promise.race([soxtaApi("data", 5000), timeout(2000)])
  .then(d => console.log(d))
  .catch(e => console.log(e)); // "Vaqt tugadi" (2s < 5s)
```

Eslatma: Promise.any ham bor — birinchi **muvaffaqiyatli** tugaganini oladi (rejectlarni e'tiborsiz qoldiradi). race esa birinchi tugaganni (xato bo'lsa ham) oladi.

14-modul masalalari (20 ta)

Mashqlar uchun: `const kechikish = ms => new Promise(r => setTimeout(r, ms));` va `const soxtaApi = (data, ms=1000) => new Promise(r => setTimeout(() => r(data), ms));`

1. `resolve` bilan oddiy Promise yarating va `.then` bilan oling.
2. `reject` bilan Promise yarating va `.catch` bilan ushlang.
3. `.then`, `.catch`, `.finally` uchchalasini ham ishlatib.
4. `kechikish(ms)` funksiyasini o'zingiz yozing.
5. `kechikish(1000).then(...)` bilan 1 soniyadan keyin xabar chiqaring.
6. Shartli Promise: tasodifiy `Math.random() > 0.5` bo'lsa resolve, aks holda reject.
7. `Promise.resolve(42)` ni `.then` bilan oling.

8. `soxtaApi({ism:"Oqil"})` natijasini oling.
9. `.then` zanjiri: 5 ni olib, `*2`, `+1`, keyin chiqaring.
10. `.then` ichida yangi promise qaytarib, zanjir hosil qiling.
11. `Promise.all` bilan 3 ta `soxtaApi` ni parallel oling.
12. `Promise.all` da bittasi reject bo'lsa nima bo'ladi — sinab ko'ring.
13. `Promise.race` bilan tez va sekin orasidan tezni oling.
14. `Promise.allSettled` bilan muvaffaqiyat + xatoni birga oling.
15. Zanjirning o'rtasida xato tashlab (`throw`), oxirgi `.catch` da ushlang.
16. Promise zanjirida user → postlar → natijani uzating (callback hell'siz).
17. `timeout(ms)` funksiyasi yozing (ms dan keyin reject qiladigan).
18. `Promise.race([soxtaApi(..5000..), timeout(2000)])` bilan timeout patternini quring.
19. Ketma-ket (sequential) 3 ta promise: birini kutib, keyingisini boshlang.
20. **Murakkab:** 3 ta foydalanuvchini (`soxtaApi`) **parallel** olib, ularning ismlarini bitta massivga yig'ing.



```
const kechikish = ms => new Promise(r => setTimeout(r, ms));
const soxtaApi = (data, ms = 1000) => new Promise(r => setTimeout(() =>
r(data), ms));

// 1
new Promise(resolve => resolve("OK")).then(x => console.log(x));

// 2
new Promise( (_, reject) => reject("Xato")).catch(e => console.log(e));

// 3
soxtaApi("data")
  .then(d => console.log("then:", d))
  .catch(e => console.log("catch:", e))
  .finally(() => console.log("finally"));

// 4
function kechikish4(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

// 5
kechikish(1000).then(() => console.log("1 soniya o'tdi"));

// 6
new Promise((resolve, reject) => {
  Math.random() > 0.5 ? resolve("Omad") : reject("Omadsizlik");
}).then(x => console.log(x)).catch(e => console.log(e));

// 7 -> 42
Promise.resolve(42).then(x => console.log(x));

// 8
soxtaApi({ ism: "Oqil" }).then(u => console.log(u));

// 9 -> 11
soxtaApi(5).then(x => x * 2).then(x => x + 1).then(x => console.log(x));

// 10
soxtaApi(1)
  .then(x => soxtaApi(x + 10))
  .then(x => console.log(x)); // 11

// 11
Promise.all([soxtaApi("A"), soxtaApi("B"), soxtaApi("C")])
  .then(r => console.log(r)); // ["A", "B", "C"]

// 12 -> butun all reject bo'ladi
Promise.all([soxtaApi("A"), Promise.reject("Xato")])
  .then(r => console.log(r))
  .catch(e => console.log("all xato:", e)); // "all xato: Xato"

// 13 -> "Tez"
Promise.race([soxtaApi("Tez", 500), soxtaApi("Sekin", 2000)])
  .then(r => console.log(r));

// 14
```

```

Promise.allSettled([soxtaApi("A"), Promise.reject("E")])
  .then(r => console.log(r));

// 15
soxtaApi(5)
  .then(x => { if (x === 5) throw new Error("5 bo'lmasin"); return x; })
  .then(x => console.log(x))
  .catch(e => console.log("Ushlandi:", e.message));

// 16
const user01 = () => soxtaApi({ id: 1 });
const postlar01 = id => soxtaApi([`post-${id}`]);
user01()
  .then(user => postlar01(user.id))
  .then(postlar => console.log(postlar))
  .catch(e => console.log(e));

// 17
function timeout(ms) {
  return new Promise((_, reject) => setTimeout(() => reject("Vaqt tugadi"), ms));
}

// 18
Promise.race([soxtaApi("data", 5000), timeout(2000)])
  .then(d => console.log(d))
  .catch(e => console.log(e)); // "Vaqt tugadi"

// 19 - ketma-ket
soxtaApi("1", 500)
  .then(r => { console.log(r); return soxtaApi("2", 500); })
  .then(r => { console.log(r); return soxtaApi("3", 500); })
  .then(r => console.log(r));

// 20
Promise.all([
  soxtaApi({ ism: "Ali" }),
  soxtaApi({ ism: "Vali" }),
  soxtaApi({ ism: "Hasan" })
]).then(users => {
  const ismlar = users.map(u => u.ism);
  console.log(ismlar); // ["Ali", "Vali", "Hasan"]
});

```

15-MODUL: async / await

async / await nima?

`async / await` — Promise'larni ishlatishning **zamonaviy va toza** usuli. U Promise ustiga "shirin qobiq" (syntactic sugar) — asinxron kodni **sinxrondek** o'qishga imkon beradi.



```
// Promise (.then bilan):
function ol() {
  soxtaApi("data").then(d => console.log(d));
}

// async/await bilan (xuddi shu, lekin tozaroq):
async function ol() {
  const d = await soxtaApi("data");
  console.log(d);
}
```

await — natijani "kutish"

`await` Promise tugashini kutadi va **natijasini** qaytaradi. Kod xuddi pastga oqayotgandek o'qiladi:

```
async function malumot() {
  console.log("Boshlandi");
  const user = await soxtaApi({ ism: "Oqil" }); // promise tugashini
  kutadi
  console.log(user.ism); // "Oqil"
  console.log("Tugadi");
}
```

Ikki muhim qoida: - `await` faqat `async` funksiya **ichida** ishlaydi. - `async` funksiya **doim Promise qaytaradi** (sen `return` qilgan qiymat avtomatik Promise'ga o'raladi).

```
async function son() {
  return 42; // aslida Promise<42> qaytaradi
}
son().then(x => console.log(x)); // 42
```

Xatoni `try/catch` bilan ushlash

Promise'dagi `.catch` o'rniga, `async/await` da oddiy `try/catch` ishlatasan:

```
async function malumot() {
  try {
    const data = await soxtaApi("data");
    console.log(data);
  } catch (xato) {
    console.log("Xato yuz berdi:", xato);
  } finally {
    console.log("Tozalash");
  }
}
```

Why try/catch yaxshiroq: Bu — oddiy sinxron koddagidek xato boshqaruvi.

`.then().catch()` zanjiridan ko'ra tabiiyroq va bir nechta `await` ni bitta `try` blokida o'rab olasan.

Callback hell → async/await (taqqoslash)

Eslab ko'r, 13-modulda callback hell shunday edi:

```
userOl((user) => {
  postlarOl(user.id, (postlar) => {
    izohlarOl(postlar[0].id, (izohlar) => {
      console.log(izohlar);
    });
  });
});
```

`async/await` bilan **bir xil mantiq**, lekin tekis va o'qilishi oson:

```
async function hammasi() {
  const user = await userOl();
  const postlar = await postlarOl(user.id);
  const izohlar = await izohlarOl(postlar[0].id);
  console.log(izohlar);
}
```

Mana shu — `async` dunyodagi eng katta yutuq. Bir xil ish, lekin sinxron koddek o'qiladi.

⚠ Ketma-ket vs Parallel (eng ko'p uchraydigan xato)

```
// ❌ SEKIN — ketma-ket (jami ~3 soniya):
async function sekin() {
  const a = await soxtaApi("A", 1000);
  const b = await soxtaApi("B", 1000);
  const c = await soxtaApi("C", 1000);
  return [a, b, c];
}
```

```
// ✅ TEZ — parallel (jami ~1 soniya):
async function tez() {
  const [a, b, c] = await Promise.all([
    soxtaApi("A", 1000),
    soxtaApi("B", 1000),
    soxtaApi("C", 1000),
  ]);
}
```

```
    return [a, b, c];  
  }
```

Why diqqat (juda muhim): So'rovlar **bir-biriga bog'liq bo'lmasa** (B uchun A ning natijasi kerak emas), `await` ni ketma-ket yozma — vaqt behuda ketadi.

`Promise.all` bilan parallel ishlat. Faqat keyingisi avvalgisining natijasiga **bog'liq** bo'lsa, ketma-ket `await` to'g'ri.

`async` arrow funksiya

```
const ol = async () => {  
  const d = await soxtaApi("data");  
  return d;  
};
```

`for` loop ichida `await`

```
async function ketmaKet() {  
  const idlar = [1, 2, 3];  
  for (const id of idlar) {  
    const data = await soxtaApi(`user-${id}`);  
    console.log(data); // birin-ketin  
  }  
}
```

`.map` + `async` (parallel)

`.map` ichida `await` ishlatsang, massiv Promise'lar massivini qaytaradi — `Promise.all` bilan o'rab ol:

```
async function parallel() {  
  const idlar = [1, 2, 3];  
  const natijalar = await Promise.all(  
    idlar.map(id => soxtaApi(`user-${id}`))  
  );  
  console.log(natijalar); // hammasi parallel  
}
```

```
Yana: const soxtaApi = (data, ms=1000) => new Promise(r => setTimeout(() => r(data), ms));
```

1. `async` funksiya yozing, ichida `await soxtaApi(...)` natijasini chiqaring.
2. `async` funksiya `return 42` qilsa, uni `.then` bilan oling (Promise qaytarishini ko'rsating).
3. `await` kechikish(1000) bilan 1 soniya kutib, keyin xabar chiqaring.
4. `try/catch` bilan reject bo'lgan promise'ni ushlang.
5. `try/catch/finally` uchchalasini ishlatiing.
6. `soxtaApi({ism:"Oqil"})` natijasidan `.ism` ni `await` bilan oling.
7. Ikki `await` ni ketma-ket yozing (biri tugagach ikkinchisi).
8. Birinchi `await` natijasini ikkinchisiga uzating (bog'liq).
9. `async` arrow funksiya yozing.
10. `for...of` ichida `await` bilan 3 ta so'rovni ketma-ket bajaring.
11. `Promise.all` + `await` bilan 3 ta so'rovni parallel bajaring.
12. **Vaqt farqi:** xuddi shu 3 so'rovni ketma-ket va parallel yozib, qaysi tezroq — izohlang.
13. `throw new Error(...)` bilan xato tashlab, `catch` da `.message` ni oling.
14. `.map` + `Promise.all` bilan idlar massividan parallel natija oling.
15. Shartli `await`: agar `x > 0` bo'lsa so'rov yuboring, aks holda yo'q.
16. `async` funksiyaning boshqa `async` funksiya ichida `await` qiling.
17. `catch` ichida xatoni qayta tashlang (`throw`) va tashqarida ushlang.
18. Timeout: `Promise.race([soxtaApi(..3000..), timeout(1000)])` ni `await` qiling.
19. Loading holatini boshqaring: `try` dan oldin `loading=true`, `finally` da `false`.
20. **Murakkab:** `[1,2,3]` idlar uchun foydalanuvchilarni **parallel** olib, har birining `ism` ini chiqaring, va biror xato bo'lsa butun jarayonni `try/catch` da ushlang.



► Yechimlar



```
const kechikish = ms => new Promise(r => setTimeout(r, ms));
const soxtaApi = (data, ms = 1000) => new Promise(r => setTimeout(() =>
r(data), ms));
const timeout = ms => new Promise(., rej) => setTimeout(() => rej(new
Error("Vaqt tugadi")), ms));

// 1
async function f1() {
  const d = await soxtaApi("data");
  console.log(d);
}
f1();

// 2 -> 42
async function f2() { return 42; }
f2().then(x => console.log(x));

// 3
async function f3() {
  await kechikish(1000);
  console.log("1 soniya o'tdi");
}
f3();

// 4
async function f4() {
  try {
    await Promise.reject("Xato!");
  } catch (e) {
    console.log("Ushlandi:", e);
  }
}
f4();

// 5
async function f5() {
  try { await soxtaApi("ok"); console.log("ok"); }
  catch (e) { console.log(e); }
  finally { console.log("finally"); }
}
f5();

// 6
async function f6() {
  const u = await soxtaApi({ ism: "Oqil" });
  console.log(u.ism);
}
f6();

// 7
async function f7() {
  const a = await soxtaApi("A", 500);
  const b = await soxtaApi("B", 500);
  console.log(a, b);
}
f7();
```

```

// 8
async function f8() {
  const user = await soxtaApi({ id: 5 });
  const postlar = await soxtaApi(['post-${user.id}']);
  console.log(postlar);
}
f8();

// 9
const f9 = async () => {
  const d = await soxtaApi("arrow");
  console.log(d);
};
f9();

// 10
async function f10() {
  for (const id of [1, 2, 3]) {
    console.log(await soxtaApi(`user-${id}`, 300));
  }
}
f10();

// 11
async function f11() {
  const r = await Promise.all([soxtaApi("A"), soxtaApi("B"),
soxtaApi("C")]);
  console.log(r);
}
f11();

// 12
// Ketma-ket: 3x1000 = ~3s. Parallel (Promise.all): ~1s. Parallel tezroq,
// chunki so'rovlar bir vaqtda boshlanadi.

// 13
async function f13() {
  try { throw new Error("Maxsus xato"); }
  catch (e) { console.log(e.message); } // "Maxsus xato"
}
f13();

// 14
async function f14() {
  const r = await Promise.all([1, 2, 3].map(id => soxtaApi(`u-${id}`)));
  console.log(r);
}
f14();

// 15
async function f15(x) {
  if (x > 0) {
    const d = await soxtaApi("yuborildi");
    console.log(d);
  } else {
    console.log("So'rov yuborilmadi");
  }
}
f15(5);

// 16

```

```

async function ichki() { return await soxtaApi("ichki natija"); }
async function tashqi() {
  const r = await ichki();
  console.log(r);
}
tashqi();

// 17
async function f17() {
  try {
    try { throw new Error("birinchi"); }
    catch (e) { throw new Error("qayta: " + e.message); }
  } catch (e) { console.log(e.message); } // "qayta: birinchi"
}
f17();

// 18
async function f18() {
  try {
    const d = await Promise.race([soxtaApi("data", 3000), timeout(1000)]);
    console.log(d);
  } catch (e) { console.log(e.message); } // "Vaqt tugadi"
}
f18();

// 19
async function f19() {
  let loading = true;
  console.log("loading:", loading);
  try {
    const d = await soxtaApi("data");
    console.log(d);
  } finally {
    loading = false;
    console.log("loading:", loading);
  }
}
f19();

// 20
async function f20() {
  try {
    const users = await Promise.all(
      [1, 2, 3].map(id => soxtaApi({ id, ism: `User${id}` }))
    );
    users.forEach(u => console.log(u.ism));
  } catch (e) {
    console.log("Jarayon xatosi:", e);
  }
}
f20();

```

16-MODUL: Fetch API va REST

REST API nima (qisqacha)?

Backend bilan ma'lumot almashishning eng keng tarqalgan usuli — **REST API**. Sen serverga **so'rov (request)** yuborasan, u **javob (response)** qaytaradi. Javob odatda **JSON** formatida.

HTTP metodlari (CRUD): | Metod | Maqsad | Misol | |-----|-----|-----| | **GET** | O'qish | foydalanuvchini olish | | **POST** | Yaratish | yangi post qo'shish | | **PUT / PATCH** | Yangilash | postni tahrirlash | | **DELETE** | O'chirish | postni o'chirish |

Status kodlari (server javobining holati): | Kod | Ma'no | |----|-----| | 200 | OK (muvaffaqiyat) | | 201 | Created (yaratildi) | | 400 | Bad Request (so'rov xato) | | 401 | Unauthorized (ruxsat yo'q) | | 404 | Not Found (topilmadi) | | 500 | Server Error (server xatosi) |

Sen backend (Laravel) yozasan — bu kodlar tanish. Frontend tomonida `response.status` orqali shularni tekshirasan.

`fetch` — so'rov yuborish

`fetch` Promise qaytaradi. Shuning uchun `await` yoki `.then` bilan ishlatamiz.

Mashqlar uchun bepul test API: <https://jsonplaceholder.typicode.com> (haqiqiy so'rovlar yuborsa bo'ladi).

GET (`.then` bilan)

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json()) // javobni JSON qilib o'qiyamiz
  .then(data => console.log(data))
  .catch(error => console.log("Xato:", error));
```

GET (`async/await` bilan — tavsiya etiladi)

```
async function post01() {
  const response = await
  fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await response.json();
  console.log(data);
}
```

Why ikki `await`: Birinchi `await fetch(...)` — server javobini (headerlar, status) oladi. Ikkinchi `await response.json()` — javob **tanasini** (body) o'qib, JSON'ga

aylantiradi. `response.json()` ham Promise qaytaradi, shuning uchun yana `await`.

`response.ok` va xato boshqaruvi (muhim!)

⚠ **Tuzoq:** `fetch` 404 yoki 500 kabi xatolarda ham `.catch` ga **tushmaydi!** U faqat tarmoq uzilganda xato beradi. Server xatosini **o'zing** tekshirishing kerak:

```
async function postOl(id) {
  try {
    const response = await
    fetch(`https://jsonplaceholder.typicode.com/posts/${id}`);

    if (!response.ok) {
      // 404, 500 va h.k. shu yerda ushlanadi
      throw new Error(`Server xatosi: ${response.status}`);
    }

    const data = await response.json();
    return data;
  } catch (error) {
    console.log("Xato:", error.message);
  }
}
```

Why bu muhim: Bu — junior'lar 100% kalishadigan joy. `fetch` "muvaqqafiyat" deb hisoblaydi, agar server **javob bersa** — javob 404 bo'lsa ham. Shuning uchun `if (!response.ok) throw ...` shart. `response.ok` — status 200-299 bo'lsa `true`.

POST — ma'lumot yuborish

```
async function postYarat() {
  const response = await
  fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json", // JSON yuborayotganimizni
      aytamiz
    },
    body: JSON.stringify({ // obyektini JSON matnga aylantiramiz
      title: "Yangi post",
      body: "Matn",
      userId: 1,
    }),
  });

  const data = await response.json();
}
```

```
console.log(data); // server qaytargan yangi obyekt (id bilan)
}
```

POST/PUT uchun 3 narsa kerak: 1. `method: "POST"` 2. `headers — "Content-Type": "application/json"` 3. `body: JSON.stringify(obyekt)` — **doim** stringify qil

PUT (yangilash) va DELETE (o'chirish)

```
// PUT — to'liq yangilash
await fetch("https://jsonplaceholder.typicode.com/posts/1", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ id: 1, title: "Yangilangan" }),
});

// DELETE
await fetch("https://jsonplaceholder.typicode.com/posts/1", {
  method: "DELETE",
});
```

Query parametrlari

```
// userId=1 bo'lgan postlar:
const response = await
fetch("https://jsonplaceholder.typicode.com/posts?userId=1");
const data = await response.json();
console.log(data.length);
```

Fetch + DOM (real loyiha namunasi)

```
<button id="yukla">Postlarni yuklash</button>
<p id="holat"></p>
<ul id="royxat"></ul>
<script>
  const holat = document.querySelector("#holat");
  const ul = document.querySelector("#royxat");

  document.querySelector("#yukla").addEventListener("click", async ()
=> {
    holat.textContent = "Yuklanmoqda..."; // loading holati
    ul.innerHTML = "";

    try {
      const res = await
fetch("https://jsonplaceholder.typicode.com/posts?_limit=5");
```

```

    if (!res.ok) throw new Error(`Xato: ${res.status}`);

    const postlar = await res.json();
    postlar.forEach(post => {
      const li = document.createElement("li");
      li.textContent = post.title;
      ul.appendChild(li);
    });
    holat.textContent = ""; // loading'ni o'chiramiz
  } catch (error) {
    holat.textContent = "Xatolik: " + error.message; // xatoni
    ko'rsatamiz
  }
});
</script>

```

Mana bu — 3-qism (DOM, events) va 4-qism (async, fetch) ning birlashmasi. Real frontend ilovasi aynan shunday ishlaydi: tugma → loading → fetch → render → (xato bo'lsa) xabar.

16-modul masalalari (20 ta)

API: <https://jsonplaceholder.typicode.com> (/posts , /posts/1 , /users , /comments).

1. `fetch ( .then )` bilan `/posts/1` ni olib, `response.json()` qilib chiqaring.
2. Xuddi shuni `async/await` bilan qayta yozing.
3. `/posts/1` javobidan faqat `title` ni chiqaring.
4. `response.ok` ni tekshirib, `false` bo'lsa `throw` qiling.
5. `response.status` ni konsolga chiqaring.
6. `/posts` (barchasi) ni olib, nechta post borligini chiqaring.
7. `.map` bilan barcha postlardan faqat `title` larni ajratib oling.
8. `try/catch` bilan fetch xatosini ushlang.
9. Mavjud bo'lmagan URL (`/posts/1`) ga so'rov yuborib, xatoni ushlang (404).
10. `POST` bilan yangi post yarating (`title` , `body` , `userId`).
11. `POST`'da `headers` va `JSON.stringify(body)` ni to'g'ri qo'ying.
12. `PUT` bilan `/posts/1` ni yangilang.
13. `DELETE` bilan `/posts/1` ni o'chiring va `response.status` ni tekshiring.
14. Query param bilan `?userId=1` postlarini oling.
15. `?_limit=5` bilan faqat 5 ta post oling.

16. `Promise.all` bilan `/posts/1` va `/users/1` ni **parallel** oling.
17. **DOM:** postlarni `<ul>` ichida ro'yxat qilib chiqaring.
18. **DOM:** so'rovdan oldin "Yuklanmoqda..." ko'rsatib, tugagach o'chiring.
19. **DOM:** xato bo'lsa foydalanuvchiga ko'rinadigan xabar chiqaring.
20. **Murakkab:** `/users/1` ni va o'sha userning postlarini (`/posts?userId=1`) parallel olib, "`<Ism> - N ta post`" ko'rinishida DOM'da chiqaring.



```
const BASE = "https://jsonplaceholder.typicode.com";

// 1
fetch(`${BASE}/posts/1`)
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(e => console.log(e));

// 2
async function f2() {
  const res = await fetch(`${BASE}/posts/1`);
  const data = await res.json();
  console.log(data);
}
f2();

// 3
async function f3() {
  const res = await fetch(`${BASE}/posts/1`);
  const data = await res.json();
  console.log(data.title);
}
f3();

// 4
async function f4() {
  const res = await fetch(`${BASE}/posts/1`);
  if (!res.ok) throw new Error(`Xato: ${res.status}`);
  console.log("OK");
}
f4();

// 5
async function f5() {
  const res = await fetch(`${BASE}/posts/1`);
  console.log(res.status); // 200
}
f5();

// 6
async function f6() {
  const res = await fetch(`${BASE}/posts`);
  const data = await res.json();
  console.log(data.length); // 100
}
f6();

// 7
async function f7() {
  const res = await fetch(`${BASE}/posts`);
  const postlar = await res.json();
  console.log(postlar.map(p => p.title));
}
f7();

// 8 & 9
async function f9() {
```

```

    try {
      const res = await fetch(`${BASE}/postss/1`); // noto'g'ri URL
      if (!res.ok) throw new Error(`Topilmadi: ${res.status}`);
      console.log(await res.json());
    } catch (e) {
      console.log("Xato ushlandi:", e.message); // 404
    }
  }
  f9();

  // 10 & 11
  async function f10() {
    const res = await fetch(`${BASE}/posts`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title: "Yangi", body: "Matn", userId: 1 }),
    });
    console.log(await res.json()); // { id: 101, ... }
  }
  f10();

  // 12
  async function f12() {
    const res = await fetch(`${BASE}/posts/1`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ id: 1, title: "Yangilangan", body: "...", userId:
1 }),
    });
    console.log(await res.json());
  }
  f12();

  // 13
  async function f13() {
    const res = await fetch(`${BASE}/posts/1`, { method: "DELETE" });
    console.log(res.status); // 200
  }
  f13();

  // 14
  async function f14() {
    const res = await fetch(`${BASE}/posts?userId=1`);
    console.log((await res.json()).length); // 10
  }
  f14();

  // 15
  async function f15() {
    const res = await fetch(`${BASE}/posts?_limit=5`);
    console.log((await res.json()).length); // 5
  }
  f15();

  // 16
  async function f16() {
    const [postRes, userRes] = await Promise.all([
      fetch(`${BASE}/posts/1`),
      fetch(`${BASE}/users/1`),
    ]);
    const post = await postRes.json();

```

```

    const user = await userRes.json();
    console.log(post.title, user.name);
  }
  f16();

  // 17, 18, 19 - DOM (HTML: <button id="yukla">, <p id="holat">, <ul
  id="royxat">)
  async function f17() {
    const holat = document.querySelector("#holat");
    const ul = document.querySelector("#royxat");
    holat.textContent = "Yuklanmoqda..."; // 18
    ul.innerHTML = "";
    try {
      const res = await fetch(`${BASE}/posts?_limit=5`);
      if (!res.ok) throw new Error(`Xato: ${res.status}`);
      const postlar = await res.json();
      postlar.forEach(p => {
        const li = document.createElement("li");
        li.textContent = p.title;
        ul.appendChild(li);
      });
      holat.textContent = "";
    } catch (e) {
      holat.textContent = "Xatolik: " + e.message; // 19
    }
  }
  document.querySelector("#yukla")?.addEventListener("click", f17);

  // 20 - Murakkab
  async function f20() {
    const royxat = document.querySelector("#royxat");
    try {
      const [userRes, postRes] = await Promise.all([
        fetch(`${BASE}/users/1`),
        fetch(`${BASE}/posts?userId=1`),
      ]);
      if (!userRes.ok || !postRes.ok) throw new Error("So'rov xatosi");

      const user = await userRes.json();
      const postlar = await postRes.json();

      const li = document.createElement("li");
      li.textContent = `${user.name} - ${postlar.length} ta post`;
      royxat.appendChild(li);
    } catch (e) {
      console.log(e.message);
    }
  }
  f20();

```

4-qism yakuni

Endi sen asinxron JS'ni tushunasan — bu frontend va backend (Node) ishining yuragi: - **Event Loop** — JS qanday "kutadi" (Call Stack, microtask vs macrotask) - **Callbacks** va

callback hell muammosi - **Promises** — `.then / .catch`, zanjir,
`Promise.all / race / allSettled` - `async / await` — toza, sinxrondek o'qiladigan kod
+ `try/catch` - **Fetch API** — real server bilan ishlash, `response.ok` tuzog'i,
POST/PUT/DELETE

Eng muhim 3 ta xulosa: 1. **Promise (microtask) doim `setTimeout` (macrotask)dan oldin** ishlaydi. 2. **`fetch` xato statuslarda `.catch` ga tushmaydi** — `if (!response.ok) throw shart`. 3. **Mustaqil so'rovlarni `Promise.all` bilan parallel qil, ketma-ket `await` qilma.**

Keyingi qadam (5-qism) — Chuqur JS

Bu yerda JS'ning "ostki qavatiga" tushamiz — buni bilgan dasturchini junior'dan ajratadi:
- **Scope, Closures, Hoisting** — `var / let` nega farq qiladi, closure nima - `this` —
JS'ning eng chalkash mavzusi, nihoyat to'liq ochiladi - **Prototypes** va inheritance — JS
qanday "meros" oladi - **Classes** (ES6) — zamonaviy OOP - **ES6+** — destructuring,
spread, modullar, optional chaining (chuqurroq)

Maslahat: 5-qismga o'tishdan oldin 16-modulning #20-masalasini noldan yoz. Fetch
+ `Promise.all` + DOM + `try/catch` — bularning hammasi bitta joyda. Bu — real ish.

 [README](#) · [← 3-qism — Brauzer \(DOM\)](#) · [Keyingi: 5-qism \(1-bo'lim\) — Chuqur JS →](#)

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 [README](#) · [← 4-qism](#) — Asinxron JS · [Keyingi: 5-qism \(2-bo'lim\)](#) →

5-QISM: CHUQUR JS — 1-bo'lim (OOP yadrosi) JS'ning "ostki qavati": o'zgaruvchilar qanday yashaydi, `this` nima, obyektlar qanday meros oladi. Bu blok — junior'ni middle'dan ajratadigan joy. Har bir moduldan keyin **20 ta masala** (yechimi bilan).

5-qism eng katta (7 modul), shuning uchun ikkiga bo'lindi: **1-bo'lim** (shu fayl): 17-Scope/Closures/Hoisting · 18- `this` · 19-Prototypes · 20-Classes **2-bo'lim** (keyingi): 21-ES6+ modullar · 22-Error handling · 23-Regex

17-MODUL: Scope, Closures, Hoisting

Scope (ko'rinish doirasi)

Scope — o'zgaruvchi **qayerdan ko'rinishi** (kirish mumkinligi). 3 xil scope bor:

```
const global = "hammaga ko'rinadi"; // 1. Global scope

function f() {
  const funksiyaIchi = "faqat f ichida"; // 2. Function scope
  console.log(global); // ✅ ko'rinadi
  console.log(funksiyaIchi); // ✅ ko'rinadi
}

console.log(funksiyaIchi); // ❌ ReferenceError - tashqarida ko'rinmaydi
```

Block scope (`let / const`) vs `var`

`let / const` — **blok** (`{ }`) ichida cheklangan. `var` esa — **funksiya** ichida (blokni e'tiborsiz qoldiradi):

```
if (true) {
  let a = 1;
  var b = 2;
}
```

```
console.log(b); // 2 - var blokdan "qochib" chiqdi  
console.log(a); // ✗ ReferenceError - let blok ichida qoldi
```

Why — var ning yana bir muammosi: var blokni hurmat qilmaydi, shuning uchun kutilmagan xatolar keltiradi (4-qismdagi sikl tuzog'ini esla). let / const aniq: ular faqat o'z {} ichida yashaydi. Bu — var ni ishlatmaslikning tub sababi.

Lexical scope (ichki funksiya tashqarini ko'radi)

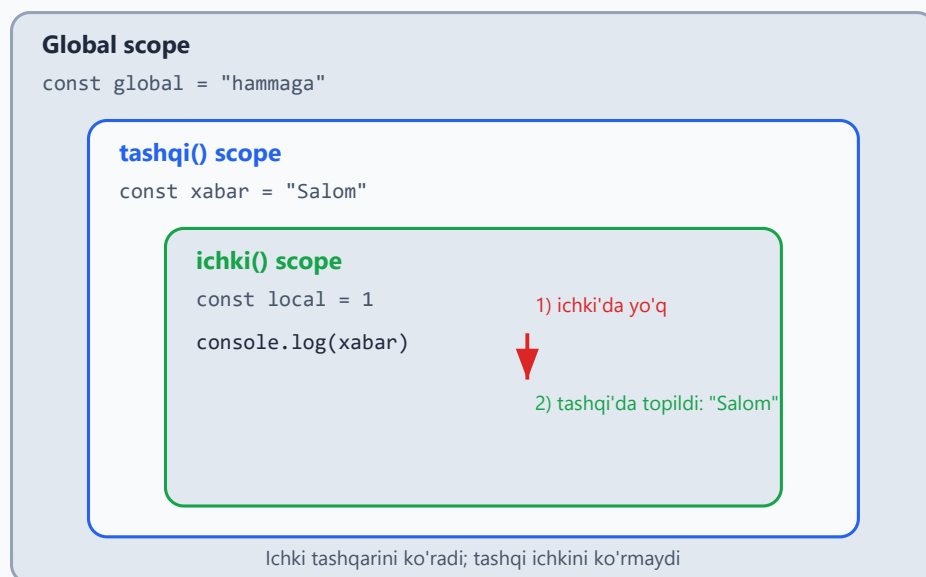
Funksiya **qayerda yozilganiga** qarab tashqi o'zgaruvchilarni ko'radi:

```
function tashqi() {  
  const xabar = "Salom";  
  function ichki() {  
    console.log(xabar); // ✔ tashqi o'zgaruvchini ko'radi  
  }  
  ichki();  
}
```

Ichki funksiya tashqi scope'ni ko'radi, lekin tashqi — ichkini ko'rmaydi. Bu zanjir **scope chain** deyiladi: o'zgaruvchi avval o'z scope'ida, topilmasa tashqarida, keyin global'da qidiriladi.

Scope chain — o'zgaruvchi qidirish yo'nalishi

topilmasa: ichkaridan tashqariga, keyin global'ga



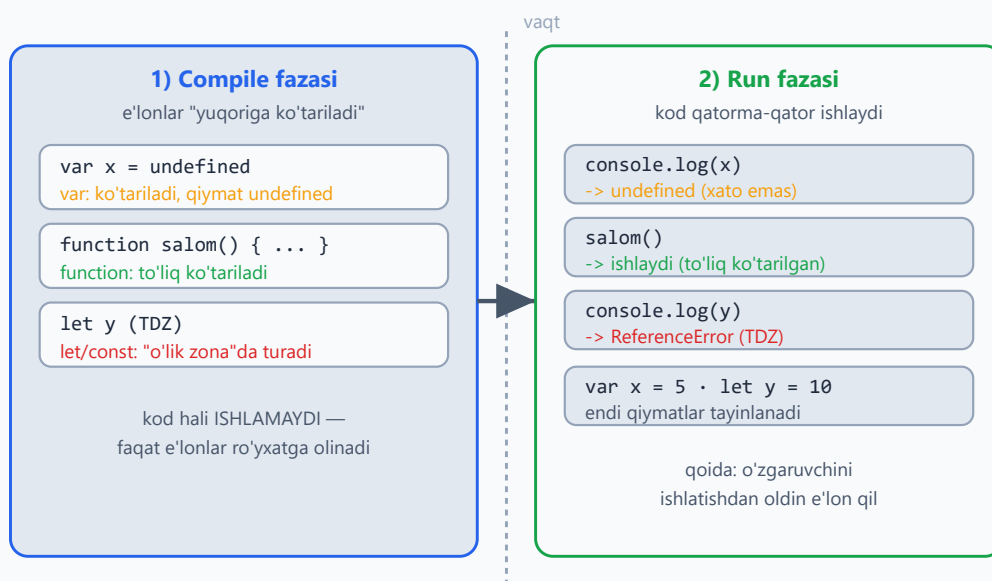
Hoisting (ko'tarilish)

JS kodni ishga tushirishdan oldin e'lonlarni "yuqoriga ko'taradi". Lekin har xil turlar har xil ko'tariladi:

```
// var – ko'tariladi, lekin undefined bo'lib:  
console.log(x); // undefined (xato emas!)  
var x = 5;  
  
// function declaration – to'liq ko'tariladi (e'londan oldin chaqirsa bo'ladi):  
salom(); // ✅ "Salom" – ishlaydi  
function salom() { console.log("Salom"); }  
  
// let/const – ko'tariladi, lekin "Temporal Dead Zone"da:  
console.log(y); // ❌ ReferenceError  
let y = 10;
```

Why – TDZ (Temporal Dead Zone): `let / const` texnik jihatdan ko'tariladi, lekin e'lon qilingan qatorgacha "o'lik zona"da turadi — ularga murojaat qilsang xato beradi. Bu — yaxshi narsa: `var` dagi `undefined` chalkashligini oldini oladi. Doim o'zgaruvchini ishlatishdan oldin e'lon qil.

Hoisting — 2 faza: avval e'lonlar, keyin ishga tushish



Diqqat: **function expression** va arrow funksiyalar `var / let` qoidasiga bo'ysunadi — ular ko'tarilmaydi:

```
salom(); // ❌ TypeError – bu function expression  
const salom = function() { console.log("Salom"); };
```

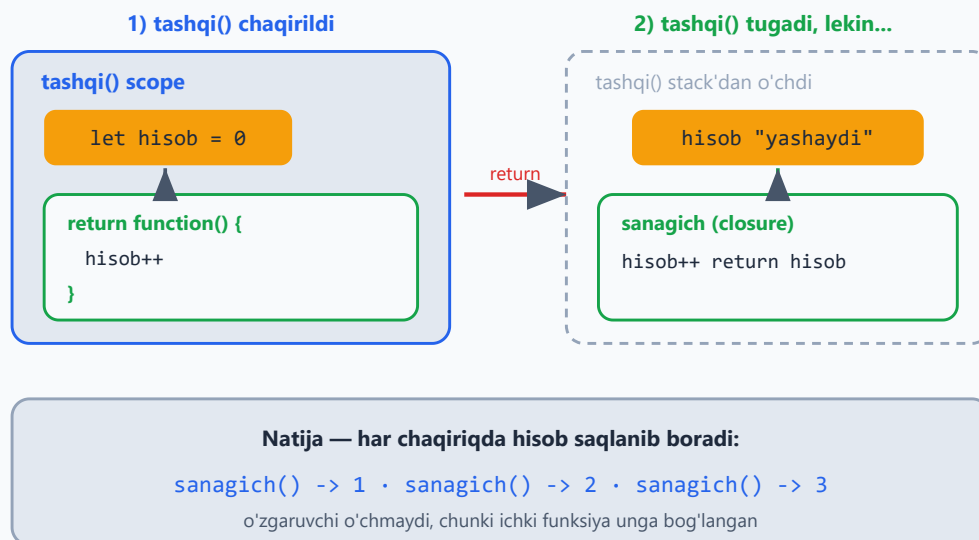
★ Closures (yopilmalar) — eng muhim mavzu

Closure — bu funksiya o'zi yaratilgan scope'dagi o'zgaruvchilarni **eslab qoladigan** hodisa. Hatto tashqi funksiya tugab ketgan bo'lsa ham, ichki funksiya o'sha o'zgaruvchilarga kirishda davom etadi.

```
function tashqi() {  
  let hisob = 0; // bu o'zgaruvchi "yashaydi"  
  
  return function() {  
    hisob++; // tashqi o'zgaruvchini eslaydi va o'zgartiradi  
    return hisob;  
  };  
}  
  
const sanagich = tashqi(); // tashqi() tugadi, LEKIN hisob yo'qolmadi  
console.log(sanagich()); // 1  
console.log(sanagich()); // 2  
console.log(sanagich()); // 3
```

Why bu sehrlil: Odatda funksiya tugagach, uning o'zgaruvchilari o'chadi. Lekin ichki funksiya `hisob` ga "bog'lanib qolgani" uchun, JS uni xotirada saqlaydi. Mana shu — closure. Bu — JS'ning eng kuchli va eng ko'p ishlatiladigan xususiyatlaridan biri.

Closure — funksiya o'zgaruvchini "eslab qoladi"



Closure foydasi 1: Function Factory (funksiya yasovchi)

Parametrni "eslab qoladigan" funksiyalar yaratish:

```
function kopaytiruvchi(n) {
  return x => x * n; // n ni eslab qoladi
}

const ikkilik = kopaytiruvchi(2);
const uchlik = kopaytiruvchi(3);

console.log(ikkilik(5)); // 10
console.log(uchlik(5)); // 15
```

Closure foydasi 2: Private o'zgaruvchi (inkapsulatsiya)

Tashqaridan ko'rinmaydigan, faqat ruxsat etilgan funksiyalar orqali o'zgaradigan ma'lumot:

```
function hisobYaratuvchi() {
  let balans = 0; // PRIVATE – tashqaridan kirib bo'lmaydi

  return {
    qoshish(summa) { balans += summa; },
    yechish(summa) { balans -= summa; },
    korish() { return balans; },
  };
}

const hisob = hisobYaratuvchi();
hisob.qoshish(100);
hisob.qoshish(50);
console.log(hisob.korish()); // 150
console.log(hisob.balans); // undefined – to'g'ridan-to'g'ri kira olmaymiz!
```

Why — OOP bog'lanishi: Bu — inkapsulatsiyaning (data hiding) JS'dagi klassik usuli. `balans` himoyalangan; uni faqat `qoshish` / `yechish` orqali o'zgartirish mumkin. Class'lardagi `private` maydonlar (#) chiqquncha, JS'da privatlik aynan shu closure orqali qilingan (20-modulda ko'rasan).

Closure va sikl tuzog'i (4-qismdan davom)

Eslaysanmi, `let` sikl ichida nega to'g'ri ishlagandi? Chunki `let` **har takrorda yangi binding** (closure) yaratadi:

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0, 1, 2
}
// Har callback o'zining alohida "i" closure'siga ega.
```

17-modul masalalari (20 ta)

1. Funksiya ichidagi o'zgaruvchiga tashqaridan kirib bo'lmasligini ko'rsating.
2. `let blok ( {} )` ichida e'lon qilib, tashqarida ishlatishga urining (xato).
3. `var ni if` bloki ichida e'lon qilib, tashqarida ko'rinishini ko'rsating.
4. Lexical scope: ichki funksiya tashqi o'zgaruvchini o'qisin.
5. Hoisting: `console.log(x); var x = 5;` natijasini bashorat qiling.
6. Function declaration'ni e'lonidan **oldin** chaqiring (ishlaydi).
7. TDZ: `let` ni e'lonidan oldin o'qishga urining (xato turini ayting).
8. Function expression hoisting ishlamasligini ko'rsating.
9. Closure: `tashqi()` ichki funksiya qaytarsin, `hisob` saqlansin.
10. Closure bilan sanagich (counter) yarating: `1, 2, 3` chiqarsin.
11. `kopaytiruvchi(n)` factory yozing — `ikkilik(5)` → 10 qaytarsin.
12. `qoshuvchi(n)` factory: `besht = qoshuvchi(5); besh(3)` → 8.
13. Closure bilan private `balans`: `qoshish / korish` metodlari bo'lsin.
14. Ikki ta mustaqil sanagich yarating — bir-biriga ta'sir qilmasin.
15. Closure bilan "faqat bir marta ishlaydigan" funksiya (`once`) yozing.
16. `setTimeout` + `let` sikli bilan `0, 1, 2` chiqaring.
17. Closure bilan oddiy memoize (kesh): bir xil argument qayta kelsa, hisoblamasin.
18. Funksiya nechta marta chaqirilganini sanaydigan wrapper yozing.
19. IIFE (darhol chaqiriladigan funksiya) bilan private holat yarating (module pattern).
20. **Murakkab:** Bank hisobi — closure bilan `deposit`, `withdraw`, `balans` metodlari; `balans` private bo'lsin, manfiy yechishga ruxsat bermasin.



► Yechimlar



```
// 1
function f1() { const x = 5; }
// console.log(x); // ✗ ReferenceError

// 2
{ let a = 1; }
// console.log(a); // ✗ ReferenceError

// 3
if (true) { var b = 2; }
console.log(b); // 2

// 4
function tashqi4() {
  const xabar = "Salom";
  function ichki() { return xabar; }
  return ichki();
}
console.log(tashqi4()); // "Salom"

// 5 -> undefined
console.log(typeof x5); var x5 = 5; // "undefined"

// 6 -> "Salom"
salom6();
function salom6() { console.log("Salom"); }

// 7 -> ReferenceError
// console.log(y7); let y7 = 10;

// 8 -> TypeError (salom8 hali undefined)
// salom8(); const salom8 = () => {};

// 9
function tashqi9() {
  let hisob = 0;
  return () => ++hisob;
}
const s9 = tashqi9();
console.log(s9(), s9(), s9()); // 1 2 3

// 10
const sanagich = (() => {
  let n = 0;
  return () => ++n;
})();
console.log(sanagich(), sanagich(), sanagich()); // 1 2 3

// 11
const kopaytiruvchi = n => x => x * n;
console.log(kopaytiruvchi(2)(5)); // 10

// 12
const qoshuvchi = n => x => x + n;
const besh = qoshuvchi(5);
console.log(besh(3)); // 8
```

```

// 13
function hisob13() {
  let balans = 0;
  return {
    qoshish: s => balans += s,
    korish: () => balans,
  };
}
const h = hisob13();
h.qoshish(100);
console.log(h.korish()); // 100

// 14
function yarat() { let n = 0; return () => ++n; }
const a14 = yarat(), b14 = yarat();
console.log(a14(), a14()); // 1 2
console.log(b14()); // 1 (mustaqil)

// 15
function once(fn) {
  let bajarildi = false;
  return (...args) => {
    if (!bajarildi) { bajarildi = true; return fn(...args); }
  };
}
const init = once(() => console.log("Faqat bir marta"));
init(); init(); // faqat bir marta chiqadi

// 16
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 100); // 0,1,2

// 17 - memoize
function memoize(fn) {
  const kesh = {};
  return (n) => {
    if (n in kesh) { console.log("keshdan"); return kesh[n]; }
    const r = fn(n);
    kesh[n] = r;
    return r;
  };
}
const kvadrat = memoize(n => n * n);
console.log(kvadrat(4)); // hisoblanadi -> 16
console.log(kvadrat(4)); // keshdan -> 16

// 18
function sanagichWrapper(fn) {
  let soni = 0;
  return (...args) => {
    soni++;
    console.log(`${soni}-marta chaqirildi`);
    return fn(...args);
  };
}
const f = sanagichWrapper(() => "ish");
f(); f();

// 19 - module pattern (IIFE)
const Modul = (() => {
  let maxfiy = 42;
  return { korish: () => maxfiy };
})();

```

```

})();
console.log(Modul.korish()); // 42
// console.log(Modul.maxfiy); // undefined

// 20 - Bank hisobi
function bankHisobi(boshlangich = 0) {
  let balans = boshlangich;
  return {
    deposit(summa) {
      if (summa > 0) balans += summa;
      return balans;
    },
    withdraw(summa) {
      if (summa > balans) { console.log("Mablag' yetarli emas"); return
balans; }
      balans -= summa;
      return balans;
    },
    get balans() { return balans; },
  };
}
const hisob20 = bankHisobi(100);
hisob20.deposit(50); // 150
hisob20.withdraw(200); // "Mablag' yetarli emas", 150
console.log(hisob20.balans); // 150

```

18-MODUL: **this**

this — JS'ning eng ko'p chalkashtiradigan mavzusi. Lekin bitta qoida hamma narsani ochadi.

Asosiy qoida

this ning qiymati funksiya QAYERDA yozilganiga emas, QANDAY CHAQIRILGANIGA bog'liq. (Arrow funksiyalar bundan mustasno — pastda.)

Funksiya chaqirilishining 4 holati bor:

1. Metod sifatida chaqirish — **this** = nuqtadan oldingi obyekt

```

const user = {
  ism: "Oqil",
  salom() {
    return `Salom, men ${this.izm}`; // this = user
  },
};
console.log(user.salom()); // "Salom, men Oqil"

```

2. Oddiy funksiya — `this` = global (yoki `undefined`)

```
function f() {  
  console.log(this);  
}  
f(); // strict mode'da (modul/class) -> undefined; oddiy script'da -> window
```

3. Konstruktor (`new`) — `this` = yangi yaratilgan obyekt

```
function User(ism) {  
  this.ism = ism; // this = yangi obyekt  
}  
const u = new User("Oqil");  
console.log(u.ism); // "Oqil"
```

4. Explicit (`call`/`apply`/`bind`) — `this` = sen bergan qiymat (pastda)

`this` — 4 qoida (qanday chaqirilganiga bog'liq)

1 Default — oddiy funksiya

```
function f() { ... }  
f()
```

this = undefined (yoki window)
nuqtasiz, alohida chaqiruv

2 Implicit — metod sifatida

```
const user = { salom() {} }  
user.salom()
```

this = nuqtadan oldingi obyekt
this = user

3 Explicit — call / apply / bind

```
f.call(obj)  
f.bind(obj)
```

this = sen bergan qiymat (obj)
this'ni qo'lda belgilaysan

4 new — konstruktor

```
function User(ism) {}  
new User("Oqil")
```

this = yangi yaratilgan obyekt
bo'sh obyekt yaratiladi

Arrow funksiya bundan mustasno: u `this`'ni yozilgan joydan (lexical) oladi

⚠ "Yo'qolgan this" muammosi

`this` chaqirilishga bog'liq bo'lgani uchun, metodni nuqtadan **ajratsang**, `this` yo'qoladi:

```
const user = {  
  ism: "Oqil",
```

```

    salom() { return this.ism; },
  };

  const f = user.salom; // metodni ajratib oldik
  console.log(f()); // undefined! – endi "user.salom()" emas, shunchaki
  "f()"

```

Xuddi shu — `setTimeout`, `event listener`, `callback`'larda sodir bo'ladi:

```

const user = {
  ism: "Oqil",
  salom() {
    setTimeout(function() {
      console.log(this.ism); // undefined – this user emas!
    }, 100);
  },
};
user.salom();

```

✓ Arrow funksiya — `this` ni "meros" qiladi

Arrow funksiyaning **o'z `this` i yo'q**. U o'zi yozilgan joydan (lexical) `this` ni oladi. Bu — yuqoridagi muammoning yechimi:

```

const user = {
  ism: "Oqil",
  salom() {
    setTimeout(() => {
      console.log(this.ism); // "Oqil" ✓ – arrow tashqi this'ni
    }, 100);
  },
};
user.salom(); // "Oqil"

```

Why — eng muhim amaliy qoida: Metod ichidagi `callback`'larda **arrow funksiya** ishlat — u tashqi `this` ni saqlaydi. Lekin obyekt metodining **o'zini** arrow qilma (pastdagi tuzoq).

⚠ Tuzoq: obyekt metodini arrow qilish

```

const user = {
  ism: "Oqil",
  salom: () => {
    console.log(this.ism); // undefined! – arrow'ning this'i user emas,
    tashqi (global)
  }
};

```

```
    },  
  };  
  user.salom(); // undefined
```

Arrow `this` ni **yozilgan joydan** oladi — bu yerda obyekt metodi darajasida `this` global'ga ishora qiladi (obyektga emas). Shuning uchun **obyekt metodlarini oddiy funksiya** qilib yoz, callback'larnigina arrow qil.

`call`, `apply`, `bind` — `this` ni qo'lda belgilash

```
function salom(salutatsiya) {  
  return `${salutatsiya}, ${this.ism}`;  
}  
const user = { ism: "Oqil" };  
  
// call - darhol chaqiradi, argumentlar vergul bilan:  
console.log(salom.call(user, "Assalom")); // "Assalom, Oqil"  
  
// apply - darhol chaqiradi, argumentlar massivda:  
console.log(salom.apply(user, ["Salom"])); // "Salom, Oqil"  
  
// bind - chaqirmaydi, this BOG'LANGAN yangi funksiya qaytaradi:  
const boglangan = salom.bind(user);  
console.log(boglangan("Hayrli kun")); // "Hayrli kun, Oqil"
```

Metod	Darhol chaqiriladimi?	Argumentlar
<code>call</code>	Ha	vergul bilan: <code>call(this, a, b)</code>
<code>apply</code>	Ha	massivda: <code>apply(this, [a, b])</code>
<code>bind</code>	Yo'q (yangi funksiya)	vergul bilan

Why `bind`: Event listener'da `this` ni saqlash uchun juda foydali — `element.addEventListener("click", obj.metod.bind(obj))`. `bind` `this` ni "muhrlaydi", keyin uni o'zgartirib bo'lmaydi. Arrow funksiyalarda `call` / `apply` / `bind` `this` ni o'zgartira **olmaydi** (ularning `this` i lexical va o'zgarmas).

Amaliy: metod "qarz olish" (borrowing)

`call` / `apply` bilan bir obyekt metodini boshqasiga qo'llash:



```
const ali = { ism: "Ali", salom() { return `Men ${this.izm}`; } };  
const vali = { ism: "Vali" };  
  
console.log(ali.salom.call(vali)); // "Men Vali" – Vali uchun ishlatdik
```

18-modul masalalari (20 ta)

1. Obyekt metodida `this.izm` ni qaytaring (`user.salom()`).
2. Metodni ajratib (`const f = user.salom; f()`) `this` yo'qolishini ko'rsating.
3. Oddiy funksiya ichida `this` nima ekanini tekshiring (strict).
4. **Tuzoq:** obyekt metodini arrow qiling – `this` ishlamasligini ko'rsating.
5. `setTimeout` ichida oddiy funksiya bilan `this` yo'qolishini ko'rsating.
6. Xuddi shuni arrow funksiya bilan tuzating.
7. `call` bilan `this` va argument bering.
8. `apply` bilan argumentlarni massivda bering.
9. `bind` bilan yangi bog'langan funksiya yarating.
10. `bind` bilan argumentni oldindan biriktiring (partial application).
11. Metod "qarz olish": bir obyekt metodini `call` bilan boshqasiga qo'llang.
12. Ichma-ich oddiy funksiya `this` yo'qolishini, arrow bilan tuzalishini ko'rsating.
13. `Math.max.apply(null, [3,9,2])` bilan massivdan eng kattani toping.
14. Konstruktor funksiya: `new User("Oqil")` – `this.izm` ni o'rnating.
15. `bind` bilan event listener uchun `this` ni saqlang (konseptual/DOM).
16. Bir nechta obyekttni bitta umumiy funksiya bilan `call` orqali boshqaring.
17. Arrow funksiya `bind this` ni o'zgartira olmasligini ko'rsating.
18. Obyekt metodida ham arrow, ham oddiy funksiyaning solishtiring (`this` farqi).
19. `forEach` callback'ida `this`: oddiy yo'qotadi, arrow saqlaydi – ko'rsating.
20. **Murakkab:** obyekt yarating – metodida `this` ishlasin; metod ichidagi `setTimeout` callback'ida ham `this` to'g'ri bo'lsin (arrow); va bitta metodni boshqa obyektga `bind` bilan biriktiring.



```
// 1
const user = { ism: "Oqil", salom() { return this.ism; } };
console.log(user.salom()); // "Oqil"

// 2 -> undefined
const f2 = user.salom;
// console.log(f2()); // undefined (this yo'qoldi)

// 3
"use strict";
function f3() { return this; }
console.log(f3()); // undefined (strict) yoki window (script)

// 4 -> undefined
const u4 = { ism: "Oqil", salom: () => this.ism };
console.log(u4.salom()); // undefined (arrow this = tashqi)

// 5
const u5 = {
  ism: "Oqil",
  salom() { setTimeout(function () { console.log(this.ism); }, 50); },
};
u5.salom(); // undefined

// 6
const u6 = {
  ism: "Oqil",
  salom() { setTimeout(() => console.log(this.ism), 50); },
};
u6.salom(); // "Oqil"

// 7
function salom7(s) { return `${s}, ${this.ism}`; }
console.log(salom7.call({ ism: "Oqil" }, "Salom")); // "Salom, Oqil"

// 8
console.log(salom7.apply({ ism: "Ali" }, ["Hey"])); // "Hey, Ali"

// 9
const bog9 = salom7.bind({ ism: "Vali" });
console.log(bog9("Assalom")); // "Assalom, Vali"

// 10
const salomOqil = salom7.bind({ ism: "Oqil" }, "Hayrli tong");
console.log(salomOqil()); // "Hayrli tong, Oqil"

// 11
const ali = { ism: "Ali", tani() { return `Men ${this.ism}`; } };
console.log(ali.tani.call({ ism: "Vali" })); // "Men Vali"

// 12
const u12 = {
  ism: "Oqil",
  test() {
    function ichkiOddiy() { return this?.ism; } // undefined
    const ichkiArrow = () => this.ism; // "Oqil"
    return [ichkiOddiy(), ichkiArrow()];
  }
};
```

```

    },
  };
  console.log(u12.test()); // [undefined, "Oqil"]

  // 13 -> 9
  console.log(Math.max.apply(null, [3, 9, 2]));
  // zamonaviy: Math.max(...[3,9,2])

  // 14
  function User14(ism) { this.ism = ism; }
  console.log(new User14("Oqil").ism); // "Oqil"

  // 15 (DOM)
  // const obj = { ism: "Oqil", bos() { console.log(this.ism); } };
  // btn.addEventListener("click", obj.bos.bind(obj)); // this saqlanadi

  // 16
  function umumiySalom() { return this.ism; }
  const obyektlar = [{ ism: "A" }, { ism: "B" }];
  obyektlar.forEach(o => console.log(umumiySalom.call(o))); // A, B

  // 17
  const ar17 = () => this;
  console.log(ar17.call({ ism: "X" })); // bind/call ta'sir qilmaydi (lexical this)

  // 18
  const u18 = {
    ism: "Oqil",
    oddiy() { return this.ism; }, // "Oqil"
    ok: () => this.ism, // undefined
  };
  console.log(u18.oddiy(), u18.ok()); // "Oqil" undefined

  // 19
  const u19 = {
    ism: "Oqil",
    raqamlar: [1, 2],
    oddiyForEach() { this.raqamlar.forEach(function () {
      console.log(this?.ism); }); }, // undefined
    arrowForEach() { this.raqamlar.forEach(() => console.log(this.ism)); }, // "Oqil"
  };
  u19.arrowForEach(); // Oqil, Oqil

  // 20
  const profil = {
    ism: "Oqil",
    salom() {
      console.log(`Men ${this.ism}`); // metod this
      setTimeout(() => console.log(this.ism), 50); // arrow -> this saqlanadi
    },
  };
  profil.salom(); // "Men Oqil", keyin "Oqil"
  const boshqa = { ism: "Vali" };
  profil.salom.bind(boshqa)(); // "Men Vali", keyin "Vali"

```

19-MODUL: Prototypes va inheritance

Bu — JS'da obyektlar **qanday ishlashining** asosi. Class'lar (keyingi modul) shuning ustiga qurilgan "shirin qobiq". Buni tushunsang, JS obyekt modelini tushunding.

Prototype zanjiri

Har bir obyektدا yashirin **prototype** (ota-obyekt) havolasi bor. Obyektда xususiyat topilmasa, JS uning prototype'iga qaraydi, keyin prototype'ning prototype'iga — **null** gacha. Bu — **prototype chain**.

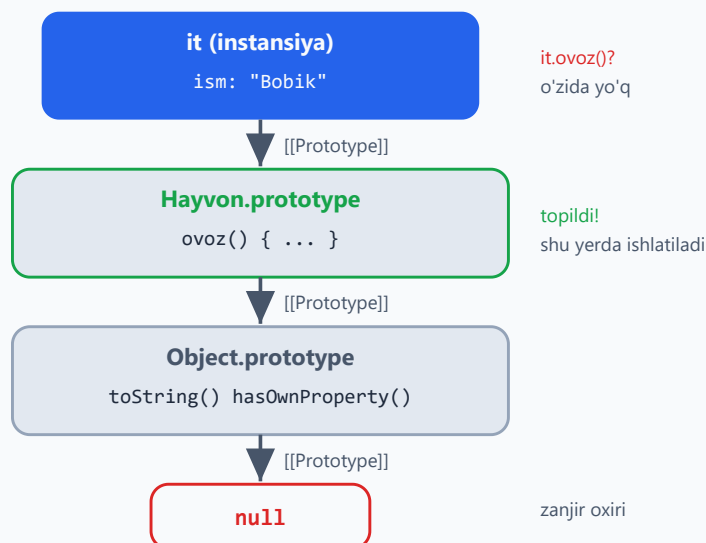
```
const arr = [1, 2, 3];
// Zanjir: arr -> Array.prototype -> Object.prototype -> null
// Shuning uchun arr.map(), arr.toString() ishlaydi — ular
// Array.prototype'da!

console.log(Object.getPrototypeOf(arr) === Array.prototype); // true
```

Why bu muhim: `[1,2,3].map(...)` nega ishlaydi? `map` massivning o'zida emas — `Array.prototype` da. JS uni zanjir bo'ylab topadi. Demak, barcha massivlar **bitta** `map` nusxasini ulashadi (xotira tejaladi). Mana shu — prototypingning kuchi.

Prototype zanjiri — xususiyat qaerdan topiladi

obyektда topilmasa, JS zanjir bo'ylab yuqoriga qaraydi



Konstruktor funksiya + `.prototype`

`new` bilan obyekt yasashning eski (class'dan oldingi) usuli:

```
function Hayvon(ism) {
  this.ism = ism; // har obyektning O'Z xususiyati
}

// Umumiy metod – prototype'ga qo'yamiz (barcha obyektlar ulashadi):
Hayvon.prototype.ovozi = function () {
  return `${this.ism} ovozi chiqaradi`;
};

const it = new Hayvon("Bobik");
const mushuk = new Hayvon("Mosi");

console.log(it.ovozi()); // "Bobik ovozi chiqaradi"
console.log(mushuk.ovozi()); // "Mosi ovozi chiqaradi"

// Ikkalasi ham bitta metodni ulashadi:
console.log(it.ovozi === mushuk.ovozi); // true
```

Why prototype'ga qo'yamiz: Agar `ovozi` ni konstruktor ichida (`this.ovozi = ...`) yozsak, har obyekt **o'z nusxasini** olardi — 1000 obyekt = 1000 nusxa (xotira isrofi). Prototype'da bo'lsa — **bitta** nusxa, hammasi ulashadi. Shuning uchun **ma'lumot** — konstruktorda (`this.ism`), **metodlar** — prototype'da.

`hasOwnProperty` — o'zini vs merosga olingan

```
const it = new Hayvon("Bobik");

console.log(it.hasOwnProperty("ism")); // true - o'zining xususiyati
console.log(it.hasOwnProperty("ovozi")); // false - prototype'dan
merosga olingan
console.log("ovozi" in it); // true - zanjirda bor (meros
ham hisoblanadi)
```

`instanceof`

Obyekt biror konstruktordan yaratilganmi (zanjirda bormi):

```
console.log(it instanceof Hayvon); // true
console.log(it instanceof Object); // true (zanjirda Object.prototype
ham bor)
```

`Object.create` — prototype'ni to'g'ridan-to'g'ri berish

```
const hayvon = {
  ovoz() { return `${this.ism} ovoz chiqaradi`; },
};

const it = Object.create(hayvon); // it'ning prototype'i = hayvon
it.ism = "Bobik";
console.log(it.ovozi()); // "Bobik ovoz chiqaradi"
```

Inheritance (meros) — class'dan oldingi usul

```
function Hayvon(ism) {
  this.ism = ism;
}
Hayvon.prototype.ovozi = function () { return `${this.ism}: ovoz`; };

function It(ism, zot) {
  Hayvon.call(this, ism); // ota konstruktorni chaqirish (this bilan)
  this.zot = zot;
}
// Prototype zanjirini ulash:
It.prototype = Object.create(Hayvon.prototype);
It.prototype.constructor = It;

// It'ga o'z metodi:
It.prototype.vovullash = function () { return `${this.ism} vovullaydi`; };

const bobik = new It("Bobik", "Alabay");
console.log(bobik.ovozi()); // "Bobik: ovoz" (merosga olingan)
console.log(bobik.vovullash()); // "Bobik vovullaydi"
console.log(bobik instanceof Hayvon); // true
```

Why class'lar yaratilgan: Bu kod ishlaydi, lekin **ko'p va chalkash** (`Object.create`, `constructor` tuzatish, `.call`). Aynan shu og'riqni kamaytirish uchun ES6'da **class** sintaksisi qo'shilgan — keyingi modulda ko'rasan. Class — aslida shu prototype mexanizmining chiroyli ko'rinishi.

19-modul masalalari (20 ta)

1. `Object.getPrototypeOf([])` — massiv prototype'ini tekshiring.
2. `[1,2].map` `Array.prototype` da ekanini ko'rsating.
3. Konstrukt `User(ism)` yozing, `new` bilan obyekt yarating.
4. `.prototype` ga `salom` metodini qo'shib, instansiyada chaqiring.

5. Ikki instansiya bitta prototype metodini ulashishini ko'rsating (`===`).
6. `hasOwnProperty` bilan o'z xususiyati va merosni farqlang.
7. `instanceof` bilan obyekt konstruktordan ekanini tekshiring.
8. `Object.create` bilan berilgan prototype'li obyekt yarating.
9. Prototype'ga keyin metod qo'shib, **mavjud** instansiyada ham ishlashini ko'rsating.
10. Instansiyada prototype metodini "ustidan yozish" (shadowing) ni ko'rsating.
11. Prototype'dagi xususiyatni zanjir orqali topilishini ko'rsating.
12. `Hayvon(ism)` konstruktor + `.prototype.ovoiz` metodi yozing.
13. Inheritance: `It` konstruktori ota `Hayvon` ni `call` bilan chaqirsin.
14. Prototype zanjirini ulang (`Object.create(Hayvon.prototype)`).
15. `instanceof` meros zanjiri bilan (`It` instansiyasi `Hayvon` ham bo'lsin).
16. `Object.getPrototypeOf` ni zanjir bo'ylab 2 marta chaqiring.
17. `String.prototype` ga maxsus metod qo'shing (ehtiyotkorlik izohi bilan).
18. `in` operatori va `hasOwnProperty` farqini ko'rsating.
19. Konstruktor + prototype: barcha instansiyalar uchun umumiy metod yozing.
20. **Murakkab:** `Hayvon` → `It` merosi quring; `It` o'z metodi (`vovullash`) qo'shsin, ota metodini (`ovoiz`) ishlatsin; `instanceof` ikkalasini ham `true` qaytarsin.



```
// 1
console.log(Object.getPrototypeOf([]) === Array.prototype); // true

// 2
console.log([1, 2].map === Array.prototype.map); // true

// 3
function User(ism) { this.ism = ism; }
const u = new User("Oqil");
console.log(u.ism); // "Oqil"

// 4
User.prototype.salom = function () { return `Salom, ${this.ism}`; };
console.log(u.salom()); // "Salom, Oqil"

// 5
const u2 = new User("Ali");
console.log(u.salom === u2.salom); // true

// 6
console.log(u.hasOwnProperty("ism")); // true
console.log(u.hasOwnProperty("salom")); // false

// 7
console.log(u instanceof User); // true

// 8
const proto = { salom() { return `Men ${this.ism}`; } };
const obj = Object.create(proto);
obj.ism = "Vali";
console.log(obj.salom()); // "Men Vali"

// 9
User.prototype.xayr = function () { return "Xayr"; };
console.log(u.xayr()); // "Xayr" (eski instansiyada ham ishlaydi)

// 10
u.salom = () => "O'zimning salom";
console.log(u.salom()); // "O'zimning salom" (shadowing)
console.log(u2.salom()); // prototype'dan (o'zgarmadi)

// 11
function A() {}
A.prototype.x = 10;
console.log(new A().x); // 10 (zanjirdan)

// 12
function Hayvon(ism) { this.ism = ism; }
Hayvon.prototype.ovoq = function () { return `${this.ism}: ovoq`; };

// 13, 14, 15, 20 - Inheritance
function It(ism, zot) {
  Hayvon.call(this, ism); // 13
  this.zot = zot;
}
It.prototype = Object.create(Hayvon.prototype); // 14
It.prototype.constructor = It;
```

```

It.prototype.vovullash = function () { return `${this.ism} vovullaydi`; };

const bobik = new It("Bobik", "Alabay");
console.log(bobik.ovoiz()); // "Bobik: ovoiz" (ota metodi)
console.log(bobik.vovullash()); // "Bobik vovullaydi"
console.log(bobik instanceof It); // true (15)
console.log(bobik instanceof Hayvon); // true (15)

// 16
console.log(Object.getPrototypeOf(Object.getPrototypeOf(bobik)) ===
Hayvon.prototype); // true

// 17 – ehtiyot bo'l: built-in prototype'ni o'zgartirish odatda tavsiya
etilmaydi
String.prototype.qaytar = function () { return
this.split("").reverse().join(""); };
console.log("abc".qaytar()); // "cba"
// (Real loyihada built-in prototype'ni kengaytirma – konfliktga olib
keladi)

// 18
console.log("ovoiz" in bobik); // true (meros ham)
console.log(bobik.hasOwnProperty("ovoiz")); // false (o'ziniki emas)

// 19
function Doira(r) { this.r = r; }
Doira.prototype.yuza = function () { return Math.PI * this.r ** 2; };
console.log(new Doira(2).yuza().toFixed(2)); // 12.57

```

20-MODUL: Classes (ES6)

Class — prototype'larning **zamonaviy, toza sintaksisi**. Ostida xuddi 19-moduldagi mexanizm ishlaydi, lekin yozish ancha qulay. (Oqil — sen Laravel'da OOP/SOLID bilan ishlaysan; bu yerdagi tushunchalar tanish bo'ladi.)

Class e'lon qilish

```

class Hayvon {
  constructor(ism) {
    this.ism = ism; // instansiya xususiyati
  }

  ovoiz() { // metod – avtomatik prototype'ga tushadi
    return `${this.ism} ovoiz chiqaradi`;
  }
}

```

```
const it = new Hayvon("Bobik");
console.log(it.ovoiz()); // "Bobik ovoiz chiqaradi"
```

constructor — new chaqirilganda ishga tushadigan maxsus metod. Metodlar avtomatik Hayvon.prototype ga tushadi (19-moduldagi qo'lda yozishni eslab ko'r — class buni avtomatik qiladi).

Inheritance: extends va super

```
class Hayvon {
  constructor(ism) { this.ism = ism; }
  ovoiz() { return `${this.ism}: ovoiz`; }
}

class It extends Hayvon {
  constructor(ism, zot) {
    super(ism); // ota konstruktorni chaqirish (MAJBURIY, this'dan oldin)
    this.zot = zot;
  }

  ovoiz() { // override — ota metodini almashtirish
    return `${this.ism} vovullaydi`;
  }

  toliq() {
    return super.ovoiz() + ` (${this.zot})`; // ota metodini chaqirish
  }
}

const bobik = new It("Bobik", "Alabay");
console.log(bobik.ovoiz()); // "Bobik vovullaydi" (override)
console.log(bobik.toliq()); // "Bobik: ovoiz (Alabay)" (super bilan)
```

Why super muhim: super(...) — ota konstruktorni chaqiradi. extends ishlatganda, this ga tegishdan oldin super() chaqirilishi shart (aks holda xato). super.metod() esa — ota metodini chaqiradi (override qilingan bo'lsa ham). 19-moduldagi chalkash Object.create + .call — endi bitta extends + super bilan hal bo'ldi.

static — instansiyasiz ishlaydigan

static metod/xususiyat — class'ning o'ziga tegishli, instansiyaga emas:

```
class MatYordam {
  static kvadrat(n) { return n * n; }
  static PI = 3.14159;
```

```
}  
  
console.log(MatYordam.kvadrat(5)); // 25 – new kerak emas  
console.log(MatYordam.PI);          // 3.14159
```

Misol: `Math.max()`, `Array.from()`, `Object.keys()` — hammasi static metodlar. Ular yordamchi (utility) funksiyalar uchun — obyekt holatiga bog'liq emas.

Getter / Setter

Xususiyatga o'qish/yozishni metod orqali boshqarish (lekin xususiyatdek ishlatiladi):

```
class Doira {  
  constructor(radius) { this._radius = radius; }  
  
  get radius() { return this._radius; }  
  
  set radius(qiymat) {  
    if (qiymat <= 0) throw new Error("Radius musbat bo'lsin");  
    this._radius = qiymat;  
  }  
  
  get yuza() { // hisoblanadigan xususiyat  
    return Math.PI * this._radius ** 2;  
  }  
}  
  
const d = new Doira(5);  
console.log(d.radius); // 5 (get – qavssiz!)  
console.log(d.yuza.toFixed(2)); // 78.54  
d.radius = 10; // set ishlaydi  
// d.radius = -1; // ❌ Error
```

Why getter/setter: Tashqaridan oddiy xususiyatdek ko'rinadi (`d.radius`), lekin ichida tekshiruv/hisob bor. `yuza` — har safar radiusga qarab hisoblanadi, alohida saqlash shart emas.

Private maydonlar (`#`) — ES2022

Haqiqiy private (closure'siz). `#` bilan boshlangan maydon tashqaridan ko'rinmaydi:

```
class BankHisobi {  
  #balans = 0; // PRIVATE  
  
  deposit(summa) {  
    if (summa > 0) this.#balans += summa;  
  }  
}
```

```

    get balans() { return this.#balans; }
}

const hisob = new BankHisobi();
hisob.deposit(100);
console.log(hisob.balans); // 100
// console.log(hisob.#balans); // ✗ SyntaxError – tashqaridan kira
olmaysan

```

Why # vs closure: 17-modulda privatlikni closure bilan qilgan eding. Endi class'larda # bor — toza va aniq. #balans ni faqat class ichidagi metodlar ko'radi. Bu — inkapsulatsiya (OOP'ning asosiy prinsipi).

Class — bu prototype'ning qobig'i (isbot)

```

class Test {
  metod() {}
}
console.log(typeof Test);           // "function" (class
— funksiya!)
console.log(Test.prototype.metod);   // metod
prototype'da
console.log(new Test() instanceof Test); // true

```

Demak, class — sintaktik shakar. Ostida 19-moduldagi prototype mexanizmi. Lekin yozish va o'qish ancha qulay — shuning uchun zamonaviy kodda class ishlatiladi.

20-modul masalalari (20 ta)

1. User class yozing — constructor da ism o'rnating.
2. new User("Oqil") bilan instansiya yaratib, ism ni chiqaring.
3. salom() metodi qo'shing — this.ism ishlatsin.
4. Ikki instansiya mustaqil ekanini ko'rsating.
5. Hayvon class'idan It class'ini extends qiling.
6. It konstruktorida super(ism) chaqiring.
7. It da ovoz() metodini override qiling.
8. super.ovoiz() bilan ota metodini It ichidan chaqiring.
9. MatYordam class'iga static kvadrat(n) qo'shing.
10. static xususiyat (PI) qo'shing va instansiyasiz ishlating.

11. `Doira` class'iga `get radius()` getter qo'shing.
12. `set radius(v)` setter qo'shing — manfiy qiymatga xato bersin.
13. `get yuza()` — radiusdan hisoblansin (qavssiz chaqiriladi).
14. `#balans` private maydon bilan `BankHisobi` yozing.
15. `#balans` ga tashqaridan kirib bo'lmasligini ko'rsating.
16. `instanceof` bilan instansiya class va ota class'dan ekanini tekshiring.
17. Class metodi `prototype` da ekanini ko'rsating (`Object.getPrototypeOf`).
18. `toString()` metodini override qiling.
19. `static` "factory" metod yozing (`User.yarat(...)` yangi instansiya qaytarsin).
20. **Murakkab:** `Shakl` bazaviy class (`yuza()` metodi); `Doira` va `Tortburchak` undan meros olsin; massivda turli shakllar bo'lsin va har birining yuzasini chiqaring (polimorfizm).



```
// 1, 2, 3, 4
class User {
  constructor(ism) { this.ism = ism; }
  salom() { return `Salom, ${this.ism}`; }
}
const u1 = new User("Oqil");
const u2 = new User("Ali");
console.log(u1.ism); // "Oqil"
console.log(u1.salom()); // "Salom, Oqil"
console.log(u2.ism); // "Ali" (mustaqil)

// 5, 6, 7, 8
class Hayvon {
  constructor(ism) { this.ism = ism; }
  ovoz() { return `${this.ism}: ovoz`; }
}
class It extends Hayvon {
  constructor(ism, zot) { super(ism); this.zot = zot; }
  ovoz() { return `${this.ism} vovullaydi`; } // override
  toliq() { return super.ovoiz() + ` (${this.zot})`; } // super
}
const bobik = new It("Bobik", "Alabay");
console.log(bobik.ovoiz()); // "Bobik vovullaydi"
console.log(bobik.toliq()); // "Bobik: ovoiz (Alabay)"

// 9, 10
class MatYordam {
  static kvadrat(n) { return n * n; }
  static PI = 3.14159;
}
console.log(MatYordam.kvadrat(5)); // 25
console.log(MatYordam.PI); // 3.14159

// 11, 12, 13
class Doira {
  constructor(r) { this._r = r; }
  get radius() { return this._r; }
  set radius(v) { if (v <= 0) throw new Error("Musbat bo'lsin"); this._r = v; }
  get yuza() { return Math.PI * this._r ** 2; }
}
const d = new Doira(5);
console.log(d.radius); // 5
console.log(d.yuza.toFixed(2)); // 78.54
d.radius = 10; // setter
// d.radius = -1; // Error

// 14, 15
class BankHisobi {
  #balans = 0;
  deposit(s) { if (s > 0) this.#balans += s; }
  get balans() { return this.#balans; }
}
const hisob = new BankHisobi();
hisob.deposit(100);
console.log(hisob.balans); // 100
// console.log(hisob.#balans); // SyntaxError
```

```

// 16
console.log(bobik instanceof It); // true
console.log(bobik instanceof Hayvon); // true

// 17
console.log(Object.getPrototypeOf(u1).salom === User.prototype.salom); //
true

// 18
class Pul {
  constructor(miqdor) { this.miqdor = miqdor; }
  toString() { return `${this.miqdor} so'm`; }
}
console.log(`${new Pul(1000)}`); // "1000 so'm"

// 19 - static factory
class User19 {
  constructor(ism) { this.ism = ism; }
  static yarat(ism) { return new User19(ism); }
}
console.log(User19.yarat("Oqil").ism); // "Oqil"

// 20 - Polimorfizm
class Shakl {
  yuza() { return 0; }
}
class Doira20 extends Shakl {
  constructor(r) { super(); this.r = r; }
  yuza() { return Math.PI * this.r ** 2; }
}
class Tortburchak extends Shakl {
  constructor(a, b) { super(); this.a = a; this.b = b; }
  yuza() { return this.a * this.b; }
}
const shakllar = [new Doira20(2), new Tortburchak(3, 4), new Doira20(1)];
shakllar.forEach(s => console.log(s.yuza().toFixed(2)));
// 12.57, 12.00, 3.14 - har biri o'z yuza() metodini ishlatadi

```

✓ 5-qism (1-bo'lim) yakuni

Endi sen JS'ning "ostki qavatini" tushunasani — bu junior'ni middle'dan ajratadigan bilim: - **Scope** — global/function/block, `var` ning yana muammolari, lexical scope - **Hoisting** — `var` (undefined), function (to'liq), `let` / `const` (TDZ) - **Closures** — funksiya scope'ni "eslab qoladi"; private ma'lumot, function factory, memoize - **this** — chaqirilishga bog'liq (4 holat); arrow `this` ni meros qiladi; `call` / `apply` / `bind` - **Prototypes** — JS obyekt modeli; prototype chain; konstruktor + `.prototype` - **Classes** — prototype ustidagi zamonaviy sintaksis; `extends` / `super` / `static` / `getter-setter` / `#private`

Eng muhim 3 xulosa: 1. **Closure** — JS'ning eng kuchli vositasi (private state, factories). Buni puxta tushun. 2. **this** — "qanday chaqirildi"ga bog'liq. Metod callback'larida **arrow** ishlat, obyekt metodining o'zini emas. 3. **Class** — **prototype'ning qobig'i**. Ostida bir xil mexanizm.

Keyingi qadam (5-qism, 2-bo'lim)

- **ES6+ modular** — `import / export`, kodni fayllarga bo'lish (real loyiha strukturasi)
- **Error handling** — `try/catch`, custom error class'lar, `throw`, xatolarni to'g'ri boshqarish
- **Regular expressions** — matn naqshlari (validatsiya, qidirish, almashtirish)

Maslahat: 20-modulning #20-masalasini (polimorfizm) puxta tushun. Class hierarchy + override + bir xil interfeys orqali turli xatti-harakat — bu OOP'ning yuragi. (Keyinchalik Vue/React komponentlari va boshqa freymvorklarda shu g'oya qayta-qayta uchraydi.)

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 README · [← 5-qism \(1-bo'lim\)](#) · [Keyingi: 6-qism \(1-bo'lim\)](#) — [Expert](#) →

5-QISM: CHUQUK JS — 2-bo'lim Zamonaviy JS xususiyatlari, modullar, xato boshqaruvi va regex. Har bir moduldan keyin **20 ta masala** (yechimi bilan). Bu bo'lim 5-qismni yakunlaydi.

1-bo'limda OOP yadrosini (closures, `this`, prototypes, classes) ko'rdik. Endi real loyihaga kerakli zamonaviy vositalar.

21-MODUL: ES6+ va modullar

Allaqachon bilganlaring (qisqa takror)

Quyidagilarni oldingi qismlarda ishlatgansan — bular ham ES6+:

```
let, const           // 1-modul
`Salom ${ism}`       // template literal - 1-modul
(a, b) => a + b        // arrow function - 5-modul
function f(x = 10) {} // default param - 5-modul
function f(...args) {} // rest param - 5-modul
const [a, b] = arr;   // array destructuring - 6-modul
const { x, y } = obj;  // object destructuring - 7-modul
[...arr1, ...arr2]    // spread - 6/7-modul
obj?.manzil?.shahar   // optional chaining - 7-modul
```

Endi yangilarini ko'ramiz.

Nullish coalescing `??` (vs `||`)

`??` — chap tomon `null` yoki `undefined` bo'lsa, o'ng tomonni beradi. `||` dan farqi: `0`, `""`, `false` ni "haqiqiy qiymat" deb hisoblaydi:

```
const a = 0;

console.log(a || 100); // 100 - chunki 0 falsy
console.log(a ?? 100); // 0 - chunki 0 null/undefined emas!
```

```
const ism = "";
console.log(ism || "Mehmon"); // "Mehmon" (bo'sh string falsy)
console.log(ism ?? "Mehmon"); // "" (bo'sh string mavjud qiymat)
```

Why ?? : `||` har qanday "falsy" qiymatni almashtiradi — bu `0` yoki `""` haqiqiy qiymat bo'lganda muammo (masalan, soni `0` ta mahsulot). `??` faqat `null` / `undefined` ni almashtiradi. Qiymat **yo'qligini** tekshirmoqchi bo'lsang `??`, "falsy"likni tekshirmoqchi bo'lsang `||`.

Logical assignment (`??=` , `||=` , `&&=`)

```
let sozlama;
sozlama ??= "default"; // null/undefined bo'lsa tayinlaydi -> "default"

let ism = "";
ism ||= "Mehmon"; // falsy bo'lsa tayinlaydi -> "Mehmon"

let user = { faol: true };
user.faol &&= "ha"; // truthy bo'lsa tayinlaydi -> "ha"
```

Optional chaining — funksiya va massiv bilan

```
const obj = {
  salom() { return "Salom"; },
};

console.log(obj.salom?.()); // "Salom" — metod bor, chaqiriladi
console.log(obj.xayr?.()); // undefined — metod yo'q, xato emas

const arr = null;
console.log(arr?.[0]); // undefined — massiv null, xato emas
```

Set — unikal qiymatlar to'plami

Set — takrorlanmaydigan qiymatlar to'plami:

```
const s = new Set([1, 2, 2, 3, 3, 3]);
console.log(s); // Set(3) {1, 2, 3} — dublikatlar yo'q

s.add(4); // qo'shish
console.log(s.has(2)); // true — bormi?
s.delete(1); // o'chirish
console.log(s.size); // 3 — nechta?
```

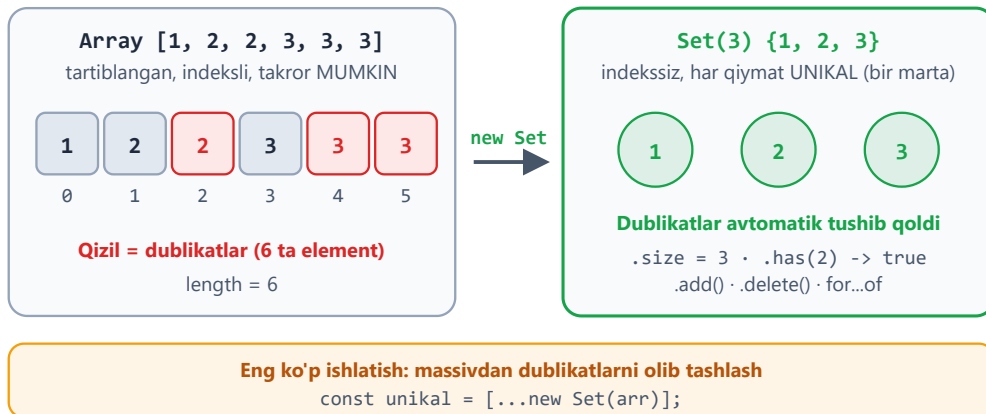
```
// Bo'ylab aylanish:  
for (const el of s) console.log(el);
```

Eng ko'p ishlatiladigan joy — **massivdan dublikatlarni olib tashlash**:

```
const arr = [1, 1, 2, 3, 3, 4];  
const unikal = [...new Set(arr)];  
console.log(unikal); // [1, 2, 3, 4]
```

Set vs Array — takror bormi?

Array dublikatni saqlaydi, Set esa har qiymatni faqat BIR marta saqlaydi



Map — kalit-qiymat to'plami (har qanday kalit)

Map — obyektga o'xshash, lekin kalit **har qanday tur** bo'lishi mumkin (obyekt, funksiya, son):

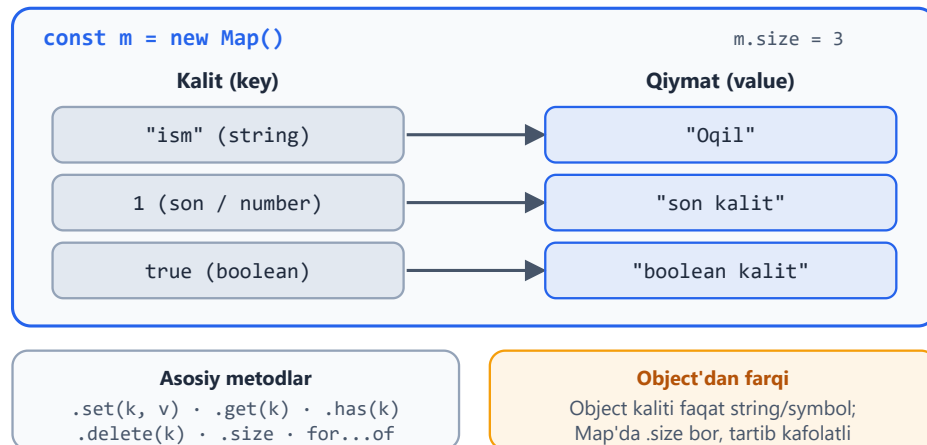
```
const m = new Map();  
m.set("ism", "Oqil");  
m.set(1, "son kalit");  
m.set(true, "boolean kalit");  
  
console.log(m.get("ism")); // "Oqil"  
console.log(m.has(1)); // true  
m.delete(true);  
console.log(m.size); // 2  
  
// Bo'ylab aylanish:  
for (const [kalit, qiymat] of m) {  
  console.log(kalit, qiymat);  
}
```

Why Map vs Object: (1) Kalit har qanday tur bo'lishi mumkin (Object'da faqat string/symbol). (2) .size bor (Object'da Object.keys().length kerak). (3)

Qo'shilish tartibi kafolatlangan. (4) Prototype kalitlari aralashmaydi. Tez-tez qo'shib-
o'chiriladigan, kalitlari noma'lum bo'lgan ma'lumot uchun `Map` ishlat; sobit strukturali
yozuv uchun `Object`.

Map — kalit va qiymat juftliklari

Har yozuv: kalit -> qiymat. Kalit HAR QANDAY tur bo'lishi mumkin (string, son, boolean, obyekt)



★ ES Modullar — `import` / `export`

Real loyihada kod **bir nechta faylga** bo'linadi. Modullar — bir fayldagi
funksiya/o'zgaruvchini boshqa faylda ishlatish imkonini beradi.

Named export / import

```
// matematika.js
export const PI = 3.14159;
export function qoshish(a, b) { return a + b; }
export function ayirish(a, b) { return a - b; }
```

```
// main.js
import { PI, qoshish, ayirish } from "./matematika.js";

console.log(qoshish(2, 3)); // 5
console.log(PI);           // 3.14159

// Nom o'zgartirish:
import { qoshish as plus } from "./matematika.js";

// Hammasini namespace sifatida:
import * as mat from "./matematika.js";
console.log(mat.qoshish(1, 2)); // 3
```

Default export / import

Bir faylda **bitta** default export bo'ladi (asosiy narsa):

```
// 📄 User.js
export default class User {
  constructor(ism) { this.ism = ism; }
}
```

```
// 📄 main.js
import User from "./User.js"; // qavssiz, istalgan nom bilan
const u = new User("Oqil");
```

Ikkalasini birga

```
// 📄 api.js
export default function fetchData() {}
export const BASE_URL = "https://api.uz";
```

```
// 📄 main.js
import fetchData, { BASE_URL } from "./api.js";
```

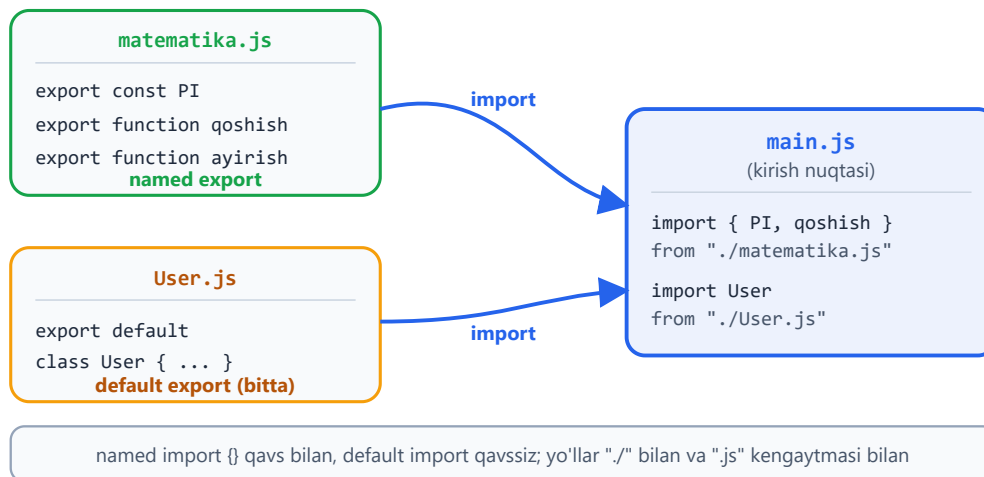
Brauzerda ishlatish

```
<script type="module" src="main.js"></script>
```

Why modular: (1) Kod tartibli — har fayl o'z vazifasini bajaradi (single responsibility). (2) Faqat kerakli narsani import qilasan. (3) Vue/Nuxt, React — hammasi modular ustiga qurilgan. **Eslatma:** brauzerda `type="module"` kerak, yo'llar `./` bilan va `.js` kengaytmasi bilan yoziladi; Node'da `package.json` da `"type": "module"`. Bundler (Vite) bu tafsilotlarni o'zi hal qiladi.

ES Modullar — export / import grafi

Har fayl o'z vazifasini bajaradi; main.js kerakli narsalarni boshqa fayllardan import qiladi



21-modul masalalari (20 ta)

1. `0 || 5` va `0 ?? 5` natijalarini taqqoslang.
2. `"" || "X"` va `"" ?? "X"` natijalarini taqqoslang.
3. `??=` bilan `undefined` o'zgaruvchiga default tayinlang.
4. `||=` bilan bo'sh stringga qiymat tayinlang.
5. `obj.metod?.()` — metod bor va yo'q holatlarni sinab ko'ring.
6. `arr?.[0]` — massiv `null` bo'lganda xato bermasligini ko'rsating.
7. `new Set([1,1,2,3,3])` yaratib, natijasini chiqaring.
8. Massiv `[5,5,1,2,2,8]` dan dublikatlarni `Set` bilan olib tashlang.
9. `Set`: `add`, `has`, `delete`, `size` ni ishlating.
10. `Set` bo'ylab `for...of` bilan aylaning.
11. `Map` yaratib, `set / get` qiling.
12. `Map`'ga son va boolean kalit qo'shing (har qanday tur).
13. `Map`: `has`, `delete`, `size` ni ishlating.
14. `Map` bo'ylab `[kalit, qiymat]` bilan aylaning.
15. `Map` va `Object` farqini bitta misolda ko'rsating.
16. **Modul**: `matematika.js` yozing — `qoshish`, `PI` ni export qiling (named).
17. **Modul**: uni `main.js` da import qilib ishlating.
18. **Modul**: default export'li `User` class yozing va import qiling.

19. **Modul:** `import * as` bilan namespace import yozing.

20. **Murakkab:** kichik loyiha — `utils.js` (bir nechta export), `main.js` (default + named import birga). To'liq strukturani yozing.



► Yechimlar



```
// 1 -> 5 va 0
console.log(0 || 5); // 5
console.log(0 ?? 5); // 0

// 2 -> "X" va ""
console.log("" || "X"); // "X"
console.log("" ?? "X"); // ""

// 3
let s3;
s3 ??= "default";
console.log(s3); // "default"

// 4
let ism4 = "";
ism4 ||= "Mehmon";
console.log(ism4); // "Mehmon"

// 5
const o5 = { salom() { return "Salom"; } };
console.log(o5.salom?.()); // "Salom"
console.log(o5.xayr?.()); // undefined

// 6
const a6 = null;
console.log(a6?.[0]); // undefined

// 7
console.log(new Set([1, 1, 2, 3, 3])); // Set(3) {1,2,3}

// 8
console.log([...new Set([5, 5, 1, 2, 2, 8])]); // [5,1,2,8]

// 9
const s9 = new Set();
s9.add("a"); s9.add("b");
console.log(s9.has("a")); // true
s9.delete("a");
console.log(s9.size); // 1

// 10
for (const el of new Set([1, 2, 3])) console.log(el);

// 11
const m11 = new Map();
m11.set("ism", "Oqil");
console.log(m11.get("ism")); // "Oqil"

// 12
const m12 = new Map();
m12.set(1, "son"); m12.set(true, "bool");
console.log(m12.get(1), m12.get(true)); // "son" "bool"

// 13
const m13 = new Map([["a", 1], ["b", 2]]);
console.log(m13.has("a")); // true
m13.delete("a");
```

```

console.log(m13.size);      // 1

// 14
const m14 = new Map([["x", 10], ["y", 20]]);
for (const [k, v] of m14) console.log(`${k}=${v}`);

// 15
const obj = {}; obj[1] = "a"; obj["1"] = "b"; // son kalit string'ga
aylanadi -> bitta kalit
console.log(Object.keys(obj).length); // 1
const map = new Map(); map.set(1, "a"); map.set("1", "b"); // ikki xil kalit
console.log(map.size); // 2

// 16, 17 - named export/import
// 📄 matematika.js
// export const PI = 3.14159;
// export function qoshish(a, b) { return a + b; }
// 📄 main.js
// import { PI, qoshish } from "./matematika.js";
// console.log(qoshish(2, 3)); // 5

// 18 - default export/import
// 📄 User.js
// export default class User { constructor(ism){ this.ism = ism; } }
// 📄 main.js
// import User from "./User.js";
// const u = new User("Oqil");

// 19 - namespace
// import * as mat from "./matematika.js";
// console.log(mat.qoshish(1, 2)); // 3

// 20 - kichik loyiha
// 📄 utils.js
// export const APP = "EduCore";
// export function formatla(son) { return son.toLocaleString(); }
// export default function log(x) { console.log("[LOG]", x); }
// 📄 main.js
// import log, { APP, formatla } from "./utils.js";
// log(APP); // [LOG] EduCore
// console.log(formatla(1000000)); // "1,000,000"

```

22-MODUL: Error handling (xato boshqaruvi)

Yaxshi dastur — xato chiqqanda **qulamaydigan** dastur. Xatolarni to'g'ri ushlab va boshqarish — professional kodning belgisi.

try / catch / finally

```

try {
  // xato chiqishi mumkin bo'lgan kod

```

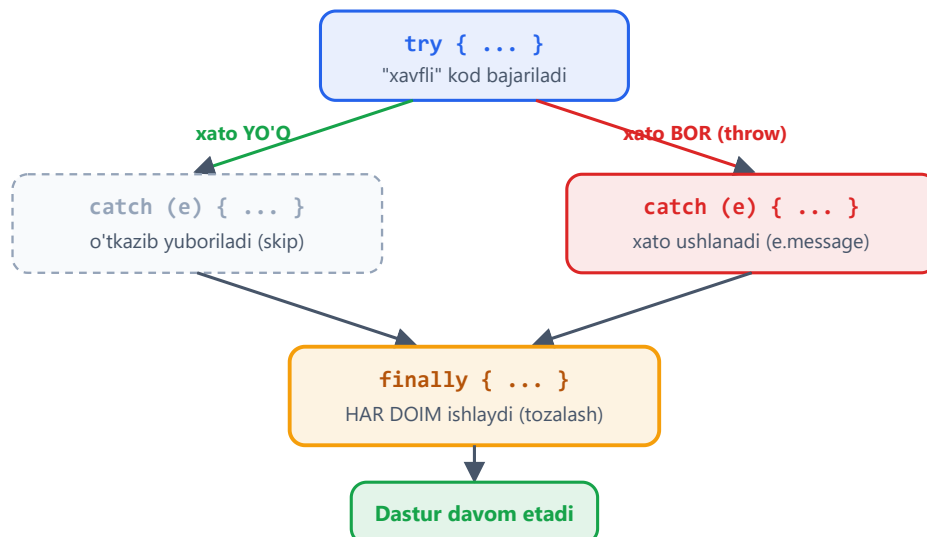


```
const data = JSON.parse("noto'g'ri json");
} catch (xato) {
  // xato bo'lsa shu ishlaydi
  console.log("Xato yuz berdi:", xato.message);
} finally {
  // har holda ishlaydi (xato bo'lsa ham, bo'lmasa ham)
  console.log("Tugadi");
}
```

- **try** — "xavfli" kod.
- **catch** — xato bo'lsa ushlaydi (xato obyektini beradi).
- **finally** — natijadan qat'i nazar (tozalash uchun: faylni yopish, loading'ni o'chirish).

try / catch / finally oqimi

try ichida xato chiqsa catch ushlaydi; finally esa HAR DOIM ishlaydi



throw — xato tashlash

Biror narsa noto'g'ri bo'lsa, o'zing xato "tashlashing" mumkin:

```
function yoshTekshir(yosh) {
  if (yosh < 0) {
    throw new Error("Yosh manfiy bo'lishi mumkin emas");
  }
  return yosh;
}

try {
  yoshTekshir(-5);
} catch (e) {
```

```
console.log(e.message); // "Yosh manfiy bo'lishi mumkin emas"
}
```

Why Error obyekti, string emas: `throw "xato"` deb **string tashlama** — `throw new Error("xato")` deb tashla. Sababi: `Error` obyektida `.message`, `.name`, va `.stack` (xato qayerda bo'lganini ko'rsatadigan iz) bo'ladi. `String`'da bu yo'q — debug qiyinlashadi.

Error obyektlarning xususiyatlari

```
try {
  throw new Error("Nimadir buzildi");
} catch (e) {
  console.log(e.name);    // "Error"
  console.log(e.message); // "Nimadir buzildi"
  console.log(e.stack);   // xato izi (qayerda, qaysi qatorda)
}
```

Built-in xato turlari

JS'da tayyor xato turlari bor:

```
// TypeError – noto'g'ri turdagi amal:
null.foo;           // TypeError: Cannot read properties of null

// ReferenceError – mavjud bo'lmagan o'zgaruvchi:
console.log(yoq);    // ReferenceError: yoq is not defined

// RangeError – ruxsat etilgan diapazondan tashqari:
new Array(-1);       // RangeError

// SyntaxError – noto'g'ri sintaksis:
JSON.parse("{xato}"); // SyntaxError
```

Custom error class'lar

O'z xato turlaringni yaratish (20-moduldagi `class` + `extends`):

```
class ValidationError extends Error {
  constructor(message, maydon) {
    super(message);
    this.name = "ValidationError";
    this.maydon = maydon; // qo'shimcha ma'lumot
  }
}
```

```

}

try {
  throw new ValidationError("Email noto'g'ri", "email");
} catch (e) {
  console.log(e.name); // "ValidationError"
  console.log(e.message); // "email"
}

```

Why custom error: Turli xil xatolarni **farqlash** uchun. `catch` da `if (e instanceof ValidationError)` deb tekshirib, har xil xatoni har xil boshqarasan. (Laravel'dagi custom Exception class'lar bilan bir xil g'oya — sen buni bilasan.)

Aniq xatoni ushlab va qayta tashlash

```

try {
  // ...
} catch (e) {
  if (e instanceof ValidationError) {
    console.log("Validatsiya xatosi:", e.message);
  } else {
    throw e; // noma'lum xatoni qayta tashlaymiz (yashirib qo'ymaymiz)
  }
}

```

Why qayta tashlash: Faqat **boshqara oladigan** xatolarni ushla. Noma'lum xatolarni `catch` da yutib yuborma (`catch {}` bo'sh qoldirma) — ularni qayta tashla yoki log qil. Yashirilgan xato — eng yomon bug.

Async/await'da xato (4-qismdan davom)

```

async function malumotOl() {
  try {
    const res = await fetch("https://api.uz/data");
    if (!res.ok) throw new Error(`Server xatosi: ${res.status}`);
    return await res.json();
  } catch (e) {
    console.log("Xato:", e.message);
    return null; // yoki xatoni qayta tashlash
  }
}

```

1. `try/catch` bilan `JSON.parse("xato")` xatosini ushlang.
2. Ushlangan xatoning `.message` ini chiqaring.
3. `throw new Error(...)` bilan xato tashlang.
4. Funksiya ichida xato tashlab, tashqarida ushlang.
5. `finally` har holda ishlashini ko'rsating (xato bor va yo'q holatda).
6. Xatoning `.name` va `.message` ini ajratib chiqaring.
7. `TypeError` ni ataylab keltirib chiqaring (`null.foo`).
8. `yoshTekshir(yosh)` — manfiy bo'lsa xato tashlasin.
9. Validatsiya funksiyasi: bo'sh string bo'lsa xato tashlasin.
10. **Why:** string tashlash vs `Error` tashlash farqini ko'rsating.
11. Custom `ValidationError` class yozing (`extends Error`).
12. Custom error tashlab, `instanceof` bilan tekshiring.
13. Custom error'ga qo'shimcha xususiyat (`maydon`) qo'shing.
14. Noma'lum xatoni `catch` da qayta tashlang (`throw e`).
15. Ikki xil xatoni (`ValidationError` va boshqa) har xil boshqaring.
16. `async` funksiyada `try/catch` bilan xato ushlang.
17. Custom error'ga `statusCode` qo'shing (HTTP uchun).
18. Ichma-ich `try/catch` yozing.
19. "Xavfsiz" funksiya: `[xato, natija]` ko'rinishida qaytarsin (Go uslubi).
20. **Murakkab:** Validatsiya tizimi — `ValidationError` (maydon bilan); user obyektini tekshiruvchi funksiya (ism, email, yosh); xatolarni ushlab, qaysi maydonda ekanini ko'rsating.



```
// 1, 2
try { JSON.parse("xato"); }
catch (e) { console.log(e.message); } // "Unexpected token..."

// 3
try { throw new Error("Buzildi"); }
catch (e) { console.log(e.message); }

// 4
function f4() { throw new Error("Funksiya xatosi"); }
try { f4(); } catch (e) { console.log(e.message); }

// 5
function test5(xatomi) {
  try { if (xatomi) throw new Error("X"); console.log("ok"); }
  catch (e) { console.log("catch"); }
  finally { console.log("finally"); }
}
test5(false); // ok, finally
test5(true); // catch, finally

// 6
try { throw new Error("Xabar"); }
catch (e) { console.log(e.name, "-", e.message); } // "Error - Xabar"

// 7
try { null.foo; } catch (e) { console.log(e.name); } // "TypeError"

// 8
function yoshTekshir(y) {
  if (y < 0) throw new Error("Yosh manfiy bo'lmasin");
  return y;
}
try { yoshTekshir(-5); } catch (e) { console.log(e.message); }

// 9
function ismTekshir(ism) {
  if (ism.trim() === "") throw new Error("Ism bo'sh bo'lmasin");
  return ism;
}

// 10
// throw "xato" -> e string bo'ladi, .message/.stack yo'q
// throw new Error("xato") -> e obyekt, .message va .stack bor (debug oson)

// 11, 12, 13
class ValidationError extends Error {
  constructor(message, maydon) {
    super(message);
    this.name = "ValidationError";
    this.maydon = maydon;
  }
}
try { throw new ValidationError("Email xato", "email"); }
catch (e) {
  console.log(e instanceof ValidationError); // true
  console.log(e.maydon); // "email"
}
```

```

}

// 14
function f14() {
  try { throw new TypeError("Tur xatosi"); }
  catch (e) {
    if (e instanceof ValidationError) console.log("validatsiya");
    else throw e; // qayta tashlanadi
  }
}
try { f14(); } catch (e) { console.log("Tashqarida:", e.name); } //
"TypeError"

// 15
function boshqar(e) {
  if (e instanceof ValidationError) console.log("Validatsiya:", e.message);
  else console.log("Boshqa xato:", e.message);
}

// 16
async function f16() {
  try { await Promise.reject(new Error("Async xato")); }
  catch (e) { console.log(e.message); }
}
f16();

// 17
class ApiError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.name = "ApiError";
    this.statusCode = statusCode;
  }
}
try { throw new ApiError("Topilmadi", 404); }
catch (e) { console.log(e.statusCode); } // 404

// 18
try {
  try { throw new Error("ichki"); }
  catch (e) { throw new Error("tashqi: " + e.message); }
} catch (e) { console.log(e.message); } // "tashqi: ichki"

// 19 - Go uslubi
function xavfsiz(fn) {
  try { return [null, fn()]; }
  catch (e) { return [e, null]; }
}
const [xato, natija] = xavfsiz(() => JSON.parse("{}"));
console.log(xato, natija); // null {}

// 20 - Validatsiya tizimi
class ValError extends Error {
  constructor(msg, maydon) { super(msg); this.name = "ValError"; this.maydon
= maydon; }
}
function userTekshir(user) {
  if (!user.ism || user.ism.length < 3) throw new ValError("Ism ≥ 3", "ism");
  if (!/^[^\s@]+\.[^\s@]+\.[^\s@]+$/.test(user.email)) throw new
ValError("Email xato", "email");
  if (user.yosh < 0 || user.yosh > 150) throw new ValError("Yosh noto'g'ri",

```

```

    "yosh");
    return true;
  }
  try {
    userTekshir({ ism: "Oqil", email: "xato", yosh: 25 });
  } catch (e) {
    console.log(`Xato (${e.maydon}): ${e.message}`); // "Xato (email): Email
    xato"
  }
}

```

23-MODUL: Regular expressions (Regex)

Regex — **matn naqshlari** bilan ishlash vositasi: tekshirish (validatsiya), qidirish, almashtirish. Boshida qo'rqinchli ko'rinadi, lekin asoslarini bilsang juda kuchli.

Regex yaratish va `test`

```

const naqsh = /salom/;           // literal (eng ko'p)
const naqsh2 = new RegExp("salom"); // konstruktor (dinamik holatda)

console.log(naqsh.test("salom dunyo")); // true - matnda bormi?
console.log(naqsh.test("xayr"));        // false

```

Bayroqlar (flags)

```

/salom/g // global - barcha mosliklarni topadi (faqat birinchi emas)
/salom/i // insensitive - katta-kichik harfni farqlamaydi
/salom/m // multiline - ko'p qatorli matn uchun

const re = /a/gi;
console.log("Apple aA".match(re)); // ["A", "a", "A"]

```

Maxsus belgilar (character classes)

```

\d // har qanday raqam (0-9)
\w // harf, raqam yoki _ (so'z belgisi)
\s // bo'sh joy (probel, tab, yangi qator)
. // har qanday belgi (yangi qatordan tashqari)

\D \W \S // teskarisi (raqam EMAS, so'z belgisi EMAS, probel EMAS)

```

```
console.log(/\d/.test("abc5")); // true - raqam bormi?  
console.log("a1b2c3".match(/\d/g)); // ["1", "2", "3"]
```

Anchorlar (boshi/oxiri)

```
^ // matn boshi  
$ // matn oxiri  
  
console.log(/^salom/.test("salom dunyo")); // true - "salom" bilan  
boshlanadimi?  
console.log(/dunyo$/ .test("salom dunyo")); // true - "dunyo" bilan  
tugaydimi?  
console.log(/^abc$/ .test("abc")); // true - to'liq "abc" mi?
```

Miqdorlar (quantifiers)

```
* // 0 yoki ko'p marta  
+ // 1 yoki ko'p marta  
? // 0 yoki 1 marta (ixtiyoriy)  
{n} // aniq n marta  
{n,m} // n dan m gacha  
{n,} // kamida n marta
```

```
console.log(/\d+/.test("abc")); // false - kamida 1 raqam yo'q  
console.log(/^\d{4}$/ .test("2024")); // true - aniq 4 ta raqam  
console.log("aaa".match(/a+/)); // ["aaa"]
```

To'plamlar va guruhlar

```
[abc] // a, b yoki c  
[a-z] // a dan z gacha  
[0-9] // raqam (= \d)  
[^abc] // a, b, c EMAS  
(...) // guruh (ushlab oladi)  
a|b // a YOKI b
```

```
console.log(/[aeiou]/.test("xyz")); // false - unli yo'q  
console.log(/^(ha|yoq)$/ .test("ha")); // true
```

`match`, `replace`, `matchAll`

```
// match – mosliklarni topadi:
console.log("Narx: 100, 200, 300".match(/\d+/g)); //
["100", "200", "300"]

// replace – almashtiradi:
console.log("salom dunyo".replace(/dunyo/, "JS")); // "salom JS"
console.log("a-b-c".replace(/-/g, "+")); // "a+b+c"

// Guruh bilan almashtirish ($1):
console.log("2024-01-15".replace(/(\d{4})-(\d{2})-(\d{2})/,
"$3/$2/$1"));
// "15/01/2024" – kun oy/yil
```

Amaliy naqshlar

```
// Email:
const email = /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/;
console.log(email.test("oqil@mail.uz")); // true

// O'zbek telefon (+998 va 9 ta raqam):
const tel = /^[+998\\d{9}]$/;
console.log(tel.test("+998901234567")); // true

// Faqat raqamlar:
const faqatSon = /^[\\d+}$/;
console.log(faqatSon.test("12345")); // true

// Matndan barcha sonlarni ajratish:
console.log("3 ta olma, 5 ta uzum".match(/\\d+/g)); // ["3", "5"]
```

Why regex (va ehtiyotkorlik): Regex – validatsiya, qidirish, ma'lumot ajratishda tengsiz. Lekin (1) murakkab regex'ni o'qish qiyin – izoh yoz; (2) email kabi narsalar uchun haddan tashqari murakkab regex yozma – oddiy `/^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/` 99% holatda yetarli; (3) noma'lum regex'ni regex101.com'da sinab ko'r.

23-modul masalalari (20 ta)

1. `/salom/` regex yaratib, `test` bilan matnda borligini tekshiring.
2. `i` bayrog'i bilan katta-kichik harfsiz qidiring.
3. `\\d` bilan matnda raqam borligini tekshiring.
4. `g` bayrog'i bilan barcha raqamlarni `match` qiling.
5. `\\w` va `\\s` ni ishlatib ko'ring.

6. `^` va `$` bilan boshlanish/tugashni tekshiring.
7. `+` bilan "kamida bitta raqam" ni tekshiring.
8. `{4}` bilan aniq 4 ta raqamni tekshiring (yil).
9. `{2,4}` bilan 2-4 ta belgini tekshiring.
10. `?` bilan ixtiyoriy belgini tekshiring (`colou?r`).
11. `[aeiou]` bilan unli harf borligini tekshiring.
12. `[^0-9]` bilan raqam bo'lmagan belgi borligini tekshiring.
13. `(ha|yoq)` guruh va alternatsiya bilan tekshiring.
14. `replace` bilan barcha `-` larni `/` ga almashtiring.
15. Guruh `$1` bilan sanani `yyyy-mm-dd` dan `dd/mm/yyyy` ga aylantiring.
16. Matndan barcha sonlarni ajratib oling (`"5 olma, 3 nok"`).
17. Email'ni regex bilan tekshiring.
18. O'zbek telefon raqamini (`+998XXXXXXXXX`) tekshiring.
19. `split` ni regex bilan: bir nechta ajratuvchi (`,` `;` ; probel) bo'yicha bo'ling.
20. **Murakkab:** parol kuchini tekshiring — kamida 8 belgi, **kamida** 1 katta harf, 1 kichik harf, 1 raqam (har birini alohida regex bilan).



```
// 1
console.log(/salom/.test("salom dunyo")); // true

// 2
console.log(/salom/i.test("SALOM")); // true

// 3
console.log(/\d/.test("abc5")); // true

// 4
console.log("a1b2c3".match(/\d/g)); // ["1", "2", "3"]

// 5
console.log(/\w/.test("_")); // true
console.log(/\s/.test("a b")); // true

// 6
console.log(/^salom/.test("salom dunyo")); // true
console.log(/dunyo$/ .test("salom dunyo")); // true

// 7
console.log(/\d+/.test("abc")); // false
console.log(/\d+/.test("abc1")); // true

// 8
console.log(/^\d{4}$/ .test("2024")); // true
console.log(/^\d{4}$/ .test("202")); // false

// 9
console.log(/^w{2,4}$/ .test("abc")); // true

// 10
console.log(/colou?r/.test("color")); // true
console.log(/colou?r/.test("colour")); // true

// 11
console.log(/[aeiou]/.test("xyz")); // false
console.log(/[aeiou]/.test("xay")); // true

// 12
console.log(/[^0-9]/.test("12a34")); // true (a raqam emas)

// 13
console.log(/^(ha|yoq)$/ .test("ha")); // true
console.log(/^(ha|yoq)$/ .test("balki")); // false

// 14
console.log("2024-01-15".replace(/-/g, "/")); // "2024/01/15"

// 15
console.log("2024-01-15".replace(/(\d{4})-(\d{2})-(\d{2})/, "$3/$2/$1"));
// "15/01/2024"

// 16
console.log("5 olma, 3 nok".match(/\d+/g)); // ["5", "3"]

// 17
```

```

console.log(/^[\s@]+@[\s@]+\.[\s@]+$/ .test("oqil@mail.uz")); // true

// 18
console.log(/^+998\d{9}$/ .test("+998901234567")); // true
console.log(/^+998\d{9}$/ .test("901234567")); // false

// 19
console.log("a, b; c d".split(/[,;\s]+/)); // ["a", "b", "c", "d"]

// 20 - parol kuchi
function parolKuchi(parol) {
  const xatolar = [];
  if (parol.length < 8) xatolar.push("kamida 8 belgi");
  if (!/[A-Z]/.test(parol)) xatolar.push("katta harf kerak");
  if (!/[a-z]/.test(parol)) xatolar.push("kichik harf kerak");
  if (!/\d/.test(parol)) xatolar.push("raqam kerak");
  return xatolar.length === 0 ? "Kuchli" : xatolar;
}
console.log(parolKuchi("abc")); // ["kamida 8 belgi", "katta harf
kerak", "raqam kerak"]
console.log(parolKuchi("Abc12345")); // "Kuchli"

```

✓ 5-qism to'liq yakuni (1 + 2 bo'lim)

5-qismni tugatding — endi JS'ni **chuqur** bilasan:

1-bo'lim (OOP yadrosi): Scope · Closures · Hoisting · `this` · Prototypes · Classes
2-bo'lim (zamonaviy vositalar): ES6+ (`??` , Set, Map) · Modullar (`import / export`) · Error handling (custom errors) · Regex

Bu darajada sen: - Closure va `this` ni tushunasan (junior'ni middle'dan ajratadigan bilim) - Class hierarchy va polimorfizm yozasan - Kodni modullarga bo'lasan (real loyiha strukturasini) - Xatolarni professional boshqarasan - Regex bilan validatsiya/qidirish qilasan

Keyingi qadam (6-qism) — EXPERT

Oxirgi qism. Bu yerda "yaxshi dasturchi"ni "expert"dan ajratadigan mavzular: -
Functional Programming — pure functions, immutability, composition, currying -
Iterators & Generators — `function*` , `yield` , lazy evaluation; Symbols - **Proxy & Reflect** — obyekt xatti-harakatini ushlab qolish (Vue 3 reaktivligi shu ustiga qurilgan!) -
Performance & Memory — debounce/throttle, memoization, memory leaks - **Design Patterns** — Singleton, Factory, Observer, Module (real arxitektura) - **Bundlers & Build Tools** — Vite, npm, tree-shaking (konseptual) - **TypeScript'ga ko'prik** — JS'dan keyingi tabiiy qadam

Maslahat: 6-qismga o'tishdan oldin 22-modul #20 (validatsiya tizimi) va 23-modul #20 (parol kuchi)ni birlashtirib, to'liq forma validatsiya moduli yoz — custom error + regex + class birga. Bu — real loyihada ishlatadigan kod.

 [README](#) · [← 5-qism \(1-bo'lim\)](#) · [Keyingi: 6-qism \(1-bo'lim\)](#) — [Expert](#) →

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 [README](#) · [← 5-qism \(2-bo'lim\)](#) · [Keyingi: 6-qism \(2-bo'lim\)](#) — [Yakun](#) →

6-QISM: EXPERT — 1-bo'lim "Yaxshi dasturchi"ni "expert"dan ajratadigan mavzular: FP, generatorlar, Proxy, performance. Har bir moduldan keyin **20 ta masala** (yechimi bilan). Bu — oxirgi qism.

6-qism eng katta (7 modul), ikkiga bo'lindi: **1-bo'lim** (shu fayl): 24-FP · 25-Generators/Iterators/Symbols · 26-Proxy/Reflect · 27-Performance/Memory **2-bo'lim** (keyingi): 28-Design Patterns · 29-Bundlers · 30-TypeScript ko'prigi

24-MODUL: Functional Programming (FP)

FP — kodni **sof funksiyalar** va **o'zgarmas ma'lumot** orqali qurish uslubi. Bashorat qilinadigan, test qilish oson, kam bug'li kod beradi. (Sening DDD/SOLID foning bilan yaxshi qovushadi.)

Pure functions (sof funksiyalar) — chuqurroq

Sof funksiya: (1) bir xil kirishga **doim bir xil chiqish**; (2) **side-effect yo'q** (tashqi narsani o'zgartirmaydi).

```
// ✅ Pure: faqat kirishga bog'liq, tashqarini buzmaydi
const qoshish = (a, b) => a + b;

// ❌ Impure: tashqi o'zgaruvchini o'zgartiradi
let jami = 0;
const qoshImpure = (n) => { jami += n; };

// ❌ Impure: tashqi holatga bog'liq (har safar boshqa natija)
const tasodif = () => Math.random();
```

Why pure: Sof funksiyani test qilish oson (mock kerak emas), qayta ishlatlsa bo'ladi, parallel ishlatlsa xavfsiz, va keshlash mumkin (memoize). Loyihaning **yadrosini** sof funksiyalardan qur; side-effect'larni (DOM, network, log) chetga sur.

Immutability (o'zgarmaslik)

Ma'lumotni **o'zgartirma** — o'zgartirilgan **yangi nusxa** qaytar (2-qismdagi spread'ni esla):

```
const arr = [1, 2, 3];

// ❌ Mutatsiya:
arr.push(4); // asilni buzadi

// ✅ Immutable:
const yangi = [...arr, 4]; // qo'shish
const ochirilgan = arr.filter(x => x !== 2); // o'chirish
const yangilangan = arr.map(x => x === 2 ? 20 : x); // o'zgartirish

const user = { ism: "Oqil", yosh: 25 };
const yangiUser = { ...user, yosh: 26 }; // user buzilmaydi
```

Why immutable: Mutatsiya — kutilmagan bug'lar manbai (boshqa joy o'sha obyektga havola qilgan bo'lishi mumkin). React/Vue state'i aynan shu prinsipga tayanadi: yangi nusxa → o'zgarish aniqlanadi → qayta render. `push` / `splice` / `sort` o'rniga `map` / `filter` / `[...]` ishlat.

Function Composition (funksiyalarni birlashtirish)

Kichik funksiyalardan kattalarini yig'ish. `pipe` — chapdan o'ngga, `compose` — o'ngdan chapga:

```
const pipe = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);

const trim = s => s.trim();
const kichik = s => s.toLowerCase();
const boshKatta = s => s[0].toUpperCase() + s.slice(1);

const formatla = pipe(trim, kichik, boshKatta);
console.log(formatla("  SALOM  ")); // "Salom"
// trim -> "SALOM" -> kichik -> "salom" -> boshKatta -> "Salom"
```

pipe(trim, kichik, boshKatta) — data oqadi



Why composition: Murakkab amalni kichik, sinaladigan bo'laklarga bo'lasan, keyin ularni "quvur" qilib ulaysan. Har bo'lak bitta ishni qiladi (single responsibility). Bu — FP'ning yuragi.

Currying (karrilash)

Ko'p argumentli funktsiyani **bittadan argument oluvchi** funktsiyalar zanjiriga aylantirish (17-moduldagi closure/factory'ni esla):

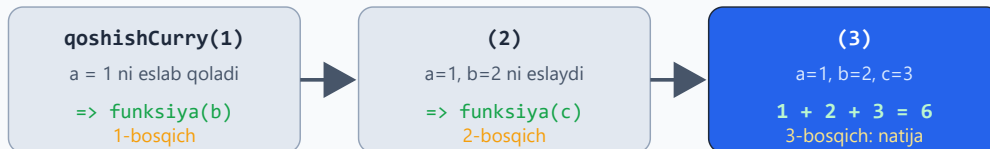
```
// Oddiy:
const qoshish = (a, b, c) => a + b + c;

// Curried:
const qoshishCurry = a => b => c => a + b + c;
console.log(qoshishCurry(1)(2)(3)); // 6

// Foydasi — partial application (qisman qo'llash):
const besh = qoshishCurry(5);
const beshUch = besh(3);
console.log(beshUch(2)); // 10
```

Currying: $a \Rightarrow b \Rightarrow c \Rightarrow a + b + c$

`qoshishCurry(1)(2)(3) // 6`



Partial application (qisman qo'llash): oraliq bosqichni saqlab qo'yasan

```
const besh = qoshishCurry(5); // a=5 biriktirildi, qayta ishlatish mumkin  
const beshUch = besh(3); // beshUch(2) => 10
```

Amaliy misol:

```
const kopaytir = a => b => a * b;  
const ikkilantir = kopaytir(2);  
const unlantir = kopaytir(10);  
  
console.log([1, 2, 3].map(ikkilantir)); // [2, 4, 6]  
console.log([1, 2, 3].map(unlantir)); // [10, 20, 30]
```

Why currying: Konfiguratsiyani "oldindan biriktirish" imkonini beradi. `kopaytir(2)` — qayta ishlatiladigan "ikkilantiruvchi" funksiya. Mapping/filtering'da juda qulay.

Declarative vs Imperative

```
const sonlar = [1, 2, 3, 4, 5, 6];  
  
// ❌ Imperative (QANDAY qilishni aytasan):  
let natija = [];  
for (let i = 0; i < sonlar.length; i++) {  
  if (sonlar[i] % 2 === 0) natija.push(sonlar[i] * 2);  
}  
  
// ✅ Declarative (NIMA kerakligini aytasan):  
const natija2 = sonlar.filter(n => n % 2 === 0).map(n => n * 2);
```

Why declarative: Niyatni ifodalaydi — "juftlarni olib, ikkilantir". Sikldagi indeks/holat boshqaruvi yo'q, shuning uchun xato kam, o'qish oson.

1. Sof funksiya yozing (deterministic, side-effect'siz).
2. Impure funksiyanı pure qiling (tashqi o'zgaruvchiga tegmasin).
3. Immutable: massivga element qo'shing (`push` emas, `spread`).
4. Immutable: obyekt xususiyatini yangilang (`spread`).
5. Immutable: massivdan element o'chiring (`filter` , `splice` emas).
6. `map` / `filter` / `reduce` bilan declarative kod yozing (sikl o'rniga).
7. Ikki funksiyanı birlashtiruvchi oddiy `compose` yozing.
8. `compose(...fns)` ni `reduceRight` bilan yozing (ko'p funksiya).
9. `pipe(...fns)` ni `reduce` bilan yozing.
10. `pipe` bilan string'ni quvurdan o'tkazing (`trim`→`kichik`→`bosh-katta`).
11. Currying: `qoshish(a)(b)(c)` yozing.
12. Curried funksiya dan partial application qiling (`besh = qoshish(5)`).
13. `kopaytir(a)(b)` curried funksiya + `ikkilantir = kopaytir(2)` .
14. Higher-order: funksiya qaytaruvchi funksiya yozing.
15. `reduce` bilan `map` ni qayta yozing (reduce universalligi).
16. `reduce` bilan `filter` ni qayta yozing.
17. Composition bilan sonni transformatsiya qiling (`+1` → `*2` → `kvadrat`).
18. Immutable ichma-ich (nested) yangilash (`spread` ichma-ich).
19. Funksiyalar zanjiri: `kopaytiruvchi(n)` factory bilan turli ko'paytiruvchilar.
20. **Murakkab:** `pipe` bilan ma'lumot pipeline — mahsulotlar massivi → narxi 100+ filterlash → 10% chegirma map → jami `reduce` . Hammasini sof funksiyalardan yig'ing.



Yechimlar



```
// 1
const qoshish = (a, b) => a + b;

// 2
// Impure: let total=0; const add=n=>total+=n;
// Pure:
const add = (total, n) => total + n; // holatni qaytaradi, o'zgartirmaydi

// 3
const arr = [1, 2, 3];
console.log([...arr, 4]); // [1,2,3,4], arr buzilmadi

// 4
const user = { ism: "Oqil", yosh: 25 };
console.log({ ...user, yosh: 26 }); // user buzilmadi

// 5
console.log([1, 2, 3].filter(x => x !== 2)); // [1,3]

// 6
console.log([1, 2, 3, 4].filter(n => n % 2 === 0).map(n => n * 2)); // [4,8]

// 7
const compose2 = (f, g) => x => f(g(x));
const inc = x => x + 1, dbl = x => x * 2;
console.log(compose2(inc, dbl)(5)); // 11 (5*2+1)

// 8
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);
console.log(compose(inc, dbl)(5)); // 11

// 9
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);
console.log(pipe(inc, dbl)(5)); // 12 ((5+1)*2)

// 10
const trim = s => s.trim();
const kichik = s => s.toLowerCase();
const boshKatta = s => s[0].toUpperCase() + s.slice(1);
console.log(pipe(trim, kichik, boshKatta)(" SALOM ")); // "Salom"

// 11
const qoshish3 = a => b => c => a + b + c;
console.log(qoshish3(1)(2)(3)); // 6

// 12
const besh = qoshish3(5);
console.log(besh(3)(2)); // 10

// 13
const kopaytir = a => b => a * b;
const ikkilantir = kopaytir(2);
console.log([1, 2, 3].map(ikkilantir)); // [2,4,6]

// 14
const qoshuvchiYasa = n => x => x + n;
console.log(qoshuvchiYasa(10)(5)); // 15
```

```

// 15
const mapReduce = (arr, fn) => arr.reduce((acc, x) => [...acc, fn(x)], []);
console.log(mapReduce([1, 2, 3], x => x * 2)); // [2,4,6]

// 16
const filterReduce = (arr, fn) => arr.reduce((acc, x) => fn(x) ? [...acc, x] : acc, []);
console.log(filterReduce([1, 2, 3, 4], x => x % 2 === 0)); // [2,4]

// 17
const kvadrat = x => x * x;
console.log(pipe(inc, dbl, kvadrat)(3)); // ((3+1)*2)^2 = 64

// 18
const state = { user: { ism: "Oqil", manzil: { shahar: "Toshkent" } } };
const yangi = { ...state, user: { ...state.user, manzil: {
...state.user.manzil, shahar: "Samarqand" } } };
console.log(yangi.user.manzil.shahar); // "Samarqand", state buzilmadi

// 19
const kop = n => x => x * n;
console.log([1, 2, 3].map(kop(3))); // [3,6,9]
console.log([1, 2, 3].map(kop(10))); // [10,20,30]

// 20 - pipeline
const mahsulotlar = [
  { nom: "A", narx: 150 },
  { nom: "B", narx: 50 },
  { nom: "C", narx: 200 },
];
const qimmatlar = arr => arr.filter(m => m.narx >= 100);
const chegirma = arr => arr.map(m => ({ ...m, narx: m.narx * 0.9 }));
const jami = arr => arr.reduce((s, m) => s + m.narx, 0);

const natija = pipe(qimmatlar, chegirma, jami)(mahsulotlar);
console.log(natija); // (150*0.9 + 200*0.9) = 315

```

25-MODUL: Iterators, Generators, Symbols

`for...of` qanday ishlaydi, cheksiz ketma-ketliklarni qanday yasash mumkin — bu modul shuni ochadi.

Iterator protokoli

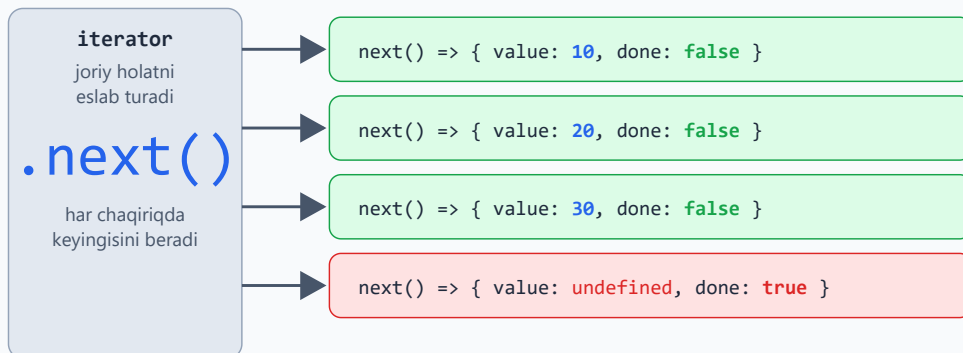
`for...of` ishlashi uchun obyekt **iterable** bo'lishi kerak — ya'ni `Symbol.iterator` metodiga ega bo'lishi va u `next()` qaytaradigan iterator berishi kerak. `next()` har safar `{ value, done }` qaytaradi:

```
const arr = [10, 20, 30];
const it = arr[Symbol.iterator](); // massiv iterator beradi

console.log(it.next()); // { value: 10, done: false }
console.log(it.next()); // { value: 20, done: false }
console.log(it.next()); // { value: 30, done: false }
console.log(it.next()); // { value: undefined, done: true }
```

Iterator protokoli: next() => { value, done }

```
const it = [10, 20, 30][Symbol.iterator]();
```



done: true bo'lganda for...of va spread to'xtaydi — qiymatlar tugadi

Why bu muhim: `for...of`, `spread ([...x])`, destructuring — hammasi shu protokolga tayanadi. Massiv, string, Set, Map — barchasi iterable (`Symbol.iterator` 'ga ega). Obyekt esa **iterable emas** — shuning uchun obyekt ustida `for...of` ishlamaydi (`Object.entries` kerak).

Custom iterable (o'z obyektini iterable qilish)

```
const oraliq = {
  from: 1,
  to: 5,
  [Symbol.iterator]() {
    let joriy = this.from;
    const oxir = this.to;
    return {
      next() {
        return joriy <= oxir
          ? { value: joriy++, done: false }
          : { value: undefined, done: true };
      },
    };
  },
};
```

```
console.log([...oraliq]); // [1, 2, 3, 4, 5]
for (const n of oraliq) console.log(n); // 1,2,3,4,5
```

Generators (function*)

Generator — iterator'ni **avtomatik** yaratadigan funksiya. `function*` bilan e'lon qilinadi, `yield` bilan qiymat "uzatadi" va **to'xtaydi**:

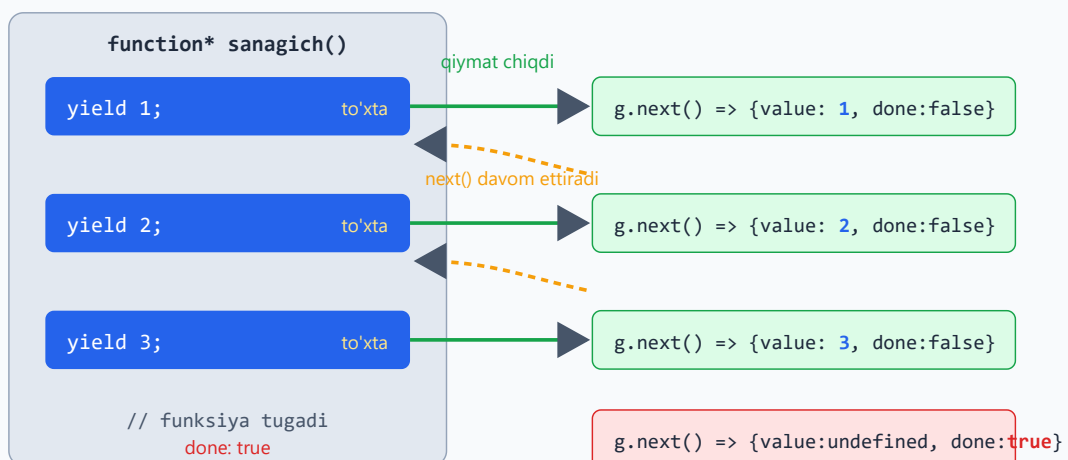
```
function* sanagich() {
  yield 1;
  yield 2;
  yield 3;
}

const g = sanagich();
console.log(g.next()); // { value: 1, done: false }
console.log(g.next()); // { value: 2, done: false }
console.log(g.next()); // { value: 3, done: false }
console.log(g.next()); // { value: undefined, done: true }

// for...of va spread bilan ham ishlaydi:
console.log([...sanagich()]); // [1, 2, 3]
```

Why generators sehrli: Oddiy funksiya bir marta ishlab tugaydi. Generator esa **to'xtab-davom etadi** — har `yield` da to'xtaydi, `next()` da davom etadi. Bu — holatni saqlab turadigan, "talab bo'yicha" qiymat beradigan funksiya. Iterator'ni qo'lda yozishdan ancha qulay (yuqoridagi `oraliq` ni taqqosla).

Generator: yield da to'xtaydi, next() da davom etadi



Cheksiz ketma-ketlik (lazy evaluation)

Generator faqat **so'ralganda** hisoblaydi — shuning uchun cheksiz ketma-ketlik ham xavfsiz:

```
function* cheksizSanagich() {  
  let i = 0;  
  while (true) {  
    yield i++; // hech qachon tugamaydi, lekin faqat so'ralganda  
    ishlaydi  
  }  
}  
  
const g = cheksizSanagich();  
console.log(g.next().value); // 0  
console.log(g.next().value); // 1  
console.log(g.next().value); // 2  
// xotira to'lib ketmaydi — faqat 3 ta qiymat hisoblendi
```

```
function* fibonachchi() {  
  let [a, b] = [0, 1];  
  while (true) {  
    yield a;  
    [a, b] = [b, a + b];  
  }  
}  
  
const fib = fibonachchi();  
console.log(fib.next().value, fib.next().value, fib.next().value); //  
0, 1, 1
```

yield* — delegatsiya

Bir generatorni boshqasiga "ulash":

```
function* abc() { yield "a"; yield "b"; }  
function* raqamlar() {  
  yield 1;  
  yield* abc(); // abc'ning barcha qiymatlarini "uzatadi"  
  yield 2;  
}  
  
console.log([...raqamlar()]); // [1, "a", "b", 2]
```

Symbols

Symbol — **noyob (unikal)** identifikator. Har bir Symbol() butunlay yangi:

```
const a = Symbol("tavsif");  
const b = Symbol("tavsif");
```

```
console.log(a === b); // false - tavsif bir xil bo'lsa ham, har biri  
noyob
```

```
// Obyekt kaliti sifatida (nom to'qnashuvidan himoya):
```

```
const ID = Symbol("id");  
const user = { ism: "Oqil", [ID]: 12345 };  
console.log(user[ID]); // 12345
```

Why Symbol: Obyektga "yashirin", to'qnashmaydigan kalit qo'shish uchun. Boshqa kod xuddi shu nomli kalit qo'shsa ham, Symbol noyob bo'lgani uchun konflikt bo'lmaydi. `Symbol.iterator` — eng mashhur "well-known symbol": u obyektни iterable qiladi.

25-modul masalalari (20 ta)

1. Generator yozing — 3 ta `yield`, `next()` bilan oling.
2. Generator bilan `for...of` aylaning.
3. Generatorni spread bilan massivga aylantiring (`[...gen]`).
4. Massivning iteratorini qo'lda oling (`arr[Symbol.iterator]()`) va `next()` chaqiring.
5. Cheksiz sanagich generator yozing va 3 ta qiymat oling (xotira to'lmasin).
6. `oraliq(from, to)` generator yozing.
7. Fibonachchi generator yozing (birinchi 5 ta son).
8. `yield*` bilan ikki generatorni ulang.
9. Generator'da `return` qiymati nima bo'lishini ko'rsating (`done: true` bilan).
10. Custom iterable: obyektga `[Symbol.iterator]` qo'shing.
11. Shu custom iterable bilan `for...of` aylaning.
12. `Symbol()` noyobligini ko'rsating (`Symbol() !== Symbol()`).
13. `Symbol`'ni obyekt kaliti sifatida ishlating.
14. Massiv `Symbol.iterator` 'ga ega ekanini ko'rsating.
15. Generator bilan lazy evaluation: faqat so'ralgan qiymat hisoblanishini ko'rsating.
16. Generator bilan auto-increment ID yaratuvchi yozing.
17. Generator'ga `next(qiymat)` orqali qiymat uzating (ikki tomonlama).
18. Iterable obyektни destructuring bilan oching (`const [a, b] = obj`).
19. `take(gen, n)` funksiyasi: cheksiz generatorдан faqat n ta oling.

20. **Murakkab:** custom `Collection` class yozing — `Symbol.iterator` bilan, `for...of` va `spread` ishlasin, `add` metodi bo'lsin.



```
// 1
function* g1() { yield 1; yield 2; yield 3; }
const it1 = g1();
console.log(it1.next().value, it1.next().value); // 1 2

// 2
for (const x of g1()) console.log(x); // 1,2,3

// 3
console.log([...g1()]); // [1,2,3]

// 4
const it4 = [10, 20][Symbol.iterator]();
console.log(it4.next()); // {value:10, done:false}

// 5
function* cheksiz() { let i = 0; while (true) yield i++; }
const c = cheksiz();
console.log(c.next().value, c.next().value, c.next().value); // 0,1,2

// 6
function* oraliq(from, to) { for (let i = from; i <= to; i++) yield i; }
console.log([...oraliq(2, 5)]); // [2,3,4,5]

// 7
function* fib() { let [a, b] = [0, 1]; while (true) { yield a; [a, b] = [b, a + b]; } }
const f = fib();
console.log([f.next().value, f.next().value, f.next().value, f.next().value, f.next().value]); // [0,1,1,2,3]

// 8
function* abc() { yield "a"; yield "b"; }
function* aralash() { yield 1; yield* abc(); yield 2; }
console.log([...aralash()]); // [1,"a","b",2]

// 9
function* g9() { yield 1; return 99; yield 2; }
const it9 = g9();
console.log(it9.next()); // {value:1, done:false}
console.log(it9.next()); // {value:99, done:true} (return)
console.log(it9.next()); // {value:undefined, done:true}

// 10, 11
const oraliqObj = {
  from: 1, to: 3,
  [Symbol.iterator]() {
    let i = this.from; const oxir = this.to;
    return { next: () => i <= oxir ? { value: i++, done: false } : { value: undefined, done: true } };
  },
};
console.log([...oraliqObj]); // [1,2,3]
for (const n of oraliqObj) console.log(n); // 1,2,3

// 12
console.log(Symbol("x") === Symbol("x")); // false
```

```

// 13
const ID = Symbol("id");
const u13 = { ism: "Oqil", [ID]: 1 };
console.log(u13[ID]); // 1

// 14
console.log(typeof [[Symbol.iterator]]); // "function"

// 15
function* lazy() { console.log("hisoblanmoqda"); yield 1; }
const l = lazy(); // hali "hisoblanmoqda" chiqmaydi
l.next(); // endi chiqadi -> lazy

// 16
function* idGen() { let id = 1; while (true) yield id++; }
const gen = idGen();
console.log(gen.next().value, gen.next().value); // 1, 2

// 17
function* ikkiTomon() {
  const ism = yield "Isming?";
  yield `Salom, ${ism}`;
}
const t = ikkiTomon();
console.log(t.next().value); // "Isming?"
console.log(t.next("Oqil").value); // "Salom, Oqil"

// 18
console.log([...oraliqObj]); // destructuring uchun:
const [birinchi, ikkinchi] = oraliqObj;
console.log(birinchi, ikkinchi); // 1 2

// 19
function take(gen, n) {
  const r = [];
  for (const x of gen) { if (r.length >= n) break; r.push(x); }
  return r;
}
console.log(take(cheksiz(), 4)); // [0,1,2,3]

// 20
class Collection {
  #items = [];
  add(x) { this.#items.push(x); return this; }
  *[Symbol.iterator]() { yield* this.#items; }
}
const col = new Collection();
col.add(1).add(2).add(3);
console.log([...col]); // [1,2,3]
for (const x of col) console.log(x); // 1,2,3

```

26-MODUL: Proxy va Reflect

Proxy — obyekt ustida amallarni (o'qish, yozish, o'chirish) **ushlab qolish** imkonini beradi. Bu — Vue 3 reaktivligi, validatsiya, log, default qiymatlar ortidagi mexanizm.

Proxy asoslari

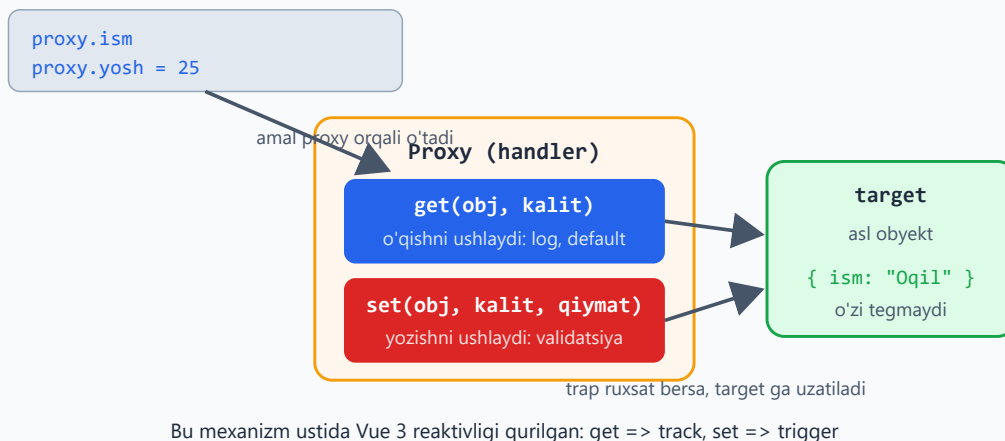
`new Proxy(target, handler)` — `target` obyektini o'rab, `handler` dagi "trap"lar orqali amallarni ushlaydi:

```
const target = { ism: "Oqil" };

const proxy = new Proxy(target, {
  get(obj, kalit) {
    console.log(`O'qildi: ${kalit}`);
    return obj[kalit];
  },
  set(obj, kalit, qiymat) {
    console.log(`Yozildi: ${kalit} = ${qiymat}`);
    obj[kalit] = qiymat;
    return true; // set MUVAFFAQIYATLI ekanini bildiradi (majburiy)
  },
});

proxy.ism; // "O'qildi: ism"
proxy.yosh = 25; // "Yozildi: yosh = 25"
```

Proxy: trap o'qish/yozishni ushlab qoladi



Eng ko'p ishlatiladigan trap'lar

```
const handler = {
  get(obj, kalit) {}, // o'qish: obj.x
  set(obj, kalit, qiymat) {}, // yozish: obj.x = 1 (true qaytarish kerak)
```

```
has(obj, kalit) {}, // "x" in obj
deleteProperty(obj, kalit) {}, // delete obj.x
};
```

Amaliy: default qiymat

```
const obj = new Proxy({}, {
  get(target, kalit) {
    return kalit in target ? target[kalit] : `${kalit} topilmadi`;
  },
});

obj.bor = 1;
console.log(obj.bor); // 1
console.log(obj.yoq); // "yoq" topilmadi (xato emas!)
```

Amaliy: validatsiya

```
const user = new Proxy({}, {
  set(target, kalit, qiymat) {
    if (kalit === "yosh" && (typeof qiymat !== "number" || qiymat < 0)) {
      throw new Error("Yosh musbat son bo'lishi kerak");
    }
    target[kalit] = qiymat;
    return true;
  },
});

user.yosh = 25; // ✅
// user.yosh = -5; // ❌ Error
```

Reflect — standart amallarning "rasmiy" versiyasi

Reflect — obyekt amallarini bajaradigan metodlar to'plami (`Reflect.get`, `Reflect.set`, `Reflect.has`). Ular Proxy trap'lari ichida ishlatiladi — "standart xatti-harakatni" toza chaqirish uchun:

```
const proxy = new Proxy({ ism: "Oqil" }, {
  get(target, kalit, receiver) {
    console.log(`O'qildi: ${kalit}`);
    return Reflect.get(target, kalit, receiver); // standart o'qish
  },
  set(target, kalit, qiymat, receiver) {
    return Reflect.set(target, kalit, qiymat, receiver); // standart
```

```
yozish (true qaytaradi)
  },
});
```

Why Reflect: `target[kalit]` o'rniga `Reflect.get(target, kalit, receiver)` ishlatish — `receiver` (getter/inheritance) holatlarini to'g'ri boshqaradi va `set` avtomatik `true / false` qaytaradi. Qoida: Proxy trap ichida standart amalni `Reflect` orqali bajar.

★ Vue 3 reaktivligi — Proxy ustida qurilgan

Vue 3'ning `reactive()` funksiyasi aynan Proxy ishlatadi: xususiyat **o'qilganda** "qaysi effekt unga bog'liq" deb yozib oladi (track), **o'zgartirilganda** o'sha effektlarni qayta ishga tushiradi (trigger). Mana minimal versiyasi:

```
let joriyEffekt = null;
const bogliqliklar = new Map(); // kalit -> effektlar to'plami

function effect(fn) {
  joriyEffekt = fn;
  fn(); // birinchi marta ishga tushirib, bog'liqliklarni
  yig'amiz
  joriyEffekt = null;
}

function reactive(obyekt) {
  return new Proxy(obyekt, {
    get(target, kalit) {
      if (joriyEffekt) {
        if (!bogliqliklar.has(kalit)) bogliqliklar.set(kalit, new
Set());
        bogliqliklar.get(kalit).add(joriyEffekt); // track
      }
      return target[kalit];
    },
    set(target, kalit, qiymat) {
      target[kalit] = qiymat;
      bogliqliklar.get(kalit)?.forEach(fn => fn()); // trigger
      return true;
    },
  });
}

// Ishlatish:
const holat = reactive({ son: 0 });

effect(() => {
  console.log("Son:", holat.son); // holat.son o'qiladi -> bog'lanadi
});
// "Son: 0"
```

```
holat.son = 5; // o'zgaradi -> effekt avtomatik qayta ishlaydi -> "Son: 5"
holat.son = 10; // "Son: 10"
```

Why bu sen uchun muhim: Sen Vue 3 ishlatasan. `ref` / `reactive` "sehrli" tuyuladi — lekin ostida aynan shu Proxy mexanizmi: o'qishni kuzatadi, o'zgarishda qayta render qiladi. Buni tushunsang, reaktivlikning chegaralarini ham bilasan (masalan, nega yangi xususiyat qo'shilganda ba'zan reaktivlik ishlamaydi — Proxy mavjud kalitlarni kuzatadi).

26-modul masalalari (20 ta)

1. Proxy yarating — `get` trap bilan o'qishni log qiling.
2. `set` trap bilan yozishni log qiling (`true` qaytaring).
3. `get` trap: mavjud bo'lmagan kalit uchun default qiymat qaytaring.
4. `set` trap: `yosh` manfiy bo'lsa xato tashlasin.
5. `has` trap bilan `in` operatorini ushlang.
6. `deleteProperty` trap bilan o'chirishni ushlang.
7. `Reflect.get` bilan standart o'qishni bajaring.
8. `Reflect.set` bilan standart yozishni bajaring.
9. Proxy + Reflect'ni birga ishlatib (toza pattern).
10. Logging proxy: har get va set'ni log qiluvchi.
11. Read-only proxy: `set` ni bloklab, ogohlantirish bersin.
12. Manfiy indeks proxy: `arr[-1]` oxirgi elementni qaytarsin.
13. Default qiymat proxy: har qanday yo'q kalit uchun `0` qaytarsin.
14. Validatsiya proxy: faqat `string` qiymat qabul qilsin.
15. `apply` trap bilan funksiya chaqiruvini ushlang.
16. Minimal `reactive()` yozing (Proxy bilan).
17. `effect()` funksiyasi yozing (track uchun).
18. `reactive` + `effect` bilan avtomatik qayta hisoblashni ko'rsating.
19. Kuzatuvchi (observer) proxy: o'zgarishda callback chaqirsin.
20. **Murakkab:** mini reaktiv tizim — `reactive` obyekt + `effect`; bir nechta xususiyat va effekt; biri o'zgarganda faqat unga bog'liq effekt qayta ishlasin (Vue 3 ruhida).



```
// 1, 2
const p = new Proxy({}, {
  get(t, k) { console.log("get:", k); return t[k]; },
  set(t, k, v) { console.log("set:", k, v); t[k] = v; return true; },
});
p.x = 5; // "set: x 5"
p.x;     // "get: x"

// 3
const p3 = new Proxy({}, {
  get(t, k) { return k in t ? t[k] : "yo'q"; },
});
console.log(p3.bor); // "yo'q"

// 4
const p4 = new Proxy({}, {
  set(t, k, v) {
    if (k === "yosh" && v < 0) throw new Error("Manfiy yosh");
    t[k] = v; return true;
  },
});
p4.yosh = 25; // ok

// 5
const p5 = new Proxy({ a: 1 }, {
  has(t, k) { console.log("has:", k); return k in t; },
});
console.log("a" in p5); // "has: a", true

// 6
const p6 = new Proxy({ a: 1 }, {
  deleteProperty(t, k) { console.log("delete:", k); delete t[k]; return true; },
});
delete p6.a; // "delete: a"

// 7, 8, 9
const p9 = new Proxy({ ism: "Oqil" }, {
  get(t, k, r) { return Reflect.get(t, k, r); },
  set(t, k, v, r) { return Reflect.set(t, k, v, r); },
});
console.log(p9.izm); // "Oqil"

// 10
const log = obj => new Proxy(obj, {
  get(t, k) { console.log(`GET ${k}`); return Reflect.get(t, k); },
  set(t, k, v) { console.log(`SET ${k}=${v}`); return Reflect.set(t, k, v); },
});

// 11
const readonly = obj => new Proxy(obj, {
  set() { console.warn("O'zgartirib bo'lmaydi"); return true; },
});
const ro = readonly({ x: 1 });
ro.x = 99; // ogohlantirish; ro.x hali 1
```

```

// 12
const manfiyIndeks = arr => new Proxy(arr, {
  get(t, k) {
    const i = Number(k);
    if (i < 0) return t[t.length + i];
    return t[k];
  },
});
const a12 = manfiyIndeks([1, 2, 3]);
console.log(a12[-1]); // 3

// 13
const p13 = new Proxy({}, { get: (t, k) => k in t ? t[k] : 0 });
console.log(p13.istalgan); // 0

// 14
const p14 = new Proxy({}, {
  set(t, k, v) {
    if (typeof v !== "string") throw new TypeError("Faqat string");
    t[k] = v; return true;
  },
});
p14.ism = "Oqil"; // ok

// 15
function f15() { return 42; }
const fp = new Proxy(f15, {
  apply(target, thisArg, args) { console.log("chaqirildi"); return
target(...args); },
});
console.log(fp()); // "chaqirildi", 42

// 16-20 - Reaktiv tizim
let joriyEffekt = null;
const bogliqliklar = new Map();

function effect(fn) { joriyEffekt = fn; fn(); joriyEffekt = null; }

function reactive(objekt) {
  return new Proxy(objekt, {
    get(t, k) {
      if (joriyEffekt) {
        if (!bogliqliklar.has(k)) bogliqliklar.set(k, new Set());
        bogliqliklar.get(k).add(joriyEffekt);
      }
      return t[k];
    },
    set(t, k, v) {
      t[k] = v;
      bogliqliklar.get(k)?.forEach(fn => fn());
      return true;
    },
  });
}

// 20 - namuna
const holat = reactive({ son: 0, ism: "Oqil" });
effect(() => console.log("Son:", holat.son)); // "Son: 0"
effect(() => console.log("Ism:", holat.ism)); // "Ism: Oqil"

```

```
holat.son = 5; // faqat "Son: 5" (ism effekti tegmaydi)
holat.ism = "Ali"; // faqat "Ism: Ali"
```

27-MODUL: Performance va Memory

Kodni tez va xotira-samarali qilish. Bular — frontend'da har kuni ishlatiladigan amaliy texnikalar.

Debounce — faollik tugaguncha kutish

Foydalanuvchi yozishni to'xtatgandan keyingina amal bajarish (qidiruv, resize):

```
function debounce(fn, kechikish) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), kechikish);
  };
}

// Ishlatish - qidiruv input'i:
const qidir = debounce((matn) => {
  console.log("Qidirilmoqda:", matn);
}, 500);

// Har bosishda qidir() chaqiriladi, lekin so'rov FAQAT 500ms jim
turgandan keyin
```

Why debounce: Foydalanuvchi "salom" deb yozsa, 5 ta harf = 5 ta API so'rovi (isrof).
Debounce bilan u yozishni tugatib, 500ms jim turgandagina **bitta** so'rov ketadi.
Qidiruv maydonlari, oyna o'lchami o'zgarishi, autosave uchun.

Throttle — chastotani cheklash

Amalni har N millisekundda **ko'pi bilan bir marta** bajarish (scroll, mousemove):

```
function throttle(fn, limit) {
  let kutmoqda = false;
  return (...args) => {
    if (!kutmoqda) {
      fn(...args);
      kutmoqda = true;
      setTimeout(() => (kutmoqda = false), limit);
    }
  };
}
```

```

    }
  };
}

// Scroll – soniyada ko'pi bilan ~10 marta:
const scrollHandler = throttle(() => console.log("Scroll!"), 100);

```

Why debounce vs throttle: **Debounce** — "tinchlangucha kut" (oxirgi holat muhim: qidiruv). **Throttle** — "muntazam, lekin cheklangan" (jarayon davomida kerak: scroll pozitsiyasi, o'yin). Ikkalasi ham ortiqcha ishlashni kamaytiradi.

Memoization — natijani keshlash

Bir xil kirish uchun qayta hisoblamaslik (17-moduldagi closure'ni esla):

```

function memoize(fn) {
  const kesh = new Map();
  return (...args) => {
    const kalit = JSON.stringify(args);
    if (kesh.has(kalit)) return kesh.get(kalit);
    const natija = fn(...args);
    kesh.set(kalit, natija);
    return natija;
  };
}

const ogirHisob = memoize((n) => {
  console.log("Hisoblanmoqda...");
  return n * n;
});

ogirHisob(5); // "Hisoblanmoqda...", 25
ogirHisob(5); // 25 (keshdan, qayta hisoblanmaydi)

```

Algoritm tezligi (Big-O intuitsiyasi)

```

// O(n²) – SEKIN: ichma-ich sikl
function dublikatlarSekin(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++)
      if (arr[i] === arr[j]) return true;
  return false;
}

// O(n) – TEZ: Set lookup (O(1))
function dublikatlarTez(arr) {
  const korilgan = new Set();
  for (const x of arr) {
    if (korilgan.has(x)) return true; // Set.has -> O(1)
  }
}

```

```
    korilgan.add(x);
  }
  return false;
}
```

Why: Massiv `includes / indexOf` — $O(n)$ (har safar boshidan qidiradi).

`Set.has / Map.get` /obyekt kaliti — $O(1)$ (darhol). Katta ma'lumotda ichma-ich sikl ($O(n^2)$) o'rniga `Set / Map` ishlatib $O(n)$ ga tushir. 1000 element: $n^2 = 1,000,000$ amal, $n = 1,000$ amal.

Memory leaks (xotira sizishi)

Eng ko'p uchraydigan sabablar:

```
// ❌ Unutilgan interval — abadiy ishlaydi:
setInterval(() => doSomething(), 1000);
// ✅ Tugatgach to'xtat:
const id = setInterval(() => doSomething(), 1000);
// keyin: clearInterval(id);

// ❌ Unutilgan event listener:
element.addEventListener("click", handler);
// ✅ Kerak bo'lmaganda olib tashla:
element.removeEventListener("click", handler);
```

WeakMap / WeakSet — xotira-xavfsiz havolalar

`WeakMap` kalitlari **obyekt** bo'ladi va ularga boshqa havola qolmasa, **avtomatik tozalanadi** (garbage collected). Bu — memory leak'siz keshlash uchun:

```
const kesh = new WeakMap();

let user = { ism: "Oqil" };
kesh.set(user, "ma'lumot");
console.log(kesh.get(user)); // "ma'lumot"

user = null; // endi user'ga havola yo'q
// kesh ham avtomatik tozalanadi (Map bo'lganda — sizardi)
```

Why WeakMap: Oddiy `Map` kalitni "ushlab turadi" — obyekt kerak bo'lmasa ham xotirada qoladi (leak). `WeakMap` esa kalitni "zaif" ushlaydi — boshqa havola yo'qolsa, GC uni tozalaydi. DOM elementlariga metadata biriktirish kabi holatlarda foydali. (Kamchilik: iteratsiya/ `size` yo'q.)

27-modul masalalari (20 ta)

1. `debounce(fn, delay)` funksiyasini yozing.
2. Debounce'ni qidiruv input'iga qo'llang (konseptual/DOM).
3. `throttle(fn, limit)` funksiyasini yozing.
4. Throttle'ni scroll uchun ishlating (konseptual).
5. `memoize(fn)` yozing (Map keshi bilan).
6. Memoize bilan og'ir hisobni keshlang (ikkinchi chaqiriqda hisoblamasin).
7. Early return bilan funksiyani optimallashtiring.
8. $O(n^2)$ (nested sikl) va $O(n)$ (Set) bilan dublikat toping — taqqoslang.
9. `includes` ($O(n)$) vs `Set.has` ($O(1)$) ni ko'rsating.
10. `DocumentFragment` bilan DOM'ni batch qiling (konseptual).
11. Unutilgan `setInterval` muammosini ko'rsating va `clearInterval` bilan tuzating.
12. Unutilgan event listener'ni `removeEventListener` bilan tuzating.
13. `WeakMap` yaratib, ishlating.
14. `WeakMap` va `Map` farqini (GC) izohlang.
15. `WeakSet` bilan obyektlarni belgilang (brief).
16. `requestAnimationFrame` bilan oddiy animatsiya (konseptual).
17. Debounce'ga "darhol bir marta" (immediate) varianti qo'shing.
18. Massiv dedup'ni `Set` bilan $O(n)$ qiling.
19. Memoize'ni Fibonachchiga qo'llang (sekin rekursiya vs tez).
20. **Murakkab:** qidiruv komponenti logikasi — `debounce` (500ms) + `memoize` (kesh) birga; bir xil so'rov keshdan, yangi so'rov debounce bilan.



```
// 1
function debounce(fn, kechikish) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), kechikish);
  };
}

// 2 (DOM)
// const qidir = debounce(matn => console.log("API:", matn), 500);
// input.addEventListener("input", e => qidir(e.target.value));

// 3
function throttle(fn, limit) {
  let kutmoqda = false;
  return (...args) => {
    if (!kutmoqda) {
      fn(...args); kutmoqda = true;
      setTimeout(() => (kutmoqda = false), limit);
    }
  };
}

// 4 (DOM)
// window.addEventListener("scroll", throttle(() => console.log("scroll"),
// 100));

// 5
function memoize(fn) {
  const kesh = new Map();
  return (...args) => {
    const k = JSON.stringify(args);
    if (kesh.has(k)) return kesh.get(k);
    const r = fn(...args);
    kesh.set(k, r);
    return r;
  };
}

// 6
const ogir = memoize(n => { console.log("hisob"); return n * 2; });
ogir(5); // "hisob", 10
ogir(5); // 10 (keshdan)

// 7
function topId(users, id) {
  if (!users || users.length === 0) return null; // early return
  return users.find(u => u.id === id) ?? null;
}

// 8
function dubSekin(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++)
      if (arr[i] === arr[j]) return true;
  return false;
}
```

```

}
function dubTez(arr) {
  const s = new Set();
  for (const x of arr) { if (s.has(x)) return true; s.add(x); }
  return false;
}
console.log(dubTez([1, 2, 3, 2])); // true

// 9
const katta = [/* 10000 element */];
// katta.includes(x) -> O(n), har safar boshidan
const set9 = new Set(katta);
// set9.has(x) -> O(1), darhol

// 10 (DOM)
// const frag = document.createDocumentFragment();
// for (...) frag.appendChild(li);
// ul.appendChild(frag); // bitta DOM operatsiyasi

// 11
const id11 = setInterval(() => console.log("tik"), 1000);
clearInterval(id11); // tuzatildi (aks holda abadiy ishlaydi)

// 12 (DOM)
// const h = () => {};
// el.addEventListener("click", h);
// el.removeEventListener("click", h); // tozalandi

// 13
const wm = new WeakMap();
let obj13 = { id: 1 };
wm.set(obj13, "data");
console.log(wm.get(obj13)); // "data"

// 14
// Map: kalit obyektni ushlab turadi -> GC tozalay olmaydi (leak xavfi)
// WeakMap: kalitni zaif ushlaydi -> havola yo'qolsa GC tozalaydi

// 15
const ws = new WeakSet();
let o15 = {};
ws.add(o15);
console.log(ws.has(o15)); // true

// 16 (DOM)
// function animatsiya() { /* harakat */ requestAnimationFrame(animatsiya);
// }
// requestAnimationFrame(animatsiya);

// 17
function debounceImmediate(fn, kechikish) {
  let timer;
  return (...args) => {
    const darhol = !timer;
    clearTimeout(timer);
    timer = setTimeout(() => (timer = null), kechikish);
    if (darhol) fn(...args); // birinchisi darhol ishlaydi
  };
}

// 18

```

```

console.log([...new Set([1, 1, 2, 3, 3])]); // [1,2,3] - O(n)

// 19
const fibMemo = memoize(function f(n) {
  return n < 2 ? n : f(n - 1) + f(n - 2);
});
console.log(fibMemo(30)); // tez (kesh tufayli)

// 20 - qidiruv komponenti
function qidiruvKomponenti() {
  const kesh = new Map();
  const soxtaApi = matn => `${matn} natijalari`;

  const qidir = debounce((matn) => {
    if (kesh.has(matn)) {
      console.log("Keshdan:", kesh.get(matn));
      return;
    }
    const natija = soxtaApi(matn);
    kesh.set(matn, natija);
    console.log("API:", natija);
  }, 500);

  return qidir;
}
const search = qidiruvKomponenti();
// search("vue"); search("vue"); -> birinchisi API, keshdagisi keshdan

```

✓ 6-qism (1-bo'lim) yakuni

Expert darajaga yaqinlashding: - **Functional Programming** — pure functions, immutability, compose/pipe, currying, declarative - **Generators & Iterators** — `function* / yield`, custom iterable, lazy/cheksiz ketma-ketlik, Symbol - **Proxy & Reflect** — amallarni ushlash; va **Vue 3 reaktivligini** noldan yasash - **Performance & Memory** — debounce/throttle, memoize, Big-O intuitsiyasi, memory leaks, WeakMap

Eng muhim xulosalar: 1. **Immutability + pure functions** — bug'siz, test-oston kodning asosi (React/Vue state shu). 2. **Proxy** — Vue 3 `reactive()` ning yuragi. Endi "sehr" seyr emas. 3. **Debounce/throttle** — frontend'da har kuni; `Set / Map` bilan $O(n^2)$ ni $O(n)$ ga tushir.

Keyingi qadam (6-qism, 2-bo'lim — qo'llanmaning yakuni)

- **Design Patterns** — Singleton, Factory, Observer, Strategy, Module (real arxitektura)
- **Bundlers & Build Tools** — npm, Vite, tree-shaking, ESM vs CommonJS (konseptual)
- **TypeScript'ga ko'prik** — JS'dan keyingi tabiiy qadam; tiplar nega kerak

Maslahat: 26-modul #20 (reaktiv tizim)ni va 27-modul #20 (qidiruv komponenti)ni puxta o'rgan. Bu ikkisi — Vue/React kabi freymvorklar "ostida" ishlaydigan haqiqiy mexanizm; ularni tushunsang, freymvork "sehr" emas, mantiq bo'lib ko'rinadi.

 [README](#) · [← 5-qism \(2-bo'lim\)](#) · [Keyingi: 6-qism \(2-bo'lim\)](#) — [Yakun](#) →

JavaScript — 0 dan Expertgacha (O'zbek tilida)

 [README](#) · [← 6-qism \(1-bo'lim\)](#) · **Yakuniy qism**

6-QISM: EXPERT — 2-bo'lim (QO'LLANMA YAKUNI) Design patterns, build tools, TypeScript'ga o'tish. Bu — 30 modulli qo'llanmaning so'nggi bo'limi. Har bir moduldan keyin **20 ta masala** (yechimi bilan).

1-bo'limda til mexanikasini chuqurlashtirdik (FP, generators, Proxy, performance).
Endi — arxitektura va keyingi qadam.

28-MODUL: Design Patterns

Design pattern — tez-tez uchraydigan muammolarga **tayyor, sinangan yechim**. (Sen Laravel'da OOP/SOLID bilan ishlaysan — bu patternlarning ko'pi tanish bo'ladi, faqat JS sintaksisida.)

1. Module Pattern (inkapsulatsiya)

Private holatni yashirib, faqat kerakli interfeysni ochish (17-moduldagi closure'ni esla):

```
const Hisoblagich = (() => {  
  let son = 0; // PRIVATE - tashqaridan ko'rinmaydi  
  
  return {  
    oshir() { return ++son; },  
    kamaytir() { return --son; },  
    qiymat() { return son; },  
  };  
})();  
  
Hisoblagich.oshir();  
Hisoblagich.oshir();  
console.log(Hisoblagich.qiymat()); // 2  
console.log(Hisoblagich.son);      // undefined - yashirin
```

Zamonaviy JS'da bu **ES modullar** bilan tabiiy hal bo'ladi (21-modul): faylda export qilinmagan narsa avtomatik private.

2. Singleton (yagona nusxa)

Butun ilovada **bitta** nusxasi bo'lishi kerak bo'lgan obyekt (konfiguratsiya, store, DB ulanishi):

```
class Store {
  static #instance = null;
  #data = {};

  static getInstance() {
    if (!Store.#instance) {
      Store.#instance = new Store();
    }
    return Store.#instance;
  }

  set(kalit, qiymat) { this.#data[kalit] = qiymat; }
  get(kalit) { return this.#data[kalit]; }
}

const a = Store.getInstance();
const b = Store.getInstance();
console.log(a === b); // true – ikkalasi ham AYNAN bir xil nusxa
```

JS'da eng oddiy singleton — **modul orqali bitta nusxa eksport qilish**:

```
// 📄 store.js
class Store { /* ... */ }
export default new Store(); // bitta nusxa – har import bir xil
obyektni oladi
```

Why singleton: Global holat (foydalanuvchi sessiyasi, sozlamalar) bitta manbadan kelishi kerak. Vuex/Pinia store, konfiguratsiya — singleton. Lekin ehtiyot bo'! haddan ortiq global holat — test va debug'ni qiyinlashtiradi.

3. Factory (yaratuvchi)

Obyekt yaratish mantig'ini yashirib, "buyurtma" bo'yicha obyekt qaytaradi:

```
function userYarat(tur, ism) {
  const asos = { ism, yaratilgan: Date.now() };

  switch (tur) {
    case "admin":
      return { ...asos, rol: "admin", huquqlar: ["o'qish", "yozish", "o'chirish"] };
    case "muharrir":
      return { ...asos, rol: "muharrir", huquqlar: ["o'qish", "yozish"] }
  }
}
```

```

};
default:
  return { ...asos, rol: "mehmon", huquqlar: ["o'qish"] };
}
}

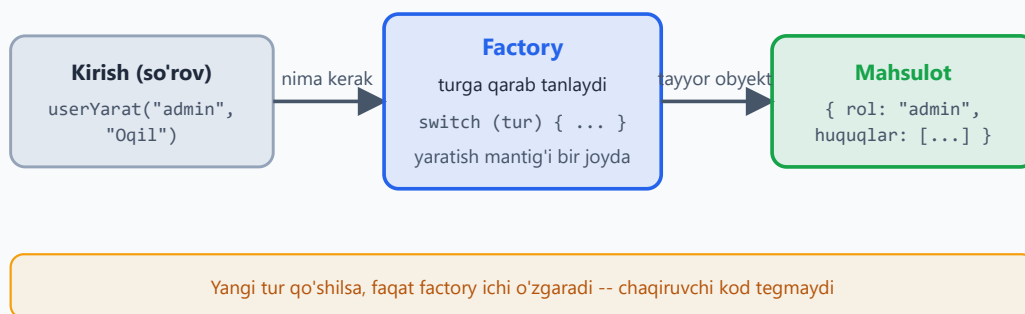
const admin = userYarat("admin", "Oqil");
const mehmon = userYarat("guest", "Ali");
console.log(admin.huquqlar); // ["o'qish", "yozish", "o'chirish"]

```

Why factory: Chaqiruvchi "qanday yaratilishini" bilishi shart emas — faqat "nima kerakligini" aytadi. Yaratish mantig'i bir joyda. (Laravel Factory bilan bir g'oya.)

Factory: "buyurtma" bo'yicha obyekt qaytaradi

Chaqiruvchi faqat "nima kerakligini" aytadi, "qanday yaratilishi" yashirin



4. Observer / Pub-Sub (kuzatuvchi)

Bir obyekt o'zgarganda, unga "obuna bo'lgan"larga xabar berish. Hodisa tizimlari, reaktivlik shu pattern (26-moduldagi Vue 3 reaktivligi — aslida Observer!):

```

class EventEmitter {
  #tinglovchilar = {};

  on(hodisa, fn) {
    (this.#tinglovchilar[hodisa] ??= []).push(fn); // obuna
    return this; // zanjir uchun
  }

  emit(hodisa, data) {
    this.#tinglovchilar[hodisa]?.forEach(fn => fn(data)); // xabar
    berish
  }

  off(hodisa, fn) {
    if (this.#tinglovchilar[hodisa]) {
      this.#tinglovchilar[hodisa] =

```

```

this.#tinglovchilar[hodisa].filter(f => f !== fn);
    }
  }
}

const bus = new EventEmitter();
bus.on("login", (user) => console.log(`${user} kirdi`));
bus.on("login", (user) => console.log(`Log: ${user}`));

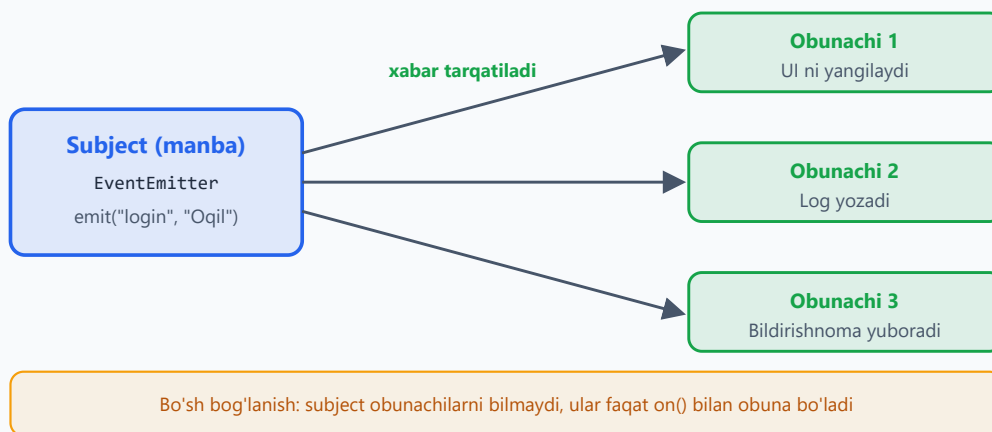
bus.emit("login", "Oqil");
// "Oqil kirdi"
// "Log: Oqil"

```

Why observer: Komponentlar bir-birini bilmasdan "gaplashadi" (bo'sh bog'lanish — loose coupling). Tugma bosildi → bir nechta joy reaksiya qiladi. Vue event bus, DOM event'lari, WebSocket — hammasi shu.

Observer / Pub-Sub: bitta xabar, ko'p obunachi

Subject o'zgaradi -> emit() -> obuna bo'lganlarning hammasi xabar oladi



5. Strategy (almashinadigan algoritmlar)

Bir xil ish uchun har xil "usul"larni almashtirib ishlatish (SOLID'ning Open/Closed prinsipi):

```

const tolovUsullari = {
  naqd: (summa) => summa, // komissiyasiz
  karta: (summa) => summa * 1.02, // 2% komissiya
  click: (summa) => summa * 1.01, // 1% komissiya
  payme: (summa) => summa * 1.015, // 1.5%
};

function tolovHisobla(usul, summa) {
  const strategiya = tolovUsullari[usul];
  if (!strategiya) throw new Error("Noma'lum to'lov usuli");
  return strategiya(summa);
}

```

```
}  
  
console.log(tolovHisobla("karta", 100000)); // 102000  
console.log(tolovHisobla("naqd", 100000)); // 100000
```

Why strategy: Yangi usul qo'shish uchun **mavjud kodni o'zgartirmaysan** — shunchaki yangi strategiya qo'shasan (Open/Closed). Katta `if/else` o'rniga toza, kengaytiriladigan struktura. To'lov, saralash, validatsiya, eksport formatlari uchun ideal.

Bonus: Decorator (o'rab, xatti-harakat qo'shish)

Funksiyani o'zgartirmasdan, uni o'rab qo'shimcha imkoniyat berish:

```
function logBilan(fn) {  
  return (...args) => {  
    console.log(`Chaqirildi: ${args}`);  
    const natija = fn(...args);  
    console.log(`Natija: ${natija}`);  
    return natija;  
  };  
}  
  
const qoshish = (a, b) => a + b;  
const qoshishLog = logBilan(qoshish);  
qoshishLog(2, 3); // log + natija
```

28-modul masalalari (20 ta)

1. Module pattern (IIFE) bilan `private` `son` va `o'shir / qiymat` yozing.
2. ES modul bilan inkapsulatsiya qiling (`export` qilinmagan = `private`) — konseptual.
3. Singleton class yozing (`getInstance`).
4. Ikki `getInstance()` bir xil nusxa ekanini (`===`) ko'rsating.
5. Modul orqali singleton yozing (bitta nusxa `export`).
6. Factory funksiya: turga qarab user obykti yarating.
7. Factory: shakl yaratuvchi (`doira / tortburchak`).
8. Factory class (`static create`).
9. `EventEmitter` yozing: `on` va `emit`.
10. Bir hodisaga bir nechta tinglovchi qo'shing.

11. `off` bilan obunani bekor qiling.
12. Pub-Sub: `emit` da data uzating, tinglovchi olsin.
13. Strategy: to'lov usullari (komissiya bilan).
14. Strategy: saralash strategiyalari (o'sish/kamayish).
15. Strategy: validatsiya strategiyalari (email/telefon).
16. Decorator: funktsiyani o'rab log qo'shing.
17. Decorator: funktsiya vaqtini o'lchovchi (timing) yozing.
18. Singleton + Observer birga: global store (`subscribe` / `set`).
19. Factory + Strategy birga ishlating.
20. **Murakkab:** mini state store — Singleton + Observer; `getState` , `setState` , `subscribe` ; holat o'zgarganda obunachilarga xabar bersin (Vuex/Pinia ruhida).



```
// 1
const Hisoblagich = (() => {
  let son = 0;
  return { oshir: () => ++son, qiymat: () => son };
})();
Hisoblagich.oshir();
console.log(Hisoblagich.qiymat()); // 1

// 2 - ES modul (konseptual)
// 📄 hisoblagich.js
//   let son = 0; // private (export qilinmagan)
//   export const oshir = () => ++son;
//   export const qiymat = () => son;

// 3, 4
class Store {
  static #instance = null;
  #data = {};
  static getInstance() {
    if (!Store.#instance) Store.#instance = new Store();
    return Store.#instance;
  }
  set(k, v) { this.#data[k] = v; }
  get(k) { return this.#data[k]; }
}
console.log(Store.getInstance() === Store.getInstance()); // true

// 5 - modul singleton (konseptual)
// 📄 store.js -> export default new Store();

// 6
function userYarat(tur, ism) {
  const asos = { ism };
  if (tur === "admin") return { ...asos, huquqlar: ["all"] };
  return { ...asos, huquqlar: ["read"] };
}
console.log(userYarat("admin", "Oqil"));

// 7
function shaklYarat(tur, ...args) {
  if (tur === "doira") return { tur, r: args[0], yuza: () => Math.PI *
args[0] ** 2 };
  if (tur === "tortburchak") return { tur, yuza: () => args[0] * args[1] };
}
console.log(shaklYarat("doira", 5).yuza().toFixed(2)); // 78.54

// 8
class UserFactory {
  static create(ism) { return { ism, id: Date.now() }; }
}
console.log(UserFactory.create("Ali").ism); // "Ali"

// 9, 10, 11, 12
class EventEmitter {
  #t = {};
  on(h, fn) { (this.#t[h] ??= []).push(fn); return this; }
  emit(h, data) { this.#t[h]?.forEach(fn => fn(data)); }
}
```

```

    off(h, fn) { if (this.#t[h]) this.#t[h] = this.#t[h].filter(f => f !==
fn); }
}
const bus = new EventEmitter();
const log = u => console.log("Log:", u);
bus.on("login", u => console.log(`${u} kirdi`)).on("login", log);
bus.emit("login", "Oqil"); // ikkala tinglovchi
bus.off("login", log);
bus.emit("login", "Ali"); // faqat birinchisi

// 13
const tolov = {
  naqd: s => s,
  karta: s => s * 1.02,
};
console.log(tolov.karta(100000)); // 102000

// 14
const saralash = {
  osish: arr => [...arr].sort((a, b) => a - b),
  kamayish: arr => [...arr].sort((a, b) => b - a),
};
console.log(saralash.osish([3, 1, 2])); // [1,2,3]

// 15
const validatsiya = {
  email: v => /^[^\\s@]+@[^\\s@]+\\.[^\\s@]+$/ .test(v),
  telefon: v => /^\\+998\\d{9}$/ .test(v),
};
console.log(validatsiya.email("a@b.uz")); // true

// 16
function logBilan(fn) {
  return (...a) => { console.log("chaqirildi:", a); return fn(...a); };
}
logBilan((a, b) => a + b)(2, 3); // log, 5

// 17
function timing(fn) {
  return (...a) => {
    const start = performance.now();
    const r = fn(...a);
    console.log(`Vaqt: ${performance.now() - start}.toFixed(2)}ms`);
    return r;
  };
}

// 18, 20 - state store (Singleton + Observer)
class StateStore {
  static #instance = null;
  #state = {};
  #obunachilar = [];

  static getInstance() {
    if (!StateStore.#instance) StateStore.#instance = new StateStore();
    return StateStore.#instance;
  }

  getState() { return { ...this.#state }; }

  setState(yangi) {

```

```

    this.#state = { ...this.#state, ...yangi };
    this.#obunachilar.forEach(fn => fn(this.#state)); // xabar berish
  }

  subscribe(fn) {
    this.#obunachilar.push(fn);
    return () => { // unsubscribe funksiyasi qaytaramiz
      this.#obunachilar = this.#obunachilar.filter(f => f !== fn);
    };
  }
}

const store = StateStore.getInstance();
const unsub = store.subscribe(s => console.log("O'zgardi:", s));
store.setState({ user: "Oqil" }); // "O'zgardi: {user: 'Oqil'}"
store.setState({ yosh: 25 }); // "O'zgardi: {user: 'Oqil', yosh: 25}"
unsub(); // obunani bekor qilamiz

// 19 - Factory + Strategy
const tolovStrategiyalari = { naqd: s => s, karta: s => s * 1.02 };
function tolovYarat(usul) {
  return { usul, hisobla: summa => tolovStrategiyalari[usul](summa) };
}
console.log(tolovYarat("karta").hisobla(1000)); // 1020

```

29-MODUL: Bundlers va build tools

Eslatma: Bu modul ko'proq **tushunchalar** haqida — kodni ishga tushirish/yetkazish jarayoni. Masalalar amaliy savol-javob ko'rinishida.

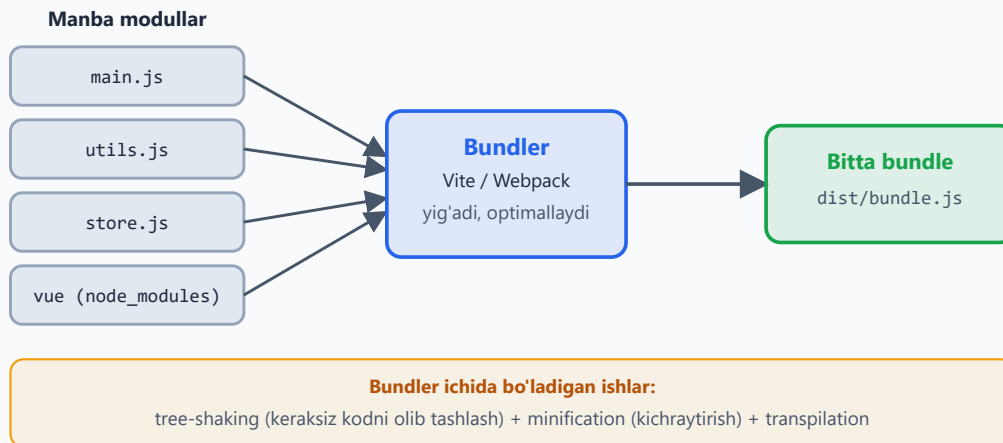
Nega bundler kerak?

Sen 30 ta `.js` faylga bo'lingan loyiha yozding. Brauzerga buni qanday yetkazasan? - Har faylni alohida yuklasa — sekin (ko'p so'rov). - Eski brauzerlar ba'zi zamonaviy sintaksisni tushunmaydi. - Ishlatilmagan kod ham yuklanib, hajm kattalashadi.

Bundler (Vite, Webpack) — barcha modullarni **bitta optimallashtirilgan faylga** yig'adi, eski brauzerlarga moslaydi, hajmni kichraytiradi.

Bundling: ko'p faylni bitta faylga yig'ish

Brauzerga 30 ta so'rov o'rniga 1 ta -- tezroq yuklanish



npm va package.json

npm — JS paketlar menejeri. package.json — loyiha "pasporti":

```
{
  "name": "educore-frontend",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "vue": "^3.4.0"
  },
  "devDependencies": {
    "vite": "^5.0.0"
  }
}
```

- **dependencies** — ishlab turishi uchun kerak (vue, axios).
- **devDependencies** — faqat ishlab chiqishda kerak (vite, eslint — build'dan keyin kerak emas).
- **scripts** — npm run dev, npm run build buyruqlari.

```
npm init -y          # package.json yaratadi
npm install vue       # dependencies'ga qo'shadi
npm install -D vite   # devDependencies'ga qo'shadi
npm run dev           # scripts'dagi "dev" ni ishga tushiradi
```

Semver (versiyalash)

`^3.4.0` kabi belgilar versiya yangilanishini boshqaradi:

```
3.4.0 -> MAJOR.MINOR.PATCH
^3.4.0 -> 3.x.x gacha (3.4.0 dan 4.0.0 gacha, 4 kirmaydi)
~3.4.0 -> 3.4.x gacha (faqat patch yangilanishlar)
3.4.0 -> aniq shu versiya
```

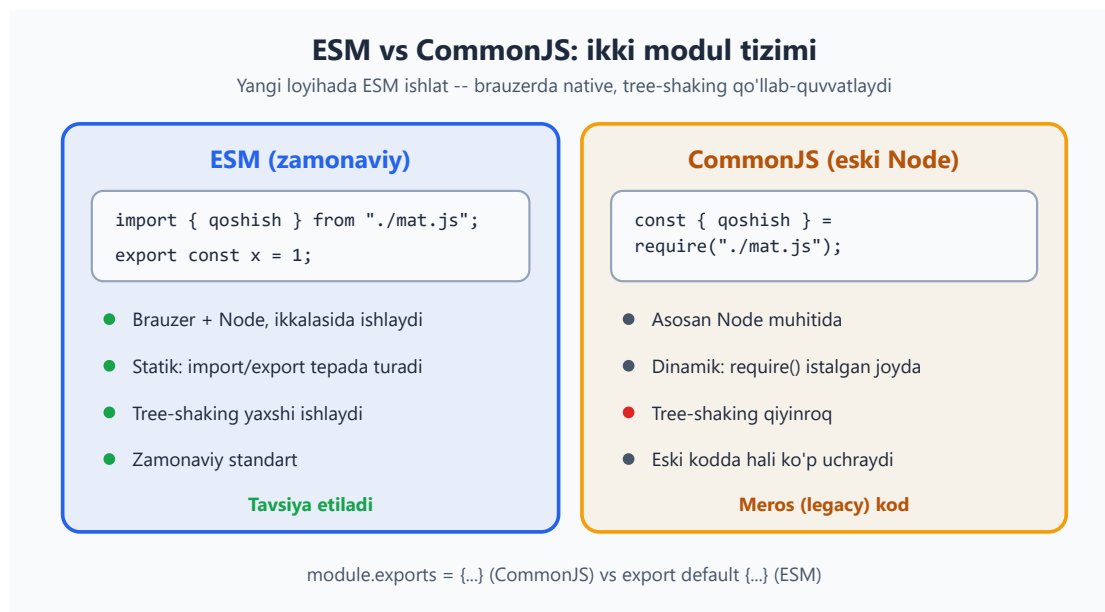
ESM vs CommonJS

JS'da ikki modul tizimi bor:

```
// ESM (zamonaviy, brauzer + Node) - 21-modulda ko'rdik:
import { qoshish } from "./mat.js";
export const x = 1;

// CommonJS (eski Node):
const { qoshish } = require("./mat.js");
module.exports = { x: 1 };
```

Why ESM: Zamonaviy standart — brauzerda native ishlaydi, **tree-shaking**ni (pastda) qo'llab-quvvatlaydi. CommonJS — eski Node kodida hali ko'p. Yangi loyihada ESM ishlat.



Build jarayonidagi tushunchalar

- **Tree-shaking** — ishlatilmagan kodni (export qilingan, lekin import qilinmagan) build'dan **olib tashlash**. Hajmni kichraytiradi. (ESM'ning statik tabiati tufayli ishlaydi.)
- **Minification** — bo'shliq, izoh, uzun nomlarni olib tashlab faylni kichraytirish (`hisoblagich` → `a`).
- **Transpilation** (Babel) — zamonaviy sintaksisni eski brauzerlar tushunadigan kodga aylantirish.
- **HMR** (Hot Module Replacement) — ishlab chiqishda kodni o'zgartirsang, sahifa to'liq yangilanmasdan faqat o'zgargan qism almashinadi (tez).
- **Source map** — minify qilingan kodni asl kod bilan bog'laydi (debug uchun).
- `dist/` — `npm run build` dan keyingi tayyor (production) fayllar.

Vite (sening stackingda)

Vue/Nuxt **Vite** ustida ishlaydi. Vite: - Ishlab chiqishda — **esbuild** bilan juda tez dev server + HMR. - Production'da — **Rollup** bilan optimallashtirilgan bundle (tree-shaking, minify).

Why Vite: Eski Webpack'dan tez (ayniqsa dev server). Vue/Nuxt/SvelteKit — barchasi Vite'ni standart qilgan. Sen Nuxt ishlatsang, Vite "ostida" ishlaydi — to'g'ridan-to'g'ri sozlash kamdan-kam kerak.

29-modul masalalari (20 ta)

Bular amaliy savollar — ba'zilarini terminalda sinab ko'rsang bo'ladi.

1. `npm init -y` nima qiladi?
2. `dependencies` va `devDependencies` farqini ayting.
3. `vue` ni `dependencies`'ga qo'shuvchi buyruqni yozing.
4. `vite` ni `devDependencies`'ga qo'shuvchi buyruqni yozing.
5. `package.json` dagi `scripts` dan `dev` ni ishga tushiruvchi buyruq.
6. `^3.4.0` qaysi versiyalarni qabul qiladi?
7. `~3.4.0` qaysi versiyalarni qabul qiladi?
8. `node_modules` nima va nega `.gitignore` ga qo'shiladi?
9. `package-lock.json` nima uchun kerak?
10. Bir modulni ESM va CommonJS'da import qilib yozing.

11. `module.exports` va `export default` farqini yozing.
12. Vite ishlab chiqishda va production'da nima qiladi?
13. Tree-shaking nima va nega faqat ESM'da yaxshi ishlaydi?
14. Minification nima qiladi?
15. Transpilation (Babel) nega kerak?
16. HMR nima va qanday foyda beradi?
17. `dist/` papkasi nima?
18. Source map nima uchun kerak?
19. Nega bundler ko'p faylni bitta qiladi (sabab)?
20. **Murakkab:** kichik Vue loyihasi uchun `package.json` yozing — `scripts` (`dev`, `build`), `dependencies` (`vue`), `devDependencies` (`vite`).



1. So'rovsiz (default qiymatlar bilan) package.json fayl yaratadi. 
2. dependencies – ilova ISHLASHI uchun kerak (vue, axios), production'da ham.
devDependencies – faqat ISHLAB CHIQUISHDA kerak (vite, eslint, jest); tayyor mahsulotga kirmaydi.
3. npm install vue (yoki: npm i vue)
4. npm install -D vite (yoki: npm install --save-dev vite)
5. npm run dev
6. ^3.4.0 -> >=3.4.0 va <4.0.0 (3.x.x dagi eng yangi; major o'zgarmaydi)
7. ~3.4.0 -> >=3.4.0 va <3.5.0 (faqat patch yangilanishlar)
8. node_modules – o'rnatilgan paketlar papkasi (juda katta).
.gitignore'ga qo'shiladi, chunki package.json'dan qayta tiklash mumkin (npm install) – git'da saqlash shart emas.
9. package-lock.json – har bir paketning ANIQ versiyasini qayd etadi.
Shunda hamma jamoa a'zosi va server bir xil versiyalarni oladi (reproducible build).
10. // ESM:
import { qoshish } from "../mat.js";
// CommonJS:
const { qoshish } = require("../mat.js");
11. // CommonJS:
module.exports = funksiya;
// ESM:
export default funksiya;
12. Dev: esbuild bilan tez dev server + HMR.
Production (vite build): Rollup bilan optimallashtirilgan bundle (tree-shaking, minification).
13. Tree-shaking – import qilinmagan export'larni build'dan olib tashlaydi.
ESM statik (import/export tepada, o'zgarmas) bo'lgani uchun bundler nima ishlatilmaganini aniq biladi. CommonJS dinamik -> qiyinroq.
14. Bo'shliq, izoh, uzun o'zgaruvchi nomlarini olib tashlab faylni kichraytiradi (yuklanish tezlashadi).
15. Zamonaviy JS'ni (optional chaining, ??, class fields) eski brauzerlar tushunadigan eskiroq sintaksisga aylantiradi.
16. HMR (Hot Module Replacement) – kod o'zgarganda butun sahifani yangilamasdan faqat o'zgargan modulni almashtiradi. Holat saqlanadi, ishlab chiqish tezlashadi.
17. dist/ – `npm run build`dan keyingi tayyor (production) fayllar (bundled, minified). Serverga shu yuklanadi.
18. Minify qilingan kodni asl kod bilan bog'laydi – brauzerda debug

qilganda asl qator/o'zgaruvchi nomlarini ko'rasan.

19. Har faylni alohida yuklash ko'p HTTP so'rov = sekin. Bitta (yoki bir nechta) bundle – kam so'rov, tezroq yuklanish.

```
20.
{
  "name": "educore-frontend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "vue": "^3.4.0"
  },
  "devDependencies": {
    "vite": "^5.0.0",
    "@vitejs/plugin-vue": "^5.0.0"
  }
}
```

30-MODUL: TypeScript'ga ko'prik

Eslatma: TypeScript (TS) brauzerda to'g'ridan-to'g'ri ishlamaydi — u JS'ga **kompilatsiya** qilinadi. Mashqlarni typescriptrang.org/play (TS Playground)da sinab ko'r.

TypeScript nima va nega?

TS — JS'ning **tiplar qo'shilgan** kengaytmasi (superset). Sen yozgan har qanday JS — to'g'ri TS. TS ustiga **tip tizimi** qo'shadi.

```
// JS - xato faqat ishga tushganda bilinadi:
function qoshish(a, b) { return a + b; }
qoshish("5", 3); // "53" - kutilmagan, lekin xato bermaydi

// TS - xato YOZAYOTGANDA bilinadi:
function qoshish(a: number, b: number): number { return a + b; }
qoshish("5", 3); // ❌ Kompilyator xatosi: "5" string, number kerak
```

Why TS: (1) Xatolar **kompilatsiya vaqtida** ushlanadi, foydalanuvchida emas. (2) Avtomatik to'ldirish (autocomplete) kuchli. (3) Kod o'zini hujjatlashtiradi (tip = hujjat). (4) Katta loyihada yo'qolib qolmaysan. Vue 3, Nuxt, Angular — hammasi TS'ni qo'llab-quvvatlaydi/tavsiya qiladi.

Asosiy tip annotatsiyalari

```
let ism: string = "Oqil";
let yosh: number = 25;
let faol: boolean = true;
let sonlar: number[] = [1, 2, 3];
let aralash: (string | number)[] = ["a", 1, "b"];

// Funksiya:
function salom(ism: string): string {
  return `Salom, ${ism}`;
}

// Hech narsa qaytarmasa - void:
function log(xabar: string): void {
  console.log(xabar);
}
```

Obyekt va interface

`interface` — obyekt "shaklini" ta'riflaydi:

```
interface User {
  ism: string;
  yosh: number;
  email?: string; // ? - ixtiyoriy (bo'lmasligi mumkin)
}

const user: User = { ism: "Oqil", yosh: 25 };
// email bermadik - ixtiyoriy bo'lgani uchun OK

function userKorsat(u: User): string {
  return `${u.izm}, ${u.yosh} yosh`;
}
```

Union va literal tiplar

```
let id: string | number; // string YOKI number
id = "abc"; // OK
id = 123; // OK

// Literal union - faqat aniq qiymatlar:
type Status = "faol" | "nofaol" | "kutilmoqda";
let holat: Status = "faol"; // OK
// let holat: Status = "boshqa"; // ❌ xato
```

type alias

```
type ID = string | number;
type Koordinata = { x: number; y: number };

const nuqta: Koordinata = { x: 10, y: 20 };
```

interface vs type: Obyekt shakllari uchun ko'pincha **interface** (kengaytiriladi), boshqa holatlarda (union, alias) **type**. Boshlang'ich uchun: obyekt → **interface**, qolgani → **type**.

Generics (umumiy tiplar)

Turli tip bilan ishlaydigan, lekin tip xavfsizligini saqlaydigan funksiya/struktura:

```
function birinchi<T>(arr: T[]): T {
  return arr[0];
}

const s = birinchi(["a", "b"]); // T = string, s: string
const n = birinchi([1, 2, 3]); // T = number, n: number
```

Why generics: **any** ishlatsang tip xavfsizligini yo'qotasan. Generic — "qanday tur kelsa, o'shani saqla". Massiv, Promise, React/Vue komponentlari — generics ustiga qurilgan (`Array<string>`, `Promise<User>`).

TS qanday ishlaydi

```
sen.ts --(tsc kompilyator / Vite)--> sen.js
```

TS kompilyatori (`tsc`) yoki Vite/Nuxt TS'ni oddiy JS'ga aylantiradi. Tiplar **o'chiriladi** (runtime'da yo'q) — ular faqat yozish va kompilatsiya vaqtidagi tekshiruv uchun. `tsconfig.json` — TS sozlamalari.

30-modul masalalari (20 ta)

TS Playground'da sinab ko'ring. Yechimlar — TypeScript kodi.

- `string`, `number`, `boolean` o'zgaruvchilarni annotatsiya qiling.
- `number[]` massivni annotatsiya qiling.

3. Tiplangan parametr va qaytish qiymatli funksiya yozing.
4. Obyektni inline annotatsiya qiling (`{ ism: string }`).
5. `User` interface yozing (`ism` , `yosh`).
6. `User` ni tip sifatida ishlating.
7. Ixtiyoriy xususiyat (`email?`) qo'shing.
8. Union tip (`string | number`) yozing.
9. Literal union (`"faol" | "nofaol"`) yozing.
10. `type` alias yozing (`type ID = ...`).
11. `void` qaytaradigan funksiya yozing.
12. Obyektlar massivini tiplang (`User[]`).
13. Ixtiyoriy funksiya parametri (`yosh?: number`).
14. Default + tiplangan parametr (`rol: string = "user"`).
15. Generic funksiya yozing (`birinchi<T>`).
16. `enum` yozing (`Ro1 { Admin, User }`) — qisqacha.
17. `readonly` xususiyat yozing.
18. Funksiya o'zgaruvchisini tiplang (tip imzosi).
19. JS funksiyani TS'ga aylantiring (annotatsiya qo'shing).
20. **Murakkab:** kichik modul tiping — `Product` interface (`nom` , `narx`); `narx` bo'yicha filterlovchi tiplangan funksiya; generic yordamchi (`oxirgi<T>`).



► Yechimlar



```
// 1
let ism: string = "Oqil";
let yosh: number = 25;
let faol: boolean = true;

// 2
let sonlar: number[] = [1, 2, 3];

// 3
function qoshish(a: number, b: number): number { return a + b; }

// 4
const user4: { ism: string; yosh: number } = { ism: "Oqil", yosh: 25 };

// 5, 6
interface User {
    ism: string;
    yosh: number;
}
const user: User = { ism: "Oqil", yosh: 25 };

// 7
interface User7 {
    ism: string;
    email?: string; // ixtiyoriy
}
const u7: User7 = { ism: "Ali" }; // email yo'q - OK

// 8
let id: string | number;
id = "abc"; id = 123;

// 9
type Status = "faol" | "nofaol";
let holat: Status = "faol";

// 10
type ID = string | number;
const x: ID = 5;

// 11
function log(xabar: string): void { console.log(xabar); }

// 12
const users: User[] = [{ ism: "A", yosh: 20 }, { ism: "B", yosh: 30 }];

// 13
function salom(ism: string, yosh?: number): string {
    return yosh ? `${ism}, ${yosh}` : ism;
}

// 14
function user14(ism: string, rol: string = "user"): string {
    return `${ism} (${rol})`;
}

// 15
```

```

function birinchi<T>(arr: T[]): T { return arr[0]; }
const s = birinchi(["a", "b"]); // string
const n = birinchi([1, 2]);    // number

// 16
enum Rol { Admin, User, Mehmon }
const rol: Rol = Rol.Admin;

// 17
interface Config {
  readonly apiUrl: string; // o'zgartirib bo'lmaydi
}
const cfg: Config = { apiUrl: "https://api.uz" };
// cfg.apiUrl = "..."; // ❌ xato

// 18
type QoshishFn = (a: number, b: number) => number;
const qosh: QoshishFn = (a, b) => a + b;

// 19
// JS: function kvadrat(n) { return n * n; }
// TS:
function kvadrat(n: number): number { return n * n; }

// 20 – modul tiping
interface Product {
  nom: string;
  narx: number;
}

function qimmatlar(mahsulotlar: Product[], minNarx: number): Product[] {
  return mahsulotlar.filter(m => m.narx >= minNarx);
}

function oxirgi<T>(arr: T[]): T {
  return arr[arr.length - 1];
}

const mahsulotlar: Product[] = [
  { nom: "A", narx: 100 },
  { nom: "B", narx: 50 },
  { nom: "C", narx: 200 },
];
console.log(qimmatlar(mahsulotlar, 100)); // A, C
console.log(oxirgi(mahsulotlar).nom);    // "C"

```



QO'LLANMA YAKUNI — 0 dan Expertgacha

Tabriklayman! Sen 30 modulli, ~600 masalali JavaScript yo'lini bosib o'tding. Mana nimalarni o'rganding:

1-QISM (Asoslar): o'zgaruvchilar, operatorlar, shartlar, sikllar **2-QISM (Strukturalar):** funksiyalar, massivlar (`map / filter / reduce`), obyektlar, string/number metodlari **3-QISM (Brauzer):** DOM, events, forms, localStorage — *to'liq Todo-app* **4-QISM (Async):** Event Loop, callbacks, Promises, async/await, Fetch **5-QISM (Chuqur JS):** scope/closures, `this`, prototypes, classes, ES6+, modullar, error handling, regex **6-QISM (Expert):** FP, generators, Proxy/reaktivlik, performance, design patterns, build tools, TypeScript

Endi nima qilish kerak?

Bilim — **yozib** mustahkamlanadi, o'qib emas. Tavsiyalar:

1. **Loyiha qur.** Eng samarali yo'l. Boshlash uchun: Todo-app (qildik) → ob-havo ilovasi (Fetch) → blog (CRUD + localStorage) → kichik o'yin.
2. **Framework'ga o't.** Sof JS asosini o'zlashtirgan bo'lsang, **Vue 3** tabiiy keyingi qadam (reaktivlik — 26-modulda ko'rgan Proxy!). Nuxt esa to'liq ilova uchun.
3. **TypeScript'ni jiddiy o'rgan.** 30-modul — faqat ko'prik. Real loyihada TS — standart.
4. **Algoritm'larni mashq qil.** LeetCode/Codewars — `map / filter / reduce` va Big-O intuitsiyangni o'tkirlaydi.
5. **Boshqalarga o'rgat.** Bironta mavzuni junior'ga tushuntira olsang — uni haqiqatan tushunding.

So'nggi maslahat

Yaxshi dasturchi hamma narsani yodlamaydi — **qaerda qarashni** va **qanday fikrlashni** biladi. Bu qo'llanma senga JS qanday **ishlashini** o'rgatdi. Sintaksisni unutib qo'ysang — qaytib qara. Lekin **tushunchani** (closure, `this`, async, reaktivlik) o'zlashtirsang, u qoladi.

JS — internetning tili. Endi sen u bilan gaplasha olasan. Omad! 🚀